

AT Command Specification

Microchip Technology Inc. ©2023-2026

Revision a59011f4b, October 23, 2026: Network Controller 3.3.0 (a59011f4b)

Table of Contents

Introduction . . . 1
- Product Support . . . 1
- Definitions . . . 1
- Asynchronous Event Codes . . . 2
- Commands and Responses . . . 3
- Numeric Mode . . . 4

Overview . . . 5
- Parameter Types . . . 5
  - Integers . . . 5
  - Fractional Integers: . . . 5
  - Strings . . . 6
  - Raw Binary . . . 7
  - Raw Binary - From DCE to DTE . . . 7
  - Raw Binary - From DTE to DCE . . . 7
- Command Input . . . 8
  - Basic Commands: . . . 8
  - Extended Commands: . . . 8
  - Output . . . 8
- Configuration Commands . . . 9
  - Single Value Parameters: . . . 9
  - Multiple Value Parameters: . . . 9
  - Query Response Elements: . . . 11
  - Complex Value Elements: . . . 11
  - Default Configurations and Presets: . . . 12
- Security Model . . . 13
  - Profiles . . . 13
  - Privilege Level . . . 13
  - Security Descriptor . . . 13

Status Response Codes . . . 14

Serial Interface . . . 20
- Basic Commands . . . 20
  - E . . . 20
  - F . . . 21
  - M . . . 23
  - V . . . 24
  - &K . . . 25

SYSTEM (Module ID = 1) . . . 26
- AEC Reference: . . . 26

+BOOT . . . 26
INTERNAL (Module ID = 2) . . . 27
Command Reference: . . . 27
+GMI . . . 27
+GMM . . . 28
+GMR . . . 29
+IPR . . . 30
CFG (Module ID = 3) . . . 31
Command Reference: . . . 31
+CFG . . . 31
+CFGCP . . . 34
Examples: . . . 38
DHCP (Module ID = 4) . . . 40
Introduction . . . 40
Address Pool . . . 40
Command Reference: . . . 41
+DHCPSC . . . 41
DNS (Module ID = 5) . . . 43
Introduction: . . . 43
Server Address Configuration . . . 43
Resource Queries . . . 43
Reverse Address Queries . . . 43
Connection Types . . . 44
DNSSEC Support . . . 45
Cache Control . . . 45
MDNS Support . . . 45
Command Reference: . . . 46
+DNSC . . . 46
+DNSRESOLV . . . 49
AEC Reference: . . . 50
+DNSRESOLV . . . 50
+DNSERR . . . 53
Examples: . . . 54
FS (Module ID = 7) . . . 58
Command Reference: . . . 58
+FS . . . 58
+FSTSFR . . . 62
AEC Reference: . . . 64
+FSUP . . . 64
Examples: . . . 65
MQTT (Module ID = 8) . . . 69

Introduction . . . 69
Key Features . . . 69
Connecting to a Broker . . . 70
Publishing Messages . . . 71
Publish Completion Events . . . 74
Subscribing and Receiving Messages . . . 74
Maximum Packet Size (MQTT v5.0) . . . 75
MQTT v5.0 Properties . . . 77
Typical Usage Flow . . . 78
Troubleshooting . . . 79
Command Reference: . . . 80
+MQTTC . . . 80
+MQTTCONN . . . 82
+MQTTSUB . . . 83
+MQTTSUBLST . . . 84
+MQTTSUBRD . . . 85
+MQTTUNSUB . . . 87
+MQTTPUB . . . 88
+MQTTLWT . . . 90
+MQTTDISCONN . . . 91
+MQTTPROPTX . . . 92
+MQTTPROPRX . . . 95
+MQTTPROPTXS . . . 98
AEC Reference: . . . 100
+MQTTCONN . . . 100
+MQTTCONNACK . . . 101
+MQTTPUBACK . . . 103
+MQTTPUBCOMP . . . 104
+MQTTPUBERR . . . 105
+MQTTSUB . . . 106
+MQTTUNSUB . . . 107
+MQTTSUBRX . . . 108
+MQTTPROPRX . . . 109
+MQTTPUBSENT . . . 110
+MQTTSUBRXERR . . . 111
Examples: . . . 112
NETIF (Module ID = 9) . . . 115
Introduction . . . 115
IPv4 . . . 115
IPv6 . . . 115
Layer 2 Access . . . 115

Command Reference: 116
+NETIFC 116
+NETIFTX 119
AEC Reference: 120
+NETIFRX 120
OTA (Module ID = 10) 121
Introduction 121
OTA Sequence 121
Timeout 123
Error Handling 123
Troubleshooting 123
Internal HTTP/S OTA Example 124
NVM OTA Example 125
Command Reference: 126
+OTAC 126
+OTADL 128
+OTAVFY 129
+OTAACT 130
+OTAINV 131
AEC Reference: 132
+OTA 132
+OTAERR 133
Examples: 134
PING (Module ID = 11) 136
Introduction 136
Command Reference: 137
+PING 137
AEC Reference: 138
+PING 138
+PINGERR 139
RST (Module ID = 12) 140
Introduction 140
Command Reference: 141
+RST 141
SNTP (Module ID = 13) 142
Introduction 142
Command Reference: 143
+SNTPC 143
AEC Reference: 145
+SNTPERR 145
SOCKET (Module ID = 14) 146

Introduction: . . . 146
Socket Lifecycle . . . 146
Reading Socket Data . . . 149
Command Reference: . . . 152
+SOCKO . . . 152
+SOCKBL . . . 153
+SOCKBR . . . 154
+SOCKBM . . . 155
+SOCKTLS . . . 156
+SOCKWR . . . 157
+SOCKWRTO . . . 158
+SOCKRD . . . 160
+SOCKRDBUF . . . 162
+SOCKCL . . . 164
+SOCKLST . . . 165
+SOCKC . . . 166
AEC Reference: . . . 168
+SOCKIND . . . 168
+SOCKRXT . . . 169
+SOCKRXU . . . 170
+SOCKCL . . . 171
+SOCKTLS . . . 172
+SOCKERR . . . 173
Examples: . . . 174
TIME (Module ID = 16) . . . 177
Introduction . . . 177
Command Reference: . . . 178
+TIME . . . 178
AEC Reference: . . . 180
+TIME . . . 180
TLS (Module ID = 17) . . . 181
Introduction . . . 181
Quick Start: TLS Client . . . 181
Troubleshooting . . . 182
TLS configuration . . . 183
TLS Cipher Suite codes . . . 186
TLS Error Codes . . . 187
Command Reference: . . . 193
+TLSC . . . 193
+TLSCSC . . . 198
AEC Reference: . . . 200

+TLSERR . . . 200
Examples: . . . 201
WAP (Module ID = 18) . . . 202
Introduction . . . 202
Command Reference: . . . 203
+WAPC . . . 203
+WAP . . . 205
AEC Reference: . . . 206
+WAPAIP . . . 206
+WAPSC . . . 207
+WAPSD . . . 208
+WAPERR . . . 209
WSCN (Module ID = 19) . . . 210
Introduction . . . 210
Periodic Scanning . . . 210
Command Reference: . . . 211
+WSCNC . . . 211
+WSCN . . . 215
+WSCNPC . . . 216
+WSCNP . . . 220
AEC Reference: . . . 221
+WSCNIND . . . 221
+WSCNDONE . . . 222
+WSCNPDONE . . . 223
+WSCNPERR . . . 224
Examples: . . . 225
WSTA (Module ID = 20) . . . 229
Introduction . . . 229
Quick Start: Connecting to an Access Point . . . 229
Configuring the STA . . . 230
Controlling STA connection . . . 231
STA information . . . 231
WPA Enterprise . . . 232
Command Reference: . . . 238
+WSTAC . . . 238
+WSTA . . . 241
AEC Reference: . . . 242
+WSTAAIP . . . 242
+WSTALD . . . 243
+WSTAERR . . . 244
+WSTALU . . . 245

+WSTAROAM . . . 246
Examples: . . . 247
ASSOC (Module ID = 22) . . . 251
Introduction . . . 251
Command Reference: . . . 252
+ASSOC . . . 252
AEC Reference: . . . 253
+ASSOC . . . 253
SI (Module ID = 23) . . . 254
Introduction . . . 254
Command Reference: . . . 255
+SI . . . 255
AEC Reference: . . . 256
+SIMSTAT . . . 256
+SIMERR . . . 257
+SIWPKTS . . . 258
+SIHEAP . . . 259
+SIHERR . . . 260
+SISTACK . . . 261
WPROV (Module ID = 25) . . . 262
Introduction . . . 262
Command Reference: . . . 263
+WPROVC . . . 263
+WPROV . . . 265
AEC Reference: . . . 266
+WPROVAT . . . 266
+WPROVDT . . . 267
DI (Module ID = 26) . . . 268
Introduction: . . . 268
Firmware image sequence numbers . . . 268
Command Reference: . . . 269
+DI . . . 269
Examples: . . . 271
EXTCRYPTO (Module ID = 27) . . . 272
Introduction . . . 272
Using External Crypto for Secure Signing . . . 272
Command Reference: . . . 273
+EXTCRYPTO . . . 273
AEC Reference: . . . 274
+EXTCRYPTO . . . 274
Examples: . . . 275

WIFI (Module ID = 28) . . . 276
Introduction . . . 276
Transmitting vendor-specific IEs . . . 276
Receiving vendor-specific IEs . . . 277
Command Reference: . . . 278
+WIFIC . . . 278
AEC Reference: . . . 282
+WIFIVSIERX . . . 282
Examples: . . . 283
NVM (Module ID = 29) . . . 285
Introduction . . . 285
Command Reference: . . . 286
+NVMC . . . 286
+NVMER . . . 288
+NVMWR . . . 289
+NVMCHK . . . 290
+NVMRD . . . 291
AEC Reference: . . . 292
+NVMER . . . 292
+NVMCHK . . . 293
+NVMERR . . . 294
Examples: . . . 295
DFU (Module ID = 30) . . . 296
Introduction . . . 296
Determine the NVM partition in which to program the firmware image . . . 296
Prepare the firmware image . . . 297
Boot the device in DFU mode . . . 298
Erase and program the NVM partition . . . 298
Reset the device . . . 298
Command Reference: . . . 299
+DFUADR . . . 299
+DFUSEQ . . . 300
Examples: . . . 301
PPS (Module ID = 31) . . . 302
Introduction . . . 302
PPS Operation Details . . . 302
EXtreme Deep Sleep (XDS) . . . 307
Summary . . . 307
Command Reference: . . . 308
+PPSC . . . 308
+PPS . . . 310

AEC Reference: 311
+PPS 311
Examples: 312
SYSLOG (Module ID = 32) 313
Introduction 313
Command Reference: 314
+SYSLOGC 314
AEC Reference: 316
+SYSLOG 316
ARB (Module ID = 33) 317
Introduction 317
Command Reference: 318
+ARB 318
Examples: 319
HTTP (Module ID = 34) 320
Introduction 320
Key Features 320
How It Works 321
Advanced Features 322
Error Handling 323
Command Reference: 324
+HTTPC 324
+HTTPGET 327
+HTTPRD 328
+HTTPCL 329
AEC Reference: 330
+HTTPRSPCODE 330
+HTTPRSPFIELD 331
+HTTPBODYRX 332
+HTTPBODYEND 333
+HTTPCL 334
+HTTPERR 335
Examples: 336
WBCN (Module ID = 35) 339
Introduction 339
Configuring the beacon 339
Transmitting beacons 340
Detecting beacons 340
Examples 341
Command Reference: 349
+WBCNC 349

+WBCN . . . 352
AEC Reference: . . . 353
+WBCNDONE . . . 353
+WBCNERR . . . 354
Examples: . . . 355
List of Examples . . . 359
Index . . . 362

Introduction

This reference manual provides information on the commands and features for Microchip products that utilize the Wi-Fi radio module command set. The Wi-Fi radio module is a complete, stand-alone embedded wireless LAN access device. The device has an on-board TCP/IP and TLS stack, and in the simplest hardware configuration, requires only four pins: Power, TX, RX, and Ground. Once the initial configuration has been performed, the device may access a Wi-Fi network and send/receive serial data.

Product Support

This reference manual refers to the Wi-Fi radio device:

Name	Version	Revision
RNWF02	3.3.0	a59011f4b

Definitions

In this document, the following naming conventions are used:

- DCE (Data Communications Equipment): the Wi-Fi module.
- DTE (Data Terminal Equipment): the terminal that issues commands to the Wi-Fi module.

The DCE interface operates in one of two modes:

- Terminal mode: the DCE waits for AT commands and interprets all received characters as commands to execute. All commands must be terminated with a literal <CR><LF> and these characters must not appear anywhere else in the command/parameter input stream. Parameters may be exchanged in one of two forms to accommodate binary data or infrequently-used ASCII control characters.
- Raw mode: the DTE transfers and accepts data "as is", i.e. in binary format with no post- or preprocessing.

Asynchronous Event Codes

In most cases any output from the DCE will be in response to the receipt of a command from the DTE. There are cases, however, where an asynchronous event occurs that the DTE will need informing of (e.g. disconnection from the network). The DCE uses Asynchronous Event Codes (AEC) to convey such events; AECs use the same format conventions as regular responses with an initial <CR> to clearly identify the start of the AEC:

```
<CR>+AECNAME:INFO<CR><LF>
```

To avoid confusion at the DTE, AECs will not be sent during the execution of a command. However the DTE must be prepared to receive AECs at any time as it is possible for the DCE to send AECs resulting from a previous command or condition while the DTE is transmitting a new command.

 ©2023-2026

Commands and Responses

Command execution starts when the command line has been completed (<CR><LF>) and ends when the result code for the command is sent consisting of either a success or error response.

The formatting of these responses depends on the current verbosity level set by the ATV command. All responses consist of the format <RESPONSE><CR><LF>.

Level	Success Response Format	Error Response Format
0	0	1
1	0	1:<STATUS_CODE>
2	OK	ERROR
3	OK	ERROR:<STATUS_CODE>
4	OK	ERROR:<STATUS_MSG> ERROR:<STATUS_CODE> (if no message defined)
5	OK	ERROR:<STATUS_CODE>[,<STATUS_MSG>]

Field	Type	Description
<STATUS_CODE>	Integer	Numeric status code, see Status Response Codes
<STATUS_MSG>	String	Descriptive text detailing the error

If a command requires longer to process the request, then the success response will be used to indicate that the command was accepted. Command processing continues asynchronously, further responses are indicated via AECs.

If an error occurs during asynchronous processing of a command the response will either be a command specific AEC or a generic error AEC of the format:

```
<CR>+CMDNAME:ERRORRSP<CR><LF>
```

Where CMDNAME is the name of the command issued and encountering the error and ERRORRSP is an error response formatted as detailed in the table above. For example:

```
<CR>+SOCKBR:ERROR:4<CR><LF>
```

Numeric Mode

To simplify command/response interactions with automated systems where it may be more problematic to generate and parse string content, the DCE supports a numeric mode. In this mode commands and responses can be represented in number strings of fixed format.

Each command, response and AEC has a two part ID: a module ID and a command/AEC ID. These two numbers are combined in the form:

```
MM.NN
```

Where MM is the hexadecimal representation of the module ID and NN is the hexadecimal representation of the command/AEC ID.

Commands can thus be issued as:

```
AT+MM.NN=
```

Responses and AECs will be formatted as:

```
+MM.NN:
```

While in numeric mode the DCE will still accept full command name strings as well as numeric format, but all responses and AECs will be in just numeric mode.

Numeric mode does not apply to the success or error responses returned by the DCE. If the DTE also wishes to use simplified formatting for these responses the command ATV can be used with either a parameter of 0 or 1 to replace 'OK' and 'ERROR' with '0' and '1'

 ©2023-2026

Overview

Parameter Types

The definition of each command specifies the data types used for any associated parameters. The data types are as follows:

- Integer
- String
- Binary - used only in commands for sending data and receiving data where the DTE either specifies a length or puts the DCE into raw mode.

Integers

Integers are used for numeric parameter values and may be specified in:

base 2 (binary)	Format is '0b' followed by either characters '0' or '1'
base 8 (octal)	Format is '0o' followed by characters '0' through '7'
base 10 (decimal)	Format is an optional '-' character followed by characters '0' through '9'
base 16 (hexadecimal)	Format is '0x' followed by characters '0' through '9' and 'A' through 'F' (case insensitive)

Examples:

99	A value of ninety-nine.
-128	A value of negative one hundred and twenty-eight.
0b10110011	A value of 179.
0o6712	A value of 3530.
0xABCD	A value of 43981.

Positive integers are limited to 0 to 4,294,967,295 (i.e. 32 bits) Negative integers are limited to -1 to -2,147,483,648.

Fractional Integers:

Some parameters and responses support fractional integers, these are base 10 (decimal) only. They are composed of two integers with a period '.' separator, the second integer must be positive.

Examples:

1.1

5.2

-10.3

Fractional integers are limited to -16384.0 to +16383.9999.

Strings

Strings may be given in one of two formats.

- ASCII with the facility for special characters (e.g. tab, carriage return, line feed) to be specified via escape sequences. ASCII strings must be enclosed in double quotes (").
- Hexadecimal string, a stream of two-digit hexadecimal values. Hexadecimal strings must be enclosed within square brackets ([ ]). Please note that each 'byte' being transferred must be specified using two digits - for instance the byte value '2' would be represented '02'.

In the case of an ASCII string, the following characters will be escaped:

Null	\0
Horizontal Tab	\t
Backslash	\\
Double Quotes	\"
Carriage Return	\r
Linefeed	\n
Terminal Bell	\a
Backspace	\b
Vertical Tab	\v
Form feed	\f
Escape	\e

Examples:

"hello" ASCII string representation of hello

"\r\nNew line" ASCII string representation of <CR><LF>New line

[68656c6c6f] Hex string representation of hello

An empty string will be represented as either an empty ASCII string ("") or an empty hexadecimal string ([]). Both representations are valid and equivalent for input, for output only the ([]) form will be used.

 ©2023-2026

Raw Binary

Raw binary can be useful for transferring data that is either transmitted or received over the network. Only a limited number of commands support this format. Binary transfers are achieved by the DCE leaving terminal mode and remaining in raw mode until one of the following conditions is met:

1. A pre-specified number of bytes has been exchanged between DTE and DCE
2. An 'escape sequence' of three consecutive '+' characters has been sent from the DTE to the DCE

NOTE	The 'escape sequence' method for returning to terminal mode relies not only on receipt of the '+++' sequence, but also the absence of any further input for a period of time. By default this is 1 second.

Raw Binary - From DCE to DTE

For retrieval of data in binary mode the DTE must either specify the number of bytes it wishes to receive or specify zero to request a persistent exit from terminal mode. On leaving terminal mode, the DCE will output a single '#' character, it will then transmit data in raw binary format.

The DCE will return to terminal mode if any of the following occurs:

1. The +++ escape sequence is received from the DTE
2. The number of bytes requested by the DTE has been sent
3. All available data has been sent to the DTE (e.g. DTE asks for 10 bytes but only 5 are available for reading)

On returning to terminal mode, the DCE will return the result of the command which initiated the transfer.

Raw Binary - From DTE to DCE

For transmission of data in binary mode the DTE must either specify the number of bytes it wishes to send to the DCE or specify zero to request a persistent exit from terminal mode. On leaving terminal mode, the DCE will accept binary format data until such time as:

1. The +++ escape sequence is received from the DTE
2. The number of bytes requested by the DTE has been sent

While in raw binary mode, the DCE forwards the received data to its destination (generally an associated socket) as it is received. When a length was specified, the data is sent once that number of bytes has been received.

Command Input

Commands consist of the following:

- Must start with 'AT'
- Zero or more basic commands
- Zero or one extended commands
- Terminate with <CR><LF>

Basic Commands:

Basic commands are single alphabetic characters which may be preceded by a single '&' character. An optional single numeric argument may then follow.

Extended Commands:

Extended commands begin with a single '+' character followed by a command name. If the command requires parameters, the command name is followed by a single '=' character with each parameter separated from the next by a comma. Parameters must follow the formatting rules for Integers or Strings.

Examples:

ATL<CR><LF>	Basic command 'L'
ATJ0<CR><LF>	Basic command 'J' with argument '0'
ATLJ0<CR><LF>	Basic command 'L' and 'J' with argument '0'
AT+CMD<CR><LF>	Extended command 'CMD'
AT+CMD=0<CR><LF>	Extended command 'CMD' with single parameter '0'
AT+CMD=0,1<CR><LF>	Extended command 'CMD' with two parameters '0' and '1'
ATJ0+CMD=0,1<CR><LF>	Combination of basic and extended commands

Output

Before transmitting token/parameter data to the DTE, the DCE will inspect each element to determine if any of the bytes would render inconsistently on a terminal display. In such cases, the element will be returned in hexadecimal format. This allows, for example, UTF-8 formatted SSIDs to be sent to a DTE in a fashion suitable for re-use, i.e. a human using a terminal application can copy this data and use it as a parameter in a command. Where multiple pieces of information are being returned, the DCE will comma separate each token.

Examples:

```
A<CR><LF>B"C\D<TAB>E            "A\r\nB\"C\\D\tE"
A©B                             [41C2A942]
```

 ©2023-2026

Configuration Commands

Certain commands are used to configure or query parts of the DCE. These commands allow the DTE to set new parameter values and get existing parameter values. The commands can query all parameter values, query a single parameter ID, or set a single parameter ID.

Each configuration parameter will consist of an ID and a value.

Commands can have preceding arguments which are specific to that particular command, but the ID and value arguments will be the last arguments.

For example:

```
AT+SOMECFGCMD=<CMD_SPEC_ARG>                - Get all parameters
AT+SOMECFGCMD=<CMD_SPEC_ARG>,<ID>           - Get single parameter
AT+SOMECFGCMD=<CMD_SPEC_ARG>,<ID>,<VALUE>   - Set single parameter
```

Single Value Parameters:

These parameters may have only one value and are accessed using the parameter ID.

Multiple Value Parameters:

These parameters may have more than one value.

When reading all values, the parameter ID can be used, however when accessing a single value within the set of all values the command will use a fractional integer form of the ID consisting of <ID>.<INDEX>

Example Command Sequence 1: Reading all parameter values

→ `AT+CMD=5`

← `+CMD:5.0,"value1"`

← `+CMD:5.1,"value2"`

← `+CMD:5.2,"value3"`

← `OK`

Example Command Sequence 2: To read a single value

→ `AT+CMD=5.1`

← `+CMD:5.1,"value2"`

← `OK`

When writing a value, the behavior will depend on the implementation of the command, however often writing a value to the parameter <ID> will cause an additional value to be appended to the set of parameter values, writing to the fractional <ID>.<INDEX> will cause that parameter index to be updated.

Example Command Sequence 3: To set an additional value

```
→    AT+CMD=5
←    +CMD:5.0,"value1"
←    OK
→    AT+CMD=5,"newvalue"
←    OK
→    AT+CMD=5
←    +CMD:5.0,"value1"
←    +CMD:5.1,"newvalue"
←    OK
```

Example Command Sequence 4: To replace an existing value

```
→    AT+CMD=5
←    +CMD:5.0,"value1"
←    +CMD:5.1,"value2"
←    OK
→    AT+CMD=5.0,"newvalue"
←    OK
→    AT+CMD=5
←    +CMD:5.0,"newvalue"
←    +CMD:5.1,"value2"
←    OK
```

In addition to the fractional integer form a command response may include a range indication primarily to indicate how many valid responses will follow. This optional response will only be present if the parameter implements it and the request was to read all values, it will not be present if a request for a single value was made using a fractional integer ID.

The format of the range indication uses the non-fractional ID of the parameter and a single integer value:

This example indicates that +CMD parameter ID 5 will return 2 values, 5.0 and 5.1:

Example Command Sequence 5: Range of IDs

```
→    AT+CMD=5
←    +CMD:5,2
←    +CMD:5.0,"value1"
←    +CMD:5.1,"value2"
←    OK
```

 ©2023-2026

Query Response Elements:

Standard configuration parameter queries return a simple <ID>,<VALUE> pair. For single value parameters the <ID> will be an integer, for multiple value parameters the <ID> will be a fractional integer in the form <ID>.0, <ID>.1 etc for each value.

Example Command Sequence 6: Simple ID/Value

```
→   AT+CMD=5
←   +CMD:5.0,"simple1"
←   +CMD:5.1,"simple2"
←   OK
```

Complex Value Elements:

When queried some configuration parameters will return a more complex value in place of the <VALUE> element of the response. A complex value is more than one comma separated value.

Example Command Sequence 7: Complex ID/Value

```
→   AT+CMD=5
←   +CMD:5.0,"complex11","complex12","complex12"
←   +CMD:5.1,"complex21","complex22","complex22"
←   OK
```

Default Configurations and Presets:

Configuration commands are initially loaded with default values, also known as **preset 0**. Many configuration commands also support reloading preset values.

The reserved <ID> value 0 is used to query and load configuration command presets. Querying this <ID> will return the current base preset used to load the configuration commands values. Setting this <ID>'s value will load a preset set of values.

NOTE	Currently only **preset 0** is defined to be the default system values. Other presets may be defined in future.

NOTE	The queried preset indicates the preset last used to load the values in the configuration command, subsequent changes may have occurred to individual values. Therefore it does not indicate that any particular value is currently set to that preset value, only that it initially was.

Example Command Sequence 8: Using preset 0

→	`AT+CMD`	Query all values
←	`+CMD:1,1234`	<ID> 1 default values is 1234
←	`+CMD:2,"A String"`	
←	`OK`	
→	`AT+CMD=1,5678`	Set <ID> 1 to be 5678
←	`OK`	
→	`AT+CMD`	Query all values
←	`+CMD:1,5678`	<ID> 1 now changed to 5678
←	`+CMD:2,"A String"`	
←	`OK`	
→	`AT+CMD=0`	Query current base preset
←	`+CMD:0,0`	Current preset is 0, default values
←	`OK`	
→	`AT+CMD=0,0`	Load preset 0 in to command values
←	`OK`	
→	`AT+CMD`	Query all values
←	`+CMD:1,1234`	<ID> 1 returned to default value
←	`+CMD:2,"A String"`	
←	`OK`	

 ©2023-2026

Security Model

Security is based around a combination of privilege levels and profiles. Various elements in the system are assigned a security descriptor which describes the minimum privilege level required to access the element for a given profile.

Profiles

Up to four profiles can be defined in the system. Each profile defines a set of functionality which is accessible for a given access path.

For example, one profile may be assigned to all accesses via a local physical interface (UART, I2C, SPI etc) while a second profile is assigned to all accesses via a remote interface (Sockets). This allows different privilege requirements to be specified depending on how the system is accessed, locally vs remotely.

Privilege Level

There are four defined privilege levels:

- Guest
- User
- Super-user
- Root

Guest having the lowest privilege while Root has the highest. Any element can be accessed by a user with a privilege level of 'at least' what is specified in its security descriptor for the current access profile.

Security Descriptor

Elements in the system are assigned a security descriptor, this consists of a minimum user privilege level for each of the four profiles. In this documentation these descriptors are shown as four characters, each character can be either `G`(uest), `U`(ser), `S`(uper-user) or `R`(oot).

For example, the descriptor `GSRG` means:

- Profile 1 - Guest
- Profile 2 - Super-user
- Profile 3 - Root
- Profile 4 - Guest

Status Response Codes

CORE Status Codes (Module ID = 0)

Value	Name	Description
0,0	OK	OK
0,1	ERROR	General Error
0,2	INVALID_CMD	Invalid AT Command
0,3	UNKNOWN_CMD	Unknown AT Command
0,4	INVALID_PARAMETER	Invalid Parameter
0,5	INCORRECT_NUM_PARAMS	Incorrect Number of Parameters
0,6	STORE_UPDATE_BLOCKED	Configuration Update Blocked
0,7	STORE_ACCESS_FAILED	Configuration Access Failed
0,8	TIMEOUT	Command Timed Out
0,9	HOST_INTERFACE_FAILED	Host Interface Failed
0,10	ACCESS_DENIED	Access Denied
0,11	CONFIG_CONFLICT	Configuration Conflict

SYSTEM Status Codes (Module ID = 1)

Value	Name	Description
1,0	WIFI_API_REQUEST_FAILED	Wi-Fi Request Failed
1,1	STA_NOT_CONNECTED	STA Not Connected
1,2	NETWORK_ERROR	Network Error
1,3	FILE_SYSTEM_ERROR	File System Error

CFG Status Codes (Module ID = 3)

Value	Name	Description
3,0	CFG_NOT_PRESENT	Configuration not present

 ©2023-2026

DNS Status Codes (Module ID = 5)

Value	Name	Description
5,0	DNS_TYPE_NOT_SUPPORTED	DNS Type Not Supported
5,1	DNS_TIMEOUT	DNS Query Timeout
5,2	DNS_ERROR	DNS Error
5,3	DNS_RECORD_NOT_FOUND	DNS Record Not Found
5,4	DNS_NON_EXISTENT_DOMAIN	DNS Non-Existent Domain
5,5	DNS_AUTH_DATA_ERROR	DNS Data Authentication Error
5,6	DNS_TRUNCATED_RESPONSE	DNS Response Truncated

FS Status Codes (Module ID = 7)

Value	Name	Description
7,0	TSFR_PROTOCOL_NOT_SUPPORTED	Unsupported File Transfer Protocol
7,1	FILE_EXISTS	File Exists
7,2	FILE_NOT_FOUND	File Not Found
7,3	INVALID_FILE_TYPE	Invalid File Type
7,4	FILE_CREATE_FAILED	File create failed
7,5	FILE_WRITE_FAILED	File write failed

MQTT Status Codes (Module ID = 8)

Value	Name	Description
8,0	MQTT_ERROR	MQTT Error

NETIF Status Codes (Module ID = 9)

Value	Name	Description
9,0	NETWORK_INTF_DOWN	Network interface down

OTA Status Codes (Module ID = 10)

Value	Name	Description
10,0	ERASE_DONE	Erase Done
10,1	WRITE_DONE	Write Done
10,2	VERIFY_DONE	Verify Done
10,3	ACTIVATE_DONE	Activate Done
10,4	INVALIDATE_DONE	Invalidate Done
10,5	OTA_ERROR	OTA Error
10,6	OTA_NO_STA_CONN	No STA Connection
10,7	OTA_PROTOCOL_ERROR	Protocol Error
10,8	OTA_TLS_ERROR	TLS Error
10,9	OTA_IMG_TOO_LARGE	Image Too Large
10,10	OTA_TIMEOUT	Timeout
10,11	OTA_VERIFY_FAILED	Image Verify Failed

PING Status Codes (Module ID = 11)

Value	Name	Description
11,0	PING_FAILED	Ping Failed

SNTP Status Codes (Module ID = 13)

Value	Name	Description
13,0	SNTP_SERVER_TMO	NTP Server Timeout
13,1	SNTP_PROTOCOL_ERROR	NTP Protocol Error

 ©2023-2026

SOCKET Status Codes (Module ID = 14)

Value	Name	Description
14,0	SOCKET_ID_NOT_FOUND	Socket ID Not Found
14,1	LENGTH_MISMATCH	Length Mismatch
14,2	NO_FREE_SOCKETS	No Free Sockets
14,3	SOCKET_INVALID_PROTOCOL	Invalid Socket Protocol
14,4	SOCKET_CLOSE_FAILED	Socket Close Failed
14,5	SOCKET_BIND_FAILED	Socket Bind Failed
14,6	SOCKET_TLS_FAILED	Socket TLS Failed
14,7	SOCKET_CONNECT_FAILED	Socket Connect Failed
14,8	SOCKET_SEND_FAILED	Socket Send Failed
14,9	SOCKET_SET_OPT_FAILED	Socket Set Option Failed
14,10	SOCKET_REMOTE_NOT_SET	Socket Destination Not Set
14,11	MULTICAST_ERROR	Multicast Error
14,12	SOCKET_NOT_READY	Socket Not Ready
14,13	SOCKET_SEQUENCE_ERROR	Socket Sequence Error

TIME Status Codes (Module ID = 16)

Value	Name	Description
16,0	TIME_ERROR	Time Error

TLS Status Codes (Module ID = 17)

Value	Name	Description
17,0	TLS_CA_CERT_MISSING	CA Cert Missing
17,1	TLS_CA_CERT_VALIDATION	CA Cert Validation
17,2	TLS_CA_CERT_DATE_VALIDATION	CA Cert Date Validation
17,3	TLS_KEY_PAIR_INCOMPLETE	Key Pair Incomplete
17,4	TLS_PEER_DOMAIN_MISSING	Peer Domain Missing

WAP Status Codes (Module ID = 18)

Value	Name	Description
18,0	WAP_STOP_REFUSED	Soft AP Stop Not Permitted
18,1	WAP_STOP_FAILED	Soft AP Stop Failed
18,2	WAP_START_REFUSED	Soft AP Start Not Permitted
18,3	WAP_START_FAILED	Soft AP Start Failed
18,4	UNSUPPORTED_SEC_TYPE	Unsupported Security Type

WSCN Status Codes (Module ID = 19)

Value	Name	Description
19,0	WSCN_SCAN_IN_PROGRESS	Scan In Progress
19,1	WSCN_PERIODIC_SCAN_IS_RUNNING	Periodic Scan is Running
19,2	WSCN_INTERVAL_TOO_SHORT	Scan Interval Too Short
19,3	WSCN_AP_MODE_ACTIVE	Soft AP Active
19,4	WSCN_NO_CHANNELS	No Channels To Scan
19,5	WSCN_STA_TRANSITIONING	STA transition In Progress

WSTA Status Codes (Module ID = 20)

Value	Name	Description
20,0	STA_DISCONN_REFUSED	STA Disconnect Not Permitted
20,1	STA_DISCONN_FAILED	STA Disconnect Failed
20,2	STA_CONN_REFUSED	STA Connection Not Permitted
20,3	STA_CONN_FAILED	STA Connection Failed

ASSOC Status Codes (Module ID = 22)

Value	Name	Description
22,0	ASSOC_NOT_FOUND	Association Not Found

NVM Status Codes (Module ID = 29)

Value	Name	Description
29,0	NVM_LOCKED	NVM Locked Until Reset

DFU Status Codes (Module ID = 30)

Value	Name	Description
30,0	DFU_ADDRESS_WARNING	No Bootable Image In Other Partition

PPS Status Codes (Module ID = 31)

Value	Name	Description
31,0	PPS_WIFI_PS_NOT_ENABLED	Wi-Fi PS Not Enabled
31,1	PPS_TIMEOUT	PPS Timeout
31,2	PPS_PAUSE_EXPIRED	PPS Pause Expired

 ©2023-2026

ARB Status Codes (Module ID = 33)

Value	Name	Description
33,0	ARB_NO_INCREASE	The Value Provided Would Not Increase The ARB
33,1	ARB_REJECTED	The Value Provided Would Invalidate The Current Image

HTTP Status Codes (Module ID = 34)

Value	Name	Description
34,0	HTTP_DNS_FAILED	HTTP DNS failed
34,1	HTTP_TLS_FAILED	HTTP TLS failed
34,2	HTTP_CONNECT_FAILED	HTTP connect failed
34,3	HTTP_MEMORY	HTTP buffer allocation
34,4	HTTP_RESPONSE_HEADERS	HTTP response format
34,5	HTTP_RECV_FAILED	HTTP receive failed
34,6	HTTP_SEND_FAILED	HTTP send failed
34,7	HTTP_TIMEOUT	HTTP timeout

WBCN Status Codes (Module ID = 35)

Value	Name	Description
35,0	WBCN_IN_PROGRESS	Beaconing In Progress
35,1	WBCN_AP_MODE_ACTIVE	Soft AP Active
35,2	WBCN_TX_ERROR	Beacon Tx Error
35,3	WBCN_STA_TRANSITIONING	STA Connection In Progress
35,4	WBCN_NO_CHANNELS	No Beacon Channels Selected
35,5	WBCN_INTERVAL_TOO_SHORT	Beacon Interval Too Short

Serial Interface

Basic Commands

E

Description

This command controls if characters received from the DTE are echoed back to the DTE.

This command can only be set; it does not support a query form.

Command Syntax:

Command	Description
ATE<N>	Set value

Command Parameter Syntax:

Parameter Name	Type	Description
<N>	Integer	Echo control. Default is 1 0 Turn off character echo. 1 Turn on character echo.

 ©2023-2026

F

Description

This command controls numeric, string and data formats.

The formatting control determines how integer, string and byte arrays are displayed.

This value is built up from three values:

Bit Mask	Control Value
0b00000011	String/Byte Array Control
0b00000100	Integer Number Base Support
0b00001000	Default Number Base

The default value for this is 0b00000111 (7).

String/Byte Array Control:

By default, string and byte arrays are displayed using safe-strings where possible or hexadecimal strings, see Strings.

If the data contains only printable ASCII characters it will be displayed as a string.

If the data contains non-printable ASCII characters which could be displayed as escaped sequences it will be displayed as a string with escape sequences.

If the data contains non-printable ASCII characters which can't be displayed as escaped sequences it will be displayed as a hexadecimal string.

Control Value	Description
0	Both string and byte arrays are displayed as hexadecimal strings.
1	Byte arrays are displayed as hexadecimal strings. Strings are displayed as ASCII strings unless they contain non-printable characters, escape characters will not be used. Non-printable characters will result in a hexadecimal string being used.
2	Byte arrays are displayed as hexadecimal strings. Strings are displayed as ASCII strings unless they contain non-printable characters, escape characters will be used if possible, otherwise as a hexadecimal string.
3	Both byte arrays and strings are displayed as strings using escape sequences if possible, otherwise hexadecimal strings.

Integer Number Base Support

The Integer Number Base Support bit determines if number bases other then the default are supported. If this bit is set support is enabled.

Default Number Base

The Default Number Base determines if decimal or hexadecimal numbers are used by default for integers. If this bit is set the default number base will be hexadecimal, otherwise decimal is used.

This command can only be set; it does not support a query form.

Command Syntax:

Command	Description
ATF<N>	Set value

Command Parameter Syntax:

Parameter Name	Type	Description
<N>	Integer	Format control bit field, see description. Default is 7 0 Bits 0-1 = 0: hex strings only. 1 Bits 0-1 = 1: hex bytes, basic strings. 2 Bits 0-1 = 2: hex bytes, escaped strings. 3 Bits 0-1 = 3: escaped strings or hex bytes. 4 Bit 2: number base support. 8 Bit 3: default to hex integers.

 ©2023-2026

M

Description

This command controls the command/response mode.

This command can only be set; it does not support a query form.

Command Syntax:

Command	Description
ATM<N>	Set value

Command Parameter Syntax:

Parameter Name	Type	Description
<N>	Integer	Command mode. Default is 0 0 Verbose string mode. 1 Numeric mode.

V

Description

This command controls the level of verbosity used when displaying command responses. The response to this command will be in the new verbosity format specified.

This command can only be set; it does not support a query form.

Command Syntax:

Command	Description
ATV<N>	Set value

Command Parameter Syntax:

Parameter Name	Type	Description
<N>	Integer	Verbosity level. Default is 5 0 Just 0 or 1. 1 0 for success, 1:<STATUS_CODE> for error. 2 Just OK or ERROR. 3 OK for success, ERROR:<STATUS_CODE> for error. 4 OK for success, either ERROR:<STATUS_MSG> or ERROR:<STATUS_CODE> for error. 5 OK for success, ERROR:<STATUS_CODE>[,<STATUS_MSG>] for error.

 ©2023-2026

&K

Description

This command is used to select the local flow control method.

This command can only be set; it does not support a query form.

Command Syntax:

Command	Description
AT&K<N>	Set value

Command Parameter Syntax:

Parameter Name	Type	Description
<N>	Integer	Flow control. Default is 0 0 All flow control is disabled. 3 RTS/CTS flow control is enabled.

SYSTEM (Module ID = 1)

AEC Reference:

+BOOT

Description

System has booted.

AEC Syntax

AEC	Description
+BOOT:<BANNER>	System boot

AEC Element Syntax

Element Name	Type	Description
<BANNER>	String	Boot banner

 ©2023-2026

INTERNAL (Module ID = 2)

Command Reference:

+GMI

Description

This command requests manufacturer identification.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+GMI	Query the manufacturers ID/name	GGGG

Response Syntax

Response	Description
+GMI:<MAN_ID>	Information Response

Response Element Syntax

Element Name	Type	Description
<MAN_ID>	String	Manufacturers ID/name

+GMM

Description

This command requests model identification.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+GMM	Query the model information	GGGG

Response Syntax

Response	Description
+GMM:<MODEL_ID>	Information Response

Response Element Syntax

Element Name	Type	Description
<MODEL_ID>	String	Model information

 ©2023-2026

+GMR

Description

This command requests revision identification.

Version Information Format

```
"2.0.0 0 ab12cdef1 [12:34:56 Feb 01 2024]"
```

The version information string is constructed from:

- Software version MAJOR.MINOR.PATCH
- Security version
- Software hash identifier
- Build date

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+GMR	Query the revision information	GGGG

Response Syntax

Response	Description
+GMR:<VERSION_INFO>	Information Response

Response Element Syntax

Element Name	Type	Description
<VERSION_INFO>	String	Version information

+IPR

Description

This command sets the DTE serial port baud rate.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+IPR	Query the current baud rate	GGGG
AT+IPR=<BAUD_RATE>	Set the serial baud rate	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<BAUD_RATE>	Integer	Baud rate Positive unsigned 32-bit value

Response Syntax

Response	Description
+IPR:<BAUD_RATE>	Information Response

Response Element Syntax

Element Name	Type	Description
<BAUD_RATE>	Integer	Baud rate Positive unsigned 32-bit value

 ©2023-2026

CFG (Module ID = 3)

Command Reference:

+CFG

Description

This command is used to read or set the system configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+CFG	Query all configuration elements	GGGG
AT+CFG=<ID>	Query a single element	GGGG
AT+CFG=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+CFG:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<DEVICE_NAME>	String	The device name Maximum length of string is 32	GGGG
10	<VERSION>	Fractional Integer (Read Only)	Software version number	GGGG
11	<PATCH>	Integer (Read Only)	Software patch number	GGGG
12	<SECURITY>	Integer (Read Only)	Security value	GGGG
13	<BUILD_HASH>	String (Read Only)	Build ID Maximum length of string is 5	GGGG
14	<BUILD_TIME>	UTC Time (Read Only)	Build time	GGGG
20	<CMD_PORT>	Integer (Read Only)	Command port 1 UART1.	GGGG
21	<CMD_BAUD>	Unsigned Integer	Command baud rate. Default is 230400 See Changing the Command Baud Rate below.	GGGG
22	<CMD_TIMEOUT>	Unsigned Integer	Command baud rate change timeout (ms), 0 disables fallback. Default is 0 See Changing the Command Baud Rate below.	GGGG
50	<ARCHIVE_FILTER>	String Unsigned Integer	Archive filter Maximum length of string is 32 This is a multiple value parameter with an ID range 50.0 to 50.7 The filter is a list of command IDs either in string form (+WSTAC) or integer form (0x1400).	GGGG
51	<ARCHIVE_SLOTS>	UTC Time	Archive slots This is a multiple value parameter with an ID range 51.1 to 51.2	GGGG
52	<AUTOEXEC_STATUS>	Status (Read Only)	Auto-execute status	GGGG
100	<DEBUG_PORT>	Unsigned Integer	Debug port 0 Off. 1 UART1. 2 UART2. See Debug Port below.	GGGG
101	<DEBUG_BAUD>	Unsigned Integer	Debug baud rate	GGGG

 ©2023-2026

Changing the Command Baud Rate

Writing CMD_BAUD changes the baud rate of the command interface. The change takes effect once the command has completed, so the OK response is sent at the original baud rate.

If CMD_TIMEOUT is non-zero, the DTE must send a valid command at the new baud rate within that many milliseconds. Otherwise the DCE returns to the previous baud rate and CMD_BAUD reverts to its previous value. If CMD_TIMEOUT is 0, there is no fallback.

+IPR also changes the baud rate, but without a fallback. Unlike +IPR, CMD_BAUD is a configuration parameter and can be stored using +CFGCP.

Debug Port

DEBUG_PORT cannot be set to UART1 while the command interface is using UART1.

+CFGCP

Description

This command is used to copy configurations to/from storage.

Configurations managed by +CFGCP are groups of configuration command parameters. The command allows these parameters to be saved to either flash files or archive slots in memory for later retrieval.

+CFGCP supports:

- Saving active configurations to flash files
- Saving active configurations to in memory archive slots
- Restoring flash files to active configuration
- Restoring flash files to in memory archive slots
- Restoring in memory archive slots to active configurations
- Duplicating in memory archive slots

NOTE	Only parameter tables of configuration commands which have been changed are stored to flash files or archive slots. If no parameters have been changed, nothing is stored; the command still succeeds but the destination slot or file is not created.

Table 1. Configuration Persistence

Location	After reset
Active configuration (slot 0)	Lost, returns to the defaults (preset 0)
Archive slots 1 and 2, and their timestamps	Lost
Configuration files (file type 20)	Kept
The 'autoexec' configuration file	Kept, and loaded at boot (see Auto-Execute Configuration)

When a configuration is stored:

- The whole parameter table of a configuration command is stored once any of its elements has been set, not only the elements which were changed.
- Loading preset 0 (<ID> 0, see Default Configurations and Presets) marks the table as unchanged again, so it is not stored.
- Only configuration elements are stored; read-only and state elements, such as ARCHIVE_SLOTS and AUTOEXEC_STATUS, are not.
- The configurations of +SOCKC and +SYSLOGC are never stored.
- The parameters of +CFG, including ARCHIVE_FILTER, are stored like any other configuration command.

 ©2023-2026

When a configuration is restored to the active configuration:

- The stored values are applied on top of the current configuration. Tables which are not in the stored configuration keep their current values; to restore a clean configuration, load preset 0 for each command first, or reset the RNWF02.
- The restore stops at the first error, and values already applied are kept.
- A value which is rejected as an invalid parameter is retried, to allow for dependencies between commands; if it is still rejected after 100 ms the restore fails with TIMEOUT.

Active Configuration

The active configuration is considered to occupy slot 0 for the purposes of the source and destination of +CFGCP.

Archive Slots

There are two archive slots, numbered 1 and 2; they represent in memory holding locations for configuration sets. The parameters in these slots are not active and not maintained through reset.

Slots can be queried through ARCHIVE_SLOTS, these parameters contain the timestamp of the configuration set when stored from an active configuration. The timestamp is the UTC time, so it is close to 1970 if the time has not been set (see +TIME). Querying an empty slot fails. A configuration copied from a file to a slot keeps the timestamp of the file.

A slot can be emptied by writing a zero value to the appropriate index within ARCHIVE_SLOTS. Writing a zero value to ARCHIVE_SLOTS without an index empties all slots.

Archive Filters

The ARCHIVE_FILTER contains a list of command IDs which are filtered out when storing or loading a configuration set. This allows a partial configuration to be stored or restored.

- Up to 8 command IDs can be added. Each write adds one ID to the list; adding an ID which is already in the list, or a ninth ID, fails.
- An ID can be given as a number or as the command name, which must match exactly including the leading + (for example `"+WSTAC"`). Queries return the numeric IDs.
- Writing 0 clears the list.
- The filter is applied both when storing and when restoring, using the filter which is active at that time. A command filtered out when the configuration was stored cannot be restored from it.

See example Storing a partial configuration.

Archive Files

Configurations are stored in 'Configuration Files' types which can be queried and deleted through +FS.

Source and Destination

The source and destination may each be a slot number (0 for the active configuration, or an archive slot) or a configuration file name. Copying from one file to another is not supported. Copying to a file that already exists fails; the existing file must first be deleted using +FS. Copying from an empty archive slot, or from a file that does not exist, fails.

Configuration Summary

The response to +CFGCP is a summary list of command IDs with the number of command elements being stored or restored and the total size of the memory used for that command. The command ID is reported in hexadecimal. The summary is only reported when the source or destination is the active configuration.

Auto-Execute Configuration

If a configuration is stored to an archive file called 'autoexec' it is automatically loaded during device boot, once the file system has been mounted. The command interface may already be available when it is loaded, so the host should not rely on the configuration being active as soon as the +BOOT AEC is received.

After +BOOT, query AUTOEXEC_STATUS to check the result:

- CFG_NOT_PRESENT: there is no 'autoexec' file, or it has not been loaded yet.
- Any other status: the status of loading the file; OK if every stored value was applied.

To replace the auto-execute configuration, first delete the 'autoexec' file (file type 20) using +FS, then store the new configuration to it.

 ©2023-2026

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+CFGCP=<CFGSRC>,<CFGDST>	Copy configuration	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<CFGSRC>	Integer String Byte Array	Configuration source: slot number (0 = active) or file name
<CFGDST>	Integer String Byte Array	Configuration destination: slot number (0 = active) or file name

Response Syntax

Response	Description
+CFGCP:<CMD_ID>,<NUM_ITEMS>,<ITEM_LEN>	copy event

Response Element Syntax

Element Name	Type	Description
<CMD_ID>	Integer	Configuration command ID (hexadecimal)
<NUM_ITEMS>	Integer	Number of items
<ITEM_LEN>	Integer	Length of items

Examples:

Example Command Sequence 9: Saving active configuration to storage

```
→   AT+WSTAC=1,"SSID"
←   OK
→   AT+WSTAC=2,3
←   OK
→   AT+WSTAC=3,"Password"
←   OK
→   AT+NETIFC=0,10,1
←   OK
→   AT+DNSC=1,"1.1.1.1"
←   OK
→   AT+DNSC=2,0
←   OK
→   AT+CFGCP=0,"saved"
←   +CFGCP:0x0500,3,49
←   +CFGCP:0x0900,11,182
←   +CFGCP:0x1400,10,136
←   OK
```

Example Command Sequence 10: Restoring configuration from storage

```
→   AT+CFGCP="saved",0
←   +CFGCP:0x0500,3,49
←   +CFGCP:0x0900,11,182
←   +CFGCP:0x1400,10,136
←   OK
```

 ©2023-2026

Example Command Sequence 11: Storing a partial configuration

→	`AT+WSTAC=1,"SSID"`	
←	`OK`	
→	`AT+NETIFC=0,10,1`	
←	`OK`	
→	`AT+DNSC=1,"1.1.1.1"`	
←	`OK`	
→	`AT+CFG=50,"+WSTAC"`	Do not store the Wi-Fi station configuration
←	`OK`	
→	`AT+CFG=50,"+CFG"`	Do not store the system configuration, including this filter
←	`OK`	
→	`AT+CFGCP=0,"netonly"`	Store the remaining configuration to a file
←	`+CFGCP:0x0500,3,49`	
←	`+CFGCP:0x0900,11,182`	
←	`OK`	
→	`AT+CFG=50,0`	Clear the archive filter
←	`OK`	

DHCP (Module ID = 4)

Introduction

The RNWF02 provides a DHCP server for clients connected to its soft AP.

The server is configured with +DHCPSC and is started and stopped together with the soft AP by +WAP, if ENABLED is set. While the server is running, its configuration cannot be changed.

Address Pool

If no pool start address is configured, the pool is placed in the half of the soft AP subnet that does not contain the RNWF02's own address. The pool holds at most 32 leases. The soft AP subnet must be /29 or larger, and the pool start address must not be the network, broadcast or RNWF02 address, otherwise the soft AP fails to start.

When the server is running, reading the configuration reports the active pool start, pool end and number of leases. When it is stopped, the configured pool start is reported.

The gateway and DNS server addresses, if configured, are provided to clients in the DHCP offer.

 ©2023-2026

Command Reference:

+DHCPSC

Description

This command is used to read or set the DHCP server configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+DHCPSC	Query pool list	GGGG
AT+DHCPSC=<IDX>	Query all configuration elements	GGGG
AT+DHCPSC=<IDX>,<ID>	Query a single element	GGGG
AT+DHCPSC=<IDX>,<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<IDX>	Integer	Pool index Value is 0
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+DHCPSC:<MIN_IDX>,<MAX_IDX>	Valid index range
+DHCPSC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<MIN_IDX>	Integer	Lowest valid index
<MAX_IDX>	Integer	Highest valid index
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<ENABLED>	Bool	DHCP server enabled when the soft AP is started 0 Disabled. 1 Enabled.	GGGG
2	<POOL_START>	IPv4 Address	Start address of DHCP server pool (0 = automatic) Unsigned 32-bit value Format of IPv4 address is 'a.b.c.d'	GGGG
3	<POOL_END>	IPv4 Address (Read Only)	End address of the active DHCP server pool (read only, while running) Unsigned 32-bit value Format of IPv4 address is 'a.b.c.d'	GGGG
4	<POOL_LEASES>	Integer (Read Only)	Number of leases in the active pool (read only, while running) Unsigned 16-bit value	GGGG
5	<NETIF_IDX>	Integer	Network interface index Valid range is 0 to 1	GGGG
10	<GATEWAY>	IPv4 Address	The address of the default gateway Unsigned 32-bit value Format of IPv4 address is 'a.b.c.d'	GGGG

 ©2023-2026

DNS (Module ID = 5)

Introduction:

Server Address Configuration

There are three methods available to configure the DNS servers used by the DCE.

1. Manually configured via +DNSC option DNS_SVR.
2. Automatically configured via IPv4 DHCP option 6.
3. Automatically configured via IPv6 Router Advertisement RNDSS.

Switching between manual and automatic configuration is performed by +DNSC option DNS_AUTO.

Resource Queries

The supported query types are listed in +DNSRESOLV parameter TYPE, other types may be specified as well. For unknown type the +DNSRESOLV response will simply include the undecoded RR data.

The DNS query ANY is indirectly supported, it has generally been deprecated as a DNS query type and some servers do not respond to it. If the ANY query is specified the DCE will generate two separate queries for A and AAAA records. When generating the queries for ANY, the DCE will use the configuration of +DNSC option DNS_PROTO_PREF to determine selection and order of A and AAAA queries.

NOTE	DNS_PROTO_PREF specifies the order of queries made, it does not control the order of responses received.

Reverse Address Queries

IP address to name queries can be made using a **PTR** (12) query in the form:

- <IPv4 Addr>.in-addr.arpa
- <IPv6 Addr>.ip6.arpa

Connection Types

The DNS modules supports UDP, TCP, TLS and Multicast-UDP for performing DNS queries.

The default connection method is to use UDP to port 53 of the servers configured either manually or through DHCP/RA.

DNS over UDP

For queries to be sent via UDP, +DNSC option DNS_CONN_PREF must be set to 0 (UDP/TLS) and TLS_CONF must be 0. These are the defaults.

The UDP port on the server used for queries will be set by DNS_SRV_PORT which defaults to 53.

DNS over TCP

For queries to be sent via TCP, +DNSC option DNS_CONN_PREF must be set to 1 (TCP/TLS) and TLS_CONF must be 0.

The TCP port on the server used for queries will be set by DNS_SRV_PORT which defaults to 53.

Option DNS_CONN_IDLE_TIMEOUT can be used to set an idle timeout, the DNS module will request the IDLE timeout from the DNS server and honour that time period.

DNS over TLS

For queries to be sent via TLS, +DNSC option DNS_CONN_PREF must be set to 1 (TCP/TLS) and TLS_CONF must be set to a TLS configuration index (see +TLSC). DTLS (DNS_CONN_PREF set to 0 with a TLS configuration) is not supported.

The TLS configuration must allow the DNS server to be authenticated. By default this requires a CA certificate and the server's domain name to be configured; see TLS.

When TLS is used, queries are sent only to the first configured DNS server.

The TLS port on the server used for queries will be set by DNS_SRV_SECURE_PORT which defaults to 853.

Option DNS_CONN_IDLE_TIMEOUT can be used to set an idle timeout, the DNS module will request the IDLE timeout from the DNS server and honour that time period.

 ©2023-2026

DNSSEC Support

Support for DNSSEC is configured by +DNSC option DNS_DNSSEC_MODE which offers three possible modes of operation:

1. No support, the DO bit of EDNS0 extended flags is not set indicating that the DCE is unaware of DNSSEC operations.
2. DNSSEC aware, this sets the DO bit of EDNS0 extended flags to indicate that DNSSEC RRs should be sent in response to queries. These RRs will be passed to the DTE for further processing.
3. DNSSEC authenticated answer only. In addition to the previous state the DNS module will not process any query responses which do not have either the AA bit set (indicating the DNS server is an authority for the domain) or the AD bit set (indicating the server has authenticated the response).

Cache Control

Query responses are held in a small answer cache to speed up further queries. Records are held in the cache based on the TTL value specified in the records. In addition the DTE can set a cache timeout via +DNSC option DNS_CACHE_TTL which, if lower than a response TTL, will override it causing records to be expired from the cache sooner.

MDNS Support

In addition to unicast DNS queries the DNS module supports multicast queries for those records in the **.local** domain.

+DNSC option DNS_MC_SRV_PREF configures if either or both IPv4 and IPv6 multicast queries should be made. If DNS_MC_SRV_PREF is non-zero and the query domain is **.local** a multicast DNS query will be made.

Command Reference:

+DNSC

Description

This command is used to read or set the DNS configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+DNSC	Query all configuration elements	GGGG
AT+DNSC=<ID>	Query a single element	GGGG
AT+DNSC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+DNSC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<DNS_SVR>	IPv4 Address IPv6 Address	DNS server IP address. 1.0-1.1 configured (writable), 1.0-1.3 active (read only) Unsigned 32-bit value Format of IPv4 address is 'a.b.c.d' Format of IPv6 address is 'a:b:c:d::e:f' This is a multiple value parameter with an ID range 1.0 to 1.3 See DNS Servers below.	GGGG
2	<DNS_AUTO>	Bool	DNS server auto-assignment 0 Manual - use DNS_SVR. 1 Auto - through DHCP etc..	GGGG
3	<DNS_TIMEOUT>	Unsigned Integer	DNS timeout in milliseconds Unsigned 16-bit value	GGGG
4	<DNS_PROTO_PREF>	Unsigned Integer	Result protocol preference 1 A preferred over AAAA records. 2 AAAA preferred over A records. 3 A only. 4 AAAA only.	GGGG
6	<DNS_CACHE_TTL>	Unsigned Integer	DNS cache maximum TTL Unsigned 16-bit value	GGGG
7	<TLS_CONF>	Integer	TLS configuration index (see +TLSC). 0 disables TLS Valid range is 0 to 4	GGGG
8	<DNS_DNSSEC_MODE>	Unsigned Integer	DNSSEC mode 0 DNSSEC awareness off. 1 DNSSEC aware, checking disabled. 2 DNSSEC authenticated answers only.	GGGG
10	<DNS_SRV_PORT>	Unsigned Integer	DNS server UDP/TCP port Positive unsigned 16-bit value	GGGG
11	<DNS_SRV_SECURE_PORT>	Unsigned Integer	DNS server TLS port Positive unsigned 16-bit value	GGGG

ID	Name	Type	Description	Sec
12	<DNS_CONN_PREF>	Unsigned Integer	Connection preference. TLS is used if TLS_CONF is set 0 UDP or TLS. 1 TCP or TLS.	GGGG
13	<DNS_MC_SRV_PREF>	Unsigned Integer	Multicast server preference 0 Not used. 1 IPv4 servers only. 2 IPv6 servers only. 3 IPv4 & IPv6 servers.	GGGG
14	<DNS_CONN_IDLE_TIMEOUT>	Unsigned Integer	DNS connection idle timeout (in milliseconds) Unsigned 16-bit value	GGGG

DNS Servers

DNS_SVR holds two lists:

- **Configured servers**, indices 1.0 and 1.1. These can be set, and are reported while the network interface is down. They are used when DNS_AUTO is 0.
- **Active servers**, indices 1.0 to 1.3. These are read only and reported while the network interface is up. They contain the servers currently in use, from the configured list or from DHCP / router advertisements.

Setting DNS_SVR without an index adds the address to the next free configured slot; this fails if both slots are in use. Setting it to 0.0.0.0 without an index removes all configured servers.

 ©2023-2026

+DNSRESOLV

Description

This command is used to resolve domain names via DNS.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+DNSRESOLV=<TYPE>,<QUERY_NAME>	Resolve a name for the specified record type	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<TYPE>	Integer	Type of record 1 A. 2 NS. 5 CNAME. 6 SOA. 12 PTR. 15 MX. 16 TXT. 28 AAAA. 33 SRV. 43 DS. 46 RRSIG. 47 NSEC. 48 DNSKEY. 255 A and/or AAAA, according to DNS_PROTO_PREF. Positive unsigned 8-bit value
<QUERY_NAME>	String	Query name to resolve Maximum length of string is 256

AEC Reference:

+DNSRESOLV

Description

Resolve results.

AEC Syntax

AEC	Description
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<ADDRESS>	A & AAAA response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<NSDNAME>	NS response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<CNAME>	CNAME response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<MNAME>,<SERIAL>,<REFRESH>,<RETRY>,<EXPIRE>,<MINIMUM>,<RNAME>	SOA response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<PTRDNAME>	PTR response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<EXCHANGE>,<PREFERENCE>	MX response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<TXT_DATA>	TXT response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<TARGET>,<PRIORITY>,<WEIGHT>,<PORT>	SRV response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<KEY_TAG>,<ALGORITHM>,<DIGEST_TYPE>,<DIGEST>	DS response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<SIGNER_NAME>,<TYPE>,<ALGORITHM>,<LABELS>,<ORIG_TTL>,<SIG_EXPIRE>,<SIG_INCEPT>,<KEY_TAG>,<SIGNATURE>	RRSIG response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<NEXT_DOMAIN>,<TYPE_BIT_MAPS>	NSEC response
+DNSRESOLV:<TYPE>,<QUERY_NAME>,<FLAGS>,<PROTOCOL>,<ALGORITHM>,<PUBLIC_KEY>	DNSKEY response

 ©2023-2026

AEC Element Syntax

Element Name	Type	Description
<TYPE>	Integer	Type of record 1 A. 2 NS. 5 CNAME. 6 SOA. 12 PTR. 15 MX. 16 TXT. 28 AAAA. 33 SRV. 43 DS. 46 RRSIG. 47 NSEC. 48 DNSKEY. 255 A and/or AAAA (DNSERR only). Positive unsigned 8-bit value
<QUERY_NAME>	String	Original query name requested
<ADDRESS>	IPv4 or IPv6 Address	IP address
<NSDNAME>	String	Name server domain name
<CNAME>	String	Canonical alias domain name
<MNAME>	String	Primary name server domain name
<SERIAL>	Integer	Serial number Unsigned 32-bit value
<REFRESH>	Integer	Zone refresh interval Unsigned 32-bit value
<RETRY>	Integer	Zone refresh retry interval Unsigned 32-bit value
<EXPIRE>	Integer	Zone expiry interval Unsigned 32-bit value
<MINIMUM>	Integer	Minimum exported RR TTL Unsigned 32-bit value
<RNAME>	String	Mailbox domain name
<PTRDNAME>	String	Pointer domain name
<EXCHANGE>	String	Exchange domain name

Element Name	Type	Description
<PREFERENCE>	Integer	Preference Unsigned 16-bit value
<TXT_DATA>	String	Descriptive text
<TARGET>	String	Target domain name
<PRIORITY>	Integer	Host priority Unsigned 16-bit value
<WEIGHT>	Integer	Relative service weight Unsigned 16-bit value
<PORT>	Integer	Service port Unsigned 16-bit value
<KEY_TAG>	Integer	Key tag Unsigned 16-bit value
<ALGORITHM>	Integer	Algorithm Positive unsigned 8-bit value
<DIGEST_TYPE>	Integer	Digest type Positive unsigned 8-bit value
<DIGEST>	Byte Array	Digest
<SIGNER_NAME>	String	Signer’s name
<LABELS>	Integer	Number of labels Unsigned 8-bit value
<ORIG_TTL>	Integer	Original TTL Unsigned 32-bit value
<SIG_EXPIRE>	Integer	Signature expiration Unsigned 32-bit value
<SIG_INCEPT>	Integer	Signature inception Unsigned 32-bit value
<SIGNATURE>	Byte Array	Signature
<NEXT_DOMAIN>	String	Next domain name
<TYPE_BIT_MAPS>	Byte Array	Type bit maps
<FLAGS>	Integer	Flags Unsigned 16-bit value
<PROTOCOL>	Integer	Protocol 3 DNSSEC.
<PUBLIC_KEY>	Byte Array	Public key

 ©2023-2026

+DNSERR

Description

Resolve failure.

AEC Syntax

AEC	Description
+DNSERR:<ERROR_CODE>,<TYPE>,<QUERY_NAME>	Resolve failure

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	Integer	Error code See Status Response Codes
<TYPE>	Integer	Type of record 1 A. 2 NS. 5 CNAME. 6 SOA. 12 PTR. 15 MX. 16 TXT. 28 AAAA. 33 SRV. 43 DS. 46 RRSIG. 47 NSEC. 48 DNSKEY. 255 A and/or AAAA (DNSERR only). Positive unsigned 8-bit value
<QUERY_NAME>	String	Original query name requested

Examples:

Example Command Sequence 12: A record query

```
→   AT+DNSRESOLV=1,"www.example.com"
←   OK
←   +DNSRESOLV:1,"www.example.com","55.123.145.67"
```

Example Command Sequence 13: NS record query

```
→   AT+DNSRESOLV=2,"example.com"
←   OK
←   +DNSRESOLV:2,"example.com","ns1.example.com"
←   +DNSRESOLV:2,"example.com","ns2.example.com"
←   +DNSRESOLV:1,"ns1.example.com","192.168.0.1"
←   +DNSRESOLV:28,"ns1.example.com","1234:5678:ABCD::123"
←   +DNSRESOLV:1,"ns2.example.com","192.168.0.2"
←   +DNSRESOLV:28,"ns2.example.com","1234:5678:ABCD::124"
```

Example Command Sequence 14: CNAME record query

```
→   AT+DNSRESOLV=5,"www.example.com"
←   OK
←   +DNSRESOLV:5,"www.example.com","ano.example.com"
```

Example Command Sequence 15: SOA record query

```
→   AT+DNSRESOLV=6,"example.com"
←   OK
←   +DNSRESOLV:6,"example.com","ns1.example.com",1728655446,10800
    ,3600,604800,3600,"admin.example.com"
```

Example Command Sequence 16: PTR record query

```
→   AT+DNSRESOLV=12,"67.145.123.55.in-addr.arpa"
←   OK
←   +DNSRESOLV:12,"67.145.123.55.in-addr.arpa","www.example.com"
```

Example Command Sequence 17: MX record query

```
→   AT+DNSRESOLV=15,"example.com"
←   OK
←   +DNSRESOLV:15,"example.com","mailx.example.com",10
←   +DNSRESOLV:1,"mailx.example.com","192.168.0.101"
←   +DNSRESOLV:28,"mailx.example.com","1234:5678:ABCD::5101"
```

 ©2023-2026

Example Command Sequence 18: TXT record query

```
→   AT+DNSRESOLV=16,"example.com"

←   OK

←   +DNSRESOLV:16,"example.com","v=spf1 mx a
    include:_spf.example.com include:_spf.example.com ~all"
```

Example Command Sequence 19: AAAA record query

```
→   AT+DNSRESOLV=28,"www.example.com"

←   OK

←   +DNSRESOLV:28,"www.example.com","1234:5678:ABC:D000::EF12"
```

Example Command Sequence 20: SRV record query

```
→   AT+DNSRESOLV=33,"_imaps._tcp.example.com"

←   OK

←   +DNSRESOLV:33,"_imaps._tcp.example.com","imap.example.com",5,
    0,993
```

Example Command Sequence 21: DS record query

```
→   AT+DNSRESOLV=43,"example.com"

←   OK

←   +DNSRESOLV:43,"example.com",2371,13,2,[E44A0F027788…
    0BEA77D781]
```

Example Command Sequence 22: NSEC record query

```
→   AT+DNSRESOLV=47,"example.com"

←   OK

←   +DNSRESOLV:47,"example.com","\\000.example.com",[00096205800C
    …01C0]
```

Example Command Sequence 23: DNSKEY record query

```
→   AT+DNSRESOLV=48,"example.com"

←   OK

←   +DNSRESOLV:48,"example.com",257,3,13,[99DB2CC14CAB…
    4BE7778A19]

←   +DNSRESOLV:48,"example.com",256,3,13,[A09311112CF9…
    806B65E148]
```

Example Command Sequence 24: ANY record query

```
→   AT+DNSRESOLV=255,"www.example.com"

←   OK

←   +DNSRESOLV:1,"www.example.com","55.123.145.67"

←   +DNSRESOLV:28,"www.example.com","1234:5678:ABC:D000::EF12"
```

Example Command Sequence 25: A record query With DNSSEC awareness

```
→   AT+DNSC=8,1

←   OK

→   AT+DNSRESOLV=1,"www.example.com"

←   OK

←   +DNSRESOLV:1,"www.example.com","123.45.67.1"

←   +DNSRESOLV:1,"www.example.com","123.45.67.2"

←   +DNSRESOLV:46,"www.example.com","example.com",1,13,3,300,1760
    198187,1760018187,34505,[D48611C4E419···48196DD1F7]
```

Example Command Sequence 26: NSEC3PARAM (unknown type) record query

```
→   AT+DNSC=8,1

←   OK

→   AT+DNSRESOLV=51,"example.com"

←   OK

←   +DNSRESOLV:51,"example.com",[010000180853···A4C2]

←   +DNSRESOLV:46,"example.com","example.com",51,8,2,86400,176120
    2386,1759992786,43065,[57BE130BD9BC3E···9A00B300EC]
```

Example Command Sequence 27: NSEC3 (unknown type) response to invalid A record query

```
→   AT+DNSRESOLV=1,"0000.example.com"

←   OK

←   +DNSRESOLV:6,"example.com","ns.example.com",2780200301,1200,1
    80,1209600,60,"techadmin.example.com"

←   +DNSRESOLV:46,"example.com","example.com",6,13,2,3600,1760329
    634,1760066834,56352,[EB66F399ECCAD5···FEE205A641]

←   +DNSRESOLV:46,"11g4e2qvorlg6fqtd14inr597c52fedl.example.com",
    "example.com",50,13,3,60,1760329634,1760066834,56352,[E90FDC2
    BDB03···E5F0A9E1]

←   +DNSRESOLV:50,"11g4e2qvorlg6fqtd14inr597c52fedl.example.com",
    [010000000014···00000002]

←   +DNSRESOLV:46,"g5nshuef875ssknoqrrcqg2cait0snsd.example.com",
    "example.com",50,13,3,60,1760329634,1760066834,56352,[331653E
    ACEF4···38765448]

←   +DNSRESOLV:50,"g5nshuef875ssknoqrrcqg2cait0snsd.example.com",
    [010000000014···00000002]

←   +DNSRESOLV:46,"q0fhrs556u4ps37jqnta5dhrufgu4ths.example.com",
    "example.com",50,13,3,60,1760329634,1760066834,56352,[AB0CB2C
    BC0A2···DDBBAE12]

←   +DNSRESOLV:50,"q0fhrs556u4ps37jqnta5dhrufgu4ths.example.com",
    [010000000014···00000290]

←   +DNSERR:5.4,"DNS Non-Existent Domain",1,"0000.example.com"
```

 ©2023-2026

Example Command Sequence 28: Multicast A record query

```
→   AT+DNSC=13,3
←   OK
→   AT+DNSRESOLV=1,"example.local"
←   OK
←   +DNSRESOLV:1,"example.local","123.45.6.101"
```

Example Command Sequence 29: MDNS-SD of local SSH services

```
→   AT+DNSC=13,3
←   OK
→   AT+DNSRESOLV=12,"_ssh._tcp.local"                    PTR lookup for _ssh._tcp
←   OK
←   +DNSRESOLV:12,"_ssh._tcp.local","example._ssh._tcp.local"
→   AT+DNSRESOLV=33,"example._ssh._tcp.local"            SRV lookup for example.local SSH service
←   OK
←   +DNSRESOLV:33,"example._ssh._tcp.local","example.local",0,0,2
    2
→   AT+DNSRESOLV=16,"example._ssh._tcp.local"            TXT lookup for example.local SSH service
←   OK
←   +DNSRESOLV:16,"example._ssh._tcp.local",[]
←   OK
→   AT+DNSRESOLV=1,"example.local"                       A record lookup for example.local
←   OK
←   +DNSRESOLV:1,"example.local","123.45.6.101"
```

FS (Module ID = 7)

Command Reference:

+FS

Description

This command performs a filesystem operation.

The filesystem operation command is split into several sub-commands:

- Load
- List
- Delete
- Info
- Store

Load Sub-Command:

The load sub-command initiates a file transfer from the DTE to the DCE.

List Sub-Command:

The list sub-command produces a list of files present in the filesystem based on the file type specified.

Delete Sub-Command:

The delete sub-command deletes a single file object of the type specified from the filesystem.

Info Sub-Command:

The info sub-command returns information on the filesystem.

Store Sub-Command:

The store sub-command initiates a file transfer from the DCE to the DTE.

Security

Default security for the command is: `GGGG`

 ©2023-2026

Command Syntax

Command	Description	Sec
AT+FS=<OP>	Operate on filesystem OP must not be 1 or 5	GGGG
AT+FS=<OP>,<FILETYPE>	Operate on file types OP must not be 1 or 5	GGGG
AT+FS=<OP>,<FILETYPE>,<FILENAME>	Operate on files OP must not be 1 or 5	GGGG
AT+FS=<OP>,<FILETYPE>,<TSFRPROT>,<FILENAME>,<FILELENGTH>	Load file (single) OP must be 1	GGGG
AT+FS=<OP>,<FILETYPE>,<TSFRPROT>	Load file (batch) OP must be 1	GGGG
AT+FS=<OP>,<FILETYPE>,<TSFRPROT>,<FILENAME>	Store file (single) OP must be 5	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<OP>	Integer	Operation 1 Load. 2 List. 3 Delete. 4 Information. 5 Store.
<FILETYPE>	Integer	File type 0 User. 1 Certificate. 2 Private Key. 3 Diffie-Hellman parameters. 20 Configuration File.
<FILENAME>	String	The name of the file Maximum length of string is 32
<TSFRPROT>	Integer	Transfer protocol 1 X Modem + checksum. 2 X Modem + CRC16. 3 X Modem 1K. 4 Y Modem. 5 FS-TSFR.
<FILELENGTH>	Integer	File length Positive unsigned 16-bit value

Response Syntax

Response	Description
+FS:<OP>,<FILETYPE>,<FILENAME>	List operation response
+FS:<OP>,<FREE_SPACE>,<FREE_HANDLES>	Information operation response
+FS:<OP>,<TSFR_HANDLE>	FS-TSFR response

 ©2023-2026

Response Element Syntax

Element Name	Type	Description
<OP>	Integer	Operation 1 Load. 2 List. 3 Delete. 4 Information. 5 Store.
<FILETYPE>	Integer	File type 0 User. 1 Certificate. 2 Private Key. 3 Diffie-Hellman parameters. 20 Configuration File.
<FILENAME>	String	The name of the file
<FREE_SPACE>	Integer	Free space Positive unsigned 16-bit value
<FREE_HANDLES>	Integer	Free file handles Positive unsigned 16-bit value
<TSFR_HANDLE>	Integer	Transfer handle Positive unsigned 16-bit value

+FSTSFR

Description

This command performs a filesystem transfer operation.

The FS-TSFR protocol transfers a file in numbered blocks. A transfer is started with +FS (load or store), which returns a transfer handle. Only one transfer may be active at a time. The syntax forms that apply depend on the direction of the transfer:

- **Load** (DTE to DCE): send each block with AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM>,<DATA>, starting at block 1. A block of 1 to 128 bytes is sent as a 128-byte block, and 129 to 1024 bytes as a 1024-byte block. Only the final block may be shorter than its block size; it is padded by the DCE. An optional CRC-16/XMODEM of <DATA> may be supplied. The transfer completes automatically when all bytes of the file have been sent. AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM> ends the transfer early.
- **Store** (DCE to DTE): read each block with AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM> (up to 128 bytes) or AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM>,<DATA_LENGTH>, starting at block 1. The transfer completes when all bytes have been read; a <BLOCK_NUM> of 0 ends it early.

In both directions <BLOCK_NUM> must be the next expected block number. AT+FSTSFR=<TSFR_HANDLE> reports the last block number transferred and the number of bytes remaining.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+FSTSFR	Transfer query (all)	GGGG
AT+FSTSFR=<TSFR_HANDLE>	Transfer query	GGGG
AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM>	Load: end transfer	GGGG
AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM>,<DATA>	Load: send block	GGGG
AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM>,<DATA>,<CRC>	Load: send block (with CRC)	GGGG
AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM>	Store: read block (up to 128 bytes), or end	GGGG
AT+FSTSFR=<TSFR_HANDLE>,<BLOCK_NUM>,<DATA_LENGTH>	Store: read block (up to DATA_LENGTH bytes)	GGGG

 ©2023-2026

Command Parameter Syntax

Parameter Name	Type	Description
<TSFR_HANDLE>	Integer	Transfer handle Positive unsigned 16-bit value
<BLOCK_NUM>	Integer	Block number Unsigned 8-bit value
<DATA>	String Byte Array	Transfer data
<CRC>	Integer	Transfer CRC-16 Unsigned 16-bit value
<DATA_LENGTH>	Integer	Data length Positive unsigned 16-bit value

Response Syntax

Response	Description
+FSTSFR:<TSFR_HANDLE>,<BLOCK_NUM>,<BYTES_REMAIN>	FS-TSFR load response
+FSTSFR:<TSFR_HANDLE>,<BLOCK_NUM>,<BYTES_REMAIN>,<DATA>	FS-TSFR store response

Response Element Syntax

Element Name	Type	Description
<TSFR_HANDLE>	Integer	Transfer handle Positive unsigned 16-bit value
<BLOCK_NUM>	Integer	Block number Unsigned 8-bit value
<BYTES_REMAIN>	Integer	Bytes remaining Positive unsigned 16-bit value
<DATA>	String Byte Array	Transfer data

AEC Reference:

+FSUP

Description

Filesystem update.

AEC Syntax

AEC	Description
+FSUP:<OP>,<FILETYPE>,<FILENAME>,<FILELENGTH>	Filesystem load

AEC Element Syntax

Element Name	Type	Description
<OP>	Integer	Operation 1 Load. 2 List. 3 Delete. 4 Information. 5 Store.
<FILETYPE>	Integer	File type 0 User. 1 Certificate. 2 Private Key. 3 Diffie-Hellman parameters. 20 Configuration File.
<FILENAME>	String	The name of the file
<FILELENGTH>	Integer	File length Positive unsigned 16-bit value

 ©2023-2026

Examples:

Example Command Sequence 30: File transfer using XModem+CRC with +FS

→	AT+FS=1,1,2,"ISRGRootX1",1391	Load 1391 byte certificate using XModem+CRC
←	#	Raw binary mode indicator
←	CC	Initial handshake, repeating C meaning CRC
→	SOH 01 FE DATA[128] CRC CRC	First 128 byte data frame
←	ACK	Receiver acknowledge
→	SOH 02 FD DATA[128] CRC CRC	
←	ACK	
→	SOH 03 FC DATA[128] CRC CRC	
←	ACK	
→	SOH 04 FB DATA[128] CRC CRC	
←	ACK	
→	SOH 05 FA DATA[128] CRC CRC	
←	ACK	
→	SOH 06 F9 DATA[128] CRC CRC	
←	ACK	
→	SOH 07 F8 DATA[128] CRC CRC	
←	ACK	
→	SOH 08 F7 DATA[128] CRC CRC	
←	ACK	
→	SOH 09 F6 DATA[128] CRC CRC	
←	ACK	
→	SOH 0A F5 DATA[128] CRC CRC	
←	ACK	
→	SOH 0B F4 DATA[128] CRC CRC	
←	ACK	
→	EOT	End of transmission
←	ACK	
←	OK	AT Command completion

Example Command Sequence 31: List and delete a file

→	`AT+FS=2,1`	Request list of certificates
←	`+FS:2,1,"AmazonRootCA1"`	
←	`+FS:2,1,"DigiCert"`	
←	`+FS:2,1,"DigiCertGlobalRootG2"`	
←	`+FS:2,1,"DigiCertSHA2"`	
←	`+FS:2,1,"EnTrust"`	
←	`+FS:2,1,"GlobalSignRoot"`	
←	`+FS:2,1,"ISRGRootX1"`	
←	`+FS:2,1,"QuoVadis_Root"`	
←	`+FS:2,1,"VeriSign"`	
←	`OK`	
→	`AT+FS=3,1,"AmazonRootCA1"`	Delete certificate
←	`OK`	

Example Command Sequence 32: Get filesystem information

→	`AT+FS=4`	Request filesystem information
←	`+FS:4,45056,17`	45056 bytes, 17 file handles free
←	`OK`	

 ©2023-2026

Example Command Sequence 33: File load using FS-TSFR protocol

→	`AT+FS=1,1,5,"ExampleCert",891`	Load certificate using FS_TSFR
←	`+FS:1,20489`	Certificate transfer handle
←	`OK`	
→	`AT+FSTSFR=20489,1,[308203773082025F…7420526F6F74301E]`	Send block 1, 128 bytes of data
←	`+FSTSFR:20489,1,763`	Ack block 1, 763 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,2,[170D303030353132…6F7430820122300D]`	Send block 2, 128 bytes of data
←	`+FSTSFR:20489,2,635`	Ack block 2, 635 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,3,[06092A864886F70D…C328EAF5AB25878A]`	Send block 3, 128 bytes of data
←	`+FSTSFR:20489,3,507`	Ack block 3, 507 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,4,[9A961CA967B83F0C…BDC63AECE76E863A]`	Send block 4, 128 bytes of data
←	`+FSTSFR:20489,4,379`	Ack block 4, 379 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,5,[6B97746333BD6818…010100850C5D8EE4]`	Send block 5, 128 bytes of data
←	`+FSTSFR:20489,5,251`	Ack block 5, 251 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,6,[6F51684205A0DDBB…A9317A18BFA02AF4]`	Send block 6, 128 bytes of data
←	`+FSTSFR:20489,6,123`	Ack block 6, 123 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,7,[1299F7A34582E33C…6339A9]`	Send block 7, 123 bytes of data
←	`+FSTSFR:20489,7,0`	Ack block 7, no bytes remain
←	`OK`	

Example Command Sequence 34: File store using FS-TSFR protocol

→	`AT+FS=5,1,5,"ExampleCert"`	Store certificate using FS-TSFR
←	`+FS:5,20489`	Certificate transfer handle
←	`OK`	
→	`AT+FSTSFR=20489`	Get transfer report
←	`+FSTSFR:20489,0,891`	No blocks sent, 891 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,1,128`	Receive block 1, 128 bytes of data
←	`+FSTSFR:20489,1,763,[308203773082025F…7420526F6F74301E]`	Block 1, 128 bytes, 763 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,2,128`	Receive block 2, 128 bytes of data
←	`+FSTSFR:20489,2,635,[170D303030353132…6F7430820122300D]`	Block 2, 128 bytes, 635 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,3,128`	Receive block 3, 128 bytes of data
←	`+FSTSFR:20489,3,507,[06092A864886F70D…C328EAF5AB25878A]`	Block 3, 128 bytes, 507 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,4,128`	Receive block 4, 128 bytes of data
←	`+FSTSFR:20489,4,379,[9A961CA967B83F0C…BDC63AECE76E863A]`	Block 4, 128 bytes, 379 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,5,128`	Receive block 5, 128 bytes of data
←	`+FSTSFR:20489,5,251,[6B97746333BD6818…010100850C5D8EE4]`	Block 5, 128 bytes, 251 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,6,128`	Receive block 6, 128 bytes of data
←	`+FSTSFR:20489,6,123,[6F51684205A0DDBB…A9317A18BFA02AF4]`	Block 6, 128 bytes, 123 bytes remain
←	`OK`	
→	`AT+FSTSFR=20489,7,123`	Receive block 7, 123 bytes of data
←	`+FSTSFR:20489,7,0,[1299F7A34582E33C5E…39A9]`	Block 7, 123 bytes, no bytes remain
←	`OK`	

 ©2023-2026

MQTT (Module ID = 8)

Introduction

The RNWF02 supports MQTT v3.1.1 and MQTT v5.0 client functionality via AT commands, enabling communication with MQTT brokers for IoT applications, telemetry, messaging, and cloud integration.

Key Features

Protocol Support

Both MQTT v3.1.1 and MQTT v5.0 protocols are supported. MQTT v5.0 includes enhanced features such as properties, reason codes, and improved error reporting.

Quality of Service (QoS)

All three QoS levels are supported:

- QoS 0 (At most once) - Fire and forget, no acknowledgment
- QoS 1 (At least once) - Guaranteed delivery with acknowledgment
- QoS 2 (Exactly once) - Guaranteed single delivery with four-way handshake

TLS Security

Secure MQTT connections (MQTTS) over TLS are supported by configuring a TLS context before connecting.

Large Message Handling

The device can send and receive messages larger than its internal buffers through message fragmentation. The MQTT stack handles reassembly of multi-chunk messages automatically.

Connecting to a Broker

To establish an MQTT connection:

1. Configure the broker parameters using +MQTTC:
   - BROKER_ADDR - Broker hostname or IP address
   - BROKER_PORT - Broker port (typically 1883 for MQTT, 8883 for MQTTS)
   - CLIENT_ID - Unique client identifier
   - USERNAME and PASSWORD - Authentication credentials (if required)
2. For secure connections, configure TLS using +MQTTC parameter TLS_CONF to reference a TLS context configured via TLSC.
3. Connect to the broker using +MQTTCONN.
4. Monitor the +MQTTCONN AEC for connection status.

 ©2023-2026

Publishing Messages

Publish messages to topics using +MQTTPUB. The command accepts the topic name, QoS level, retain flag, and message payload.

Single Message Mode (FRAG=0)

For messages that fit within the AT command buffer, use +MQTTPUB with FRAG=0:

```
AT+MQTTPUB=0,<QOS>,<RETAIN>,"<topic>",<payload>
```

For QoS 0, the response is simply OK. For QoS 1 or 2, the response includes the packet ID: +MQTTPUB:<packet_id> followed by OK. The same packet ID will appear in the subsequent +MQTTPUBACK or +MQTTPUBCOMP AEC.

The AT command interface has a maximum command line length which limits the payload size. The maximum depends on encoding format and topic name length:

Factor	Size
Maximum command line length	3072 bytes
Command overhead (AT+MQTTPUB=0,<qos>,<retain>,"<topic>",)	~30 bytes
Topic name	variable
Hex encoding [xxxx]	2x payload size + 2 bytes
String encoding "…"	1x payload size + 2 bytes

Encoding	Typical Max Payload	Notes
Hex [xxxx]	~1500 bytes	Binary-safe, recommended
String "…"	~3000 bytes	Requires escaping special characters

NOTE For reliable operation with typical topic names, limit single-message payloads to approximately 1500 bytes when using hex encoding.

Fragmented Publish Mode (FRAG>=1)

For payloads exceeding the AT command buffer limit, use fragmented publish to send large messages in multiple parts. The firmware reassembles fragments before transmitting a single MQTT PUBLISH packet.

Starting a Fragment Sequence (First Fragment)

```
AT+MQTTPUB=1,<QOS>,<RETAIN>,"<topic>"|<alias>,<payload>
```

- FRAG=1 indicates the first fragment of a new sequence
- QOS, RETAIN, and topic are specified only in the first fragment
- Topic can be a string `"topic/name"` or an integer topic alias (MQTT v5.0)
- Returns `+MQTTPUB:<next_frag>,<total_bytes>` on success

Continuation Fragments

```
AT+MQTTPUB=<FRAG>,<COMP>,<payload>
```

- FRAG = fragment number (must match expected sequence)
- COMP = 0 for continuation, 1 for final fragment
- Returns `+MQTTPUB:<next_frag>,<total_bytes>` after each non-final fragment
- For the final fragment (COMP=1) with QoS 1 or 2, returns `+MQTTPUB:<packet_id>` before OK

Query Fragment State

```
AT+MQTTPUB
```

Returns `+MQTTPUB:<expected_frag>,<total_bytes>,"<topic>"` if a sequence is in progress, or `OK` with no response if idle.

Example: Sending a 4KB Payload (QoS 0)

```
AT+MQTTPUB=1,0,0,"sensor/data",[<first_500_bytes_hex>]
+MQTTPUB:2,500
OK
AT+MQTTPUB=2,0,[<next_500_bytes_hex>]
+MQTTPUB:3,1000
OK
AT+MQTTPUB=3,0,[<next_500_bytes_hex>]
+MQTTPUB:4,1500
OK
...
AT+MQTTPUB=9,1,[<final_bytes_hex>]
OK
+MQTTPUBSENT
```

 ©2023-2026

Example: Sending with QoS 1

For QoS 1/2, the final fragment returns the packet ID, which matches the subsequent acknowledgement AEC:

```
AT+MQTTPUB=1,1,0,"sensor/data",[<first_500_bytes_hex>]
+MQTTPUB:2,500
OK
AT+MQTTPUB=2,0,[<next_500_bytes_hex>]
+MQTTPUB:3,1000
OK
...
AT+MQTTPUB=9,1,[<final_bytes_hex>]
+MQTTPUB:42
OK
+MQTTPUBACK:42
```

Fragmentation Limits

Constraint	Value	Notes
Maximum fragments	64	Per message
Maximum packet size	16,384 bytes (16KB)	Advertised protocol limit
Fragment size	~500-1500 bytes	Limited by AT command buffer
Timeout	30 seconds	Between fragments

NOTE The 16KB limit applies to the total MQTT packet size, which includes the topic name, MQTT v5 properties (if enabled), and payload. With a short topic and no properties, payloads up to ~16KB are possible. Longer topics or additional properties reduce the available payload space.

Error Handling

- Out-of-sequence fragments are rejected and the context is cleaned up
- Starting a new sequence (FRAG=1) while one is in progress is rejected
- Continuation fragments (FRAG>1) without a prior first fragment are rejected
- Invalid QoS values (>2) are rejected before allocating context
- Exceeding the maximum fragment count or payload size returns an error

Publish Completion Events

When a publish operation completes, the device emits an Asynchronous Event Code (AEC) to signal completion. This provides reliable synchronization for back-to-back publishes without requiring arbitrary delays.

QoS	AEC	Meaning
0	+MQTTPUBSENT	Publish sent to broker (fire-and-forget complete)
1	+MQTTPUBACK	PUBACK received from broker
2	+MQTTPUBCOMP	PUBCOMP received from broker (4-way handshake complete)

If a publish fails, +MQTTPUBERR is emitted (with packet_id for QoS 1/2).

IMPORTANT	Wait for the completion AEC before sending the next publish command. The device can only process one publish at a time. Sending a new publish while the previous one is in the PUBLISHING state will result in an error.

Subscribing and Receiving Messages

Subscribe to topics using +MQTTSUB. When messages arrive on subscribed topics, the +MQTTSUBRX AEC is emitted.

Inline vs Buffered Message Delivery

The READ_THRESHOLD parameter sets the largest message, in bytes, that is delivered inline. The default is 128 bytes.

- **Inline delivery**: a received message whose length is less than or equal to the threshold is included directly in the +MQTTSUBRX AEC.
- **Buffered delivery**: a received message longer than the threshold is held by the DCE. The +MQTTSUBRX AEC reports only the message ID and length, and the DTE must use +MQTTSUBRD to retrieve the data.

Messages received in more than one part are always buffered, whatever the threshold. Messages larger than approximately 1 KB will therefore always require +MQTTSUBRD.

NOTE	Setting the threshold to 0 causes all received messages to be buffered regardless of size, which is useful when consistent handling of all messages is required.

 ©2023-2026

Maximum Packet Size (MQTT v5.0)

MQTT v5.0 introduces the Maximum Packet Size property (ID 39), which informs the broker of the largest MQTT packet the client can receive. This property is configured via +MQTTPROPTX and controlled by +MQTTPROPTXS.

Default Behaviour

- The Maximum Packet Size property defaults to 16384 bytes (16KB).
- The property is **enabled by default** and will be included in the CONNECT packet.
- The broker will respect this limit and will not send packets larger than the specified size.

Configuration Options

Action	Command
Query current value	`AT+MQTTPROPTX=39`
Set to 8KB	`AT+MQTTPROPTX=39,8192`
Disable property in CONNECT	`AT+MQTTPROPTXS=39,0`
Re-enable property in CONNECT	`AT+MQTTPROPTXS=39,1`

Validation Rules

- The value cannot be set to 0 (zero).
- The value cannot exceed the system maximum of 16384 bytes.

Internal Enforcement

The device enforces the packet size limit internally regardless of broker behaviour:

- When the property is **enabled** in CONNECT: The configured value is enforced. The broker knows the limit and should not exceed it, but the device will reject oversized packets as a safety measure.
- When the property is **disabled** in CONNECT: The system maximum (16384 bytes) is enforced. Since the broker was not informed of any limit, the device uses its maximum capability.
- For **MQTT v3.1.1** connections: The system maximum (16384 bytes) is always enforced, as v3.1.1 does not support the Maximum Packet Size property.

NOTE	If you configure a lower Maximum Packet Size value, then disable the property and connect to a different broker, the device will use the system maximum (16KB) for enforcement - not your previously configured value. This ensures the device accepts the largest packets possible when the broker has not been informed of any limit.

Oversized Received Messages

A broker that does not honour the Maximum Packet Size property - or that was never told the limit because the property was disabled in CONNECT - may still forward a message larger than the device can buffer. Such a message cannot be delivered, but it also cannot simply be abandoned: the remainder of the payload would be left in the socket, leaving the MQTT byte stream out of frame.

The device therefore handles an undeliverable received message as follows:

1. The payload is read from the connection and discarded, keeping the MQTT stream framed and allowing the QoS 1/QoS 2 acknowledgement to be sent.
2. The +MQTTSUBRXERR AEC is emitted, reporting why the message was dropped along with the topic, packet identifier and total message length.
3. Once the payload has been fully drained, the session is closed cleanly with an MQTT v5.0 DISCONNECT carrying reason code `0x95` (Packet too large) for an oversized message, or `0x83` (Implementation specific error) for the other drop reasons.

The reported reasons are:

REASON	Meaning
1	The message exceeds the maximum packet size.
2	There was insufficient memory to buffer the message.
3	The message did not match any active subscription.
4	The message length was inconsistent with the receive buffer.

No data from a dropped message is delivered, and subsequent messages are unaffected. The session must be re-established after the disconnection, and any subscriptions re-created, as the subscription list is cleared when the session closes.

NOTE

`AT+MQTTPROPTXS=39,1` (the default) is the way to avoid this situation altogether: when the broker is told the limit, it will not forward an oversized message in the first place. `AT+MQTTPROPTX=39,<VALUE>` lowers the advertised limit below the 16KB system maximum if the DTE cannot handle messages that large.

 ©2023-2026

MQTT v5.0 Properties

MQTT v5.0 supports additional properties that can be included in various packet types. Properties are managed using:

- +MQTTPROPTX - Configure property values to transmit
- +MQTTPROPTXS - Select which properties to include in transmitted packets
- +MQTTPROPRX - Query properties received from the broker

Properties are only added to packets where they are applicable according to the MQTT v5.0 specification.

Properties with Fragmented Publish

MQTT v5.0 properties configured via +MQTTPROPTX and enabled via +MQTTPROPTXS are automatically included in fragmented publish messages. Configure properties before sending the first fragment (`FRAG=1`).

Supported publish properties include:

ID	Property	Description
1	Payload Format Indicator	0=binary, 1=UTF-8 text
2	Message Expiry Interval	Seconds until message expires
3	Content Type	MIME type string
8	Response Topic	Topic for request/response pattern
9	Correlation Data	Binary data for request/response correlation
35	Topic Alias	Integer alias instead of topic string
38	User Property	Custom key-value pairs

Topic Alias with Fragmented Publish

Topic aliases (MQTT v5.0) can reduce bandwidth by replacing topic strings with integers:

1. First, establish the alias by publishing with the topic name:

```
AT+MQTTPROPTX=35,1
AT+MQTTPROPTXS=35,1
AT+MQTTPUB=0,0,0,"sensor/data","first_message"
```

2. Subsequent publishes can use the alias (integer) instead of the topic string:

```
AT+MQTTPUB=1,0,0,1,[<hex_payload>]
AT+MQTTPUB=2,1,[<more_hex_payload>]
```

The alias value must be within the broker's advertised `topicAliasMax` (query with `AT+MQTTPROPRX=34`).

Typical Usage Flow

A typical MQTT session follows this pattern:

1. Configure Wi-Fi and establish network connectivity
2. (Optional) Configure SNTP for accurate timestamps
3. Configure TLS context if using secure connection
4. Configure MQTT client parameters
5. (Optional) Adjust MQTT v5.0 properties
6. Connect to the broker
7. Subscribe to topics of interest
8. Publish messages and/or receive subscribed messages
9. Disconnect when finished

 ©2023-2026

Troubleshooting

Connection Failures

Ensure the broker URL, port, and credentials are correct. For TLS connections, verify the CA certificate is loaded and the TLS context is properly configured.

Message Size Errors

If publishing fails, check that the hex-encoded payload plus command overhead does not exceed the command line limit. For received messages that appear truncated, check the Maximum Packet Size configuration.

QoS 2 Timeouts

QoS 2 requires a complete four-way handshake. Ensure network connectivity is stable and the broker supports QoS 2.

Fragmented Publish Errors

- **"MQTT Error" on continuation fragment**: Check that fragments are sent in sequence. Use `AT+MQTTPUB` (query) to see expected fragment number.
- **Fragment sequence aborted**: Ensure fragments are sent within the 30-second timeout. After timeout or error, start a new sequence with `FRAG=1`.
- **"Invalid Parameter" on first fragment**: Verify QoS is 0-2 and topic alias (if used) is within broker's limit.
- **Payload size exceeded**: Total payload across all fragments cannot exceed 16KB. Query `AT+MQTTPUB` to check accumulated size.

Publish Timing Issues

- **Error when sending consecutive publishes**: Always wait for the completion AEC (`+MQTTPUBSENT`, `+MQTTPUBACK`, or `+MQTTPUBCOMP`) before sending the next publish.
- **No completion AEC received**: Check network connectivity. For QoS 1/2, the AEC depends on broker acknowledgment.

Command Reference:

+MQTTC

Description

This command is used to read or set the MQTT configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTC	Query all configuration elements	GGGG
AT+MQTTC=<ID>	Query a single element	GGGG
AT+MQTTC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+MQTTC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<BROKER_ADDR>	String IPv4 Address IPv6 Address	Broker domain name or IP address Maximum length of string is 256 Format of IPv4 address is 'a.b.c.d' Format of IPv6 address is 'a:b:c:d::e:f'	GGGG
2	<BROKER_PORT>	Integer	Broker listening TCP port Unsigned 16-bit value	GGGG
3	<CLIENT_ID>	String	MQTT client ID Maximum length of string is 64	GGGG
4	<USERNAME>	String	Username Maximum length of string is 128	GGGG
5	<PASSWORD>	String	Password Maximum length of string is 256	GGGG
6	<KEEP_ALIVE>	Integer	Keep alive time (in seconds) Valid range is 0 to 0x7FFF	GGGG
7	<TLS_CONF>	Integer	TLS configuration index (see +TLSC). 0 disables TLS Valid range is 0 to 4 This parameter will only be visible to read if not zero. Setting a value greater than zero enables TLS and makes this parameter visible. Setting a value of zero disables TLS and makes this parameter hidden.	GGGG
8	<PROTO_VER>	Unsigned Integer	MQTT protocol version, either 3 or 5 3 V3.1.1. 5 V5.	GGGG
9	<READ_THRESHOLD>	Unsigned Integer	Largest received message size (in bytes) delivered inline, larger messages are buffered. Default is 128 Unsigned 16-bit value	GGGG
10	<SOCKET_ID>	Unsigned Integer	Socket ID	GGGG

+MQTTCONN

Description

This command is used to connect to an MQTT broker.

When issued without parameters, the current connection state is reported asynchronously using the +MQTTCONN AEC, rather than as a command response.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTCONN	Query current connection status	GGGG
AT+MQTTCONN=<CLEAN>	Start a new connection	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<CLEAN>	Integer	Clean Session 0 Use existing session, if available. 1 Use new session.

 ©2023-2026

+MQTTSUB

Description

This command is used to subscribe to an MQTT topic.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTSUB=<TOPIC_NAME>,<MAX_QOS>	Subscribe to a topic	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<TOPIC_NAME>	String	Topic Name Valid range is 0 to 128
<MAX_QOS>	Integer	Maximum QoS 0 QoS 0. 1 QoS 1. 2 QoS 2.

+MQTTSUBLST

Description

This command is used to list MQTT topic subscriptions.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTSUBLST	List subscriptions	GGGG

Response Syntax

Response	Description
+MQTTSUBLST:<TOPIC_NAME>,<QOS>	Subscription report

Response Element Syntax

Element Name	Type	Description
<TOPIC_NAME>	String	Topic Name
<QOS>	Integer	QoS 0 QoS 0. 1 QoS 1. 2 QoS 2.

 ©2023-2026

+MQTTSUBRD

Description

This command is used to receive data from subscriptions.

The +MQTTSUBRD command is used in conjunction with the setting of READ_THRESHOLD.

When a message is received the DCE will emit a +MQTTSUBRX AEC, this will normally include the received data, however when a message is received which would exceed READ_THRESHOLD the DCE will emit a +MQTTSUBRX AEC which includes a message ID and data length. This avoids saturating the DTE with data it may not be able to handle. The DTE must use this command to read the buffered message.

Setting READ_THRESHOLD to zero will cause all received messages to be buffered for reading using this command.

Buffered messages are read in the order they were received; only the oldest buffered message for the topic can be read, and requesting a different message ID returns an error.

A message may be read in several parts. If <LENGTH> is less than the remaining message data, only that many bytes are returned, and the next read of the same message ID continues from that point. The message is released once all of its data has been read. If no message is buffered for the topic, no data is returned.

The length element of the response is the number of bytes returned by this read.

NOTE	The message ID is only relevant for QoS1 and QoS2 subscriptions.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTSUBRD=<TOPIC_NAME>,<MSG_ID>,<LENGTH>	Receive data from a topic	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<TOPIC_NAME>	String	Topic Name Valid range is 0 to 128
<MSG_ID>	Integer	Message Identifier Unsigned 16-bit value
<LENGTH>	Integer	The number of bytes to receive Unsigned 16-bit value

Response Syntax

Response	Description
+MQTTSUBRD:<MSG_LENGTH>,<TOPIC_PAYLOAD>	Read subscribed data

Response Element Syntax

Element Name	Type	Description
<MSG_LENGTH>	Integer	Length of message
<TOPIC_PAYLOAD>	String	Topic Payload

 ©2023-2026

+MQTTUNSUB

Description

This command is used to unsubscribe from an MQTT topic.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTUNSUB=<TOPIC_NAME>	Unsubscribe from a topic	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<TOPIC_NAME>	String	Topic Name Valid range is 0 to 128

+MQTTPUB

Description

Publish a message (single or fragmented).

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTPUB=<FRAG>,<QOS>,<RETAIN>,<TOPIC_NAME_ID>,<TOPIC_PAYLOAD>	Publish complete or first fragment	GGGG
AT+MQTTPUB=<FRAG>,<COMP>,<TOPIC_PAYLOAD>	Publish continuation fragment	GGGG
AT+MQTTPUB	Query fragment state	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<FRAG>	Integer	Fragment Number (0=complete, 1=first, 2+=continuation) Valid range is 0 to 64
<QOS>	Integer	QoS 0 QoS 0. 1 QoS 1. 2 QoS 2.
<RETAIN>	Integer	Retain Message 0 Not retained on the server. 1 Retained on the server.
<TOPIC_NAME_ID>	Integer String Byte Array	Topic Name or Alias
<TOPIC_PAYLOAD>	String Byte Array	Topic Payload
<COMP>	Integer	Completion Flag 0 More fragments follow. 1 Final fragment, publish now.

 ©2023-2026

Response Syntax

Response	Description
+MQTTPUB:<PACKET_ID>	Publish (QoS 1 & 2)
+MQTTPUB:<FRAG>,<TOTAL_LEN>	Fragment accumulation feedback
+MQTTPUB:<FRAG>,<TOTAL_LEN>,<TOPIC_NAME>	Fragment state query

Response Element Syntax

Element Name	Type	Description
<PACKET_ID>	Integer	Packet ID
<FRAG>	Integer	Next expected fragment number
<TOTAL_LEN>	Integer	Total accumulated bytes
<TOPIC_NAME>	String	Topic Name

+MQTTLWT

Description

This command is used to define a last will message.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTLWT=<QOS>,<RETAIN>,<TOPIC_NAME>,<TOPIC_PAYLOAD>	Specify an LWT	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<QOS>	Integer	QoS 0 QoS 0. 1 QoS 1. 2 QoS 2.
<RETAIN>	Integer	Retain Message 0 Not retained on the server. 1 Retained on the server.
<TOPIC_NAME>	String	Topic Name Valid range is 0 to 128
<TOPIC_PAYLOAD>	String Byte Array	Topic Payload

 ©2023-2026

+MQTTDISCONN

Description

This command is used to disconnect from a broker.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTDISCONN	Disconnect	GGGG
AT+MQTTDISCONN=<REASON_CODE>	Disconnect with a reason	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<REASON_CODE>	Integer	Reason Code 0 Normal disconnection. 4 Disconnect with Will Message. 128 Unspecified error. 129 Malformed packet. 130 Protocol error. 131 Implementation specific error. 144 Topic name invalid. 147 Receive maximum exceeded. 148 Topic alias invalid. 149 Packet too large. 150 Message rate too high. 151 Quota exceeded. 152 Administrative action. 153 Payload format invalid. Unsigned 8-bit value

+MQTTPROPTX

Description

This command is used to read or set the MQTT transmit properties.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Any defined and selected properties are added to transmitted MQTT packets. Only properties applicable to the MQTT packet type are added. Properties must be selected via the +MQTTPROPTXS command before they are included.

Property names and identifiers are as defined in the MQTT Version 5.0 specification (OASIS), section 2.2.2.2.

NOTE Not all properties listed below may be available in any implementation of this command set.

User property (38) supports multiple entries in the form of key/value pairs. If a key/value pair is specified for the command it will be added, if only a key is specified it will be removed.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTPROPTX	Query all configuration elements	GGGG
AT+MQTTPROPTX=<ID>	Query a single element	GGGG
AT+MQTTPROPTX=<ID>,<VAL>	Set a single element ID must not be 38	GGGG
AT+MQTTPROPTX=<ID>,<KEY>,<VALS>	Set a single key/value pair ID must be 38	GGGG
AT+MQTTPROPTX=<ID>,<KEY>	Clear a single key/value pair ID must be 38	GGGG

 ©2023-2026

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value
<KEY>	String	Parameter key
<VALS>	String	Parameter value

Response Syntax

Response	Description
+MQTTPROPTX:<ID>,<VAL>	Read response
+MQTTPROPTX:<ID>,<KEY>,<VALS>	Read response (key/value)

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value
<KEY>	String	Parameter key
<VALS>	String	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<PAYLOAD_FORMAT_IND>	Unsigned Integer	Payload Format Indicator	GGGG
2	<MSG_EXPIRY_INTERVAL>	Unsigned Integer	Message Expiry Interval	GGGG
3	<CONTENT_TYPE>	String	Content Type Maximum length of string is 64	GGGG
8	<RESP_TOPIC>	String	Response Topic Maximum length of string is 64	GGGG
9	<CORRELATION_DATA>	Byte Array	Correlation Data	GGGG
11	<SUBSCRIPTION_ID>	Unsigned Integer	Subscription Identifier	GGGG
17	<SESSION_EXPIRY_INTERVAL>	Unsigned Integer	Session Expiry Interval	GGGG
23	<REQ_PROB_INFO>	Unsigned Integer	Request Problem Information	GGGG
24	<WILL_DELAY_INTERVAL>	Unsigned Integer	Will Delay Interval	GGGG
25	<REQ_RESP_INFO>	Unsigned Integer	Request Response Information	GGGG
31	<REASON_STR>	String	Reason String Maximum length of string is 64	GGGG
33	<RECEIVE_MAX>	Unsigned Integer	Receive Maximum Unsigned 16-bit value	GGGG
34	<TOPIC_ALIAS_MAX>	Unsigned Integer	Topic Alias Maximum Unsigned 16-bit value	GGGG
35	<TOPIC_ALIAS>	Unsigned Integer	Topic Alias Unsigned 16-bit value	GGGG
38	<USER_PROP>	Byte Array	User Property This is a multiple value parameter with an ID range 38.0 to 38.9	GGGG
39	<MAX_PACKET_SZ>	Unsigned Integer	Maximum Packet Size	GGGG

 ©2023-2026

+MQTTPROPRX

Description

This command is used to read or reset the MQTT receive properties.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Any defined properties are added to transmitted MQTT packets. Only properties applicable to the MQTT packet type are added.

Property names and identifiers are as defined in the MQTT Version 5.0 specification (OASIS), section 2.2.2.2.

Any properties received are added to the list of available receive properties which can be queried by this command. Updates to the received properties are announced using the +MQTTPROPRX AEC and can be read via this command.

NOTE	Not all properties listed below may be available in any implementation of this command set.

User property (38) supports multiple entries in the form of key/value pairs.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTPROPRX	Query all configuration elements	GGGG
AT+MQTTPROPRX=<ID>	Query a single element	GGGG
AT+MQTTPROPRX=<ID>,<VAL>	Set a single element ID must be 0	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+MQTTPROPRX:<ID>,<VAL>	Read response
+MQTTPROPRX:<ID>,<KEY>,<VALS>	Read response (key/value)

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value
<KEY>	String	Parameter key
<VALS>	String	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<PAYLOAD_FORMAT_IND>	Unsigned Integer (Read Only)	Payload Format Indicator	GGGG
2	<MSG_EXPIRY_INTERVAL>	Unsigned Integer (Read Only)	Message Expiry Interval	GGGG
3	<CONTENT_TYPE>	String (Read Only)	Content Type Maximum length of string is 64	GGGG
8	<RESP_TOPIC>	String (Read Only)	Response Topic Maximum length of string is 64	GGGG
9	<CORRELATION_DATA>	Byte Array (Read Only)	Correlation Data	GGGG
11	<SUBSCRIPTION_ID>	Unsigned Integer (Read Only)	Subscription Identifier	GGGG
17	<SESSION_EXPIRY_INTERVAL>	Unsigned Integer (Read Only)	Session Expiry Interval	GGGG
18	<ASSIGNED_CLIENT_ID>	String (Read Only)	Assigned Client Identifier Maximum length of string is 64	GGGG
19	<SERVER_KEEP_ALIVE>	Unsigned Integer (Read Only)	Server Keep Alive	GGGG
23	<REQ_PROB_INFO>	Unsigned Integer (Read Only)	Request Problem Information	GGGG
25	<REQ_RESP_INFO>	Unsigned Integer (Read Only)	Request Response Information	GGGG
26	<RESP_INFO>	String (Read Only)	Response Information Maximum length of string is 64	GGGG
28	<SERVER_REF>	String (Read Only)	Server Reference Maximum length of string is 64	GGGG
31	<REASON_STR>	String (Read Only)	Reason String Maximum length of string is 64	GGGG

 ©2023-2026

ID	Name	Type	Description	Sec
33	<RECEIVE_MAX>	Unsigned Integer (Read Only)	Receive Maximum Unsigned 16-bit value	GGGG
34	<TOPIC_ALIAS_MAX>	Unsigned Integer (Read Only)	Topic Alias Maximum Unsigned 16-bit value	GGGG
35	<TOPIC_ALIAS>	Unsigned Integer (Read Only)	Topic Alias Unsigned 16-bit value	GGGG
36	<MAX_QOS>	Unsigned Integer (Read Only)	Maximum QoS Valid range is 0 to 2	GGGG
37	<RETAIN_AVAIL>	Unsigned Integer (Read Only)	Retain Available	GGGG
38	<USER_PROP>	Byte Array (Read Only)	User Property This is a multiple value parameter with an ID range 38.0 to 38.9	GGGG
39	<MAX_PACKET_SZ>	Unsigned Integer (Read Only)	Maximum Packet Size	GGGG
40	<WILDCARD_SUB_AVAIL>	Unsigned Integer (Read Only)	Wildcard Subscription Available	GGGG
41	<SUBSCRIPTION_ID_AVAIL>	Unsigned Integer (Read Only)	Subscription Identifier Available	GGGG
42	<SHARED_SUBSCRIPTION_AVAIL>	Unsigned Integer (Read Only)	Shared Subscription Available	GGGG

+MQTTPROPTXS

Description

This command is used to define which transmit properties are selected.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+MQTTPROPTXS	Query all IDs	GGGG
AT+MQTTPROPTXS=<PROP_ID>	Query a single ID	GGGG
AT+MQTTPROPTXS=<PROP_ID>,<PROP_SEL>	Set a single ID	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<PROP_ID>	Integer	Property Identifier 0 All properties. 1 Payload Format Indicator. 2 Message Expiry Interval. 3 Content Type. 8 Response Topic. 9 Correlation Data. 11 Subscription Identifier. 17 Session Expiry Interval. 23 Request Problem Information. 24 Will Delay Interval. 25 Request Response Information. 31 Reason String. 33 Receive Maximum. 34 Topic Alias Maximum. 35 Topic Alias. 38 User Property. 39 Maximum Packet Size.
<PROP_SEL>	Integer	Property Selected 0 Property not selected. 1 Property is selected.

 ©2023-2026

Response Syntax

Response	Description
+MQTTPROPTXS:<PROP_ID>,<PROP_SEL>	Read response

Response Element Syntax

Element Name	Type	Description
<PROP_ID>	Integer	Property Identifier
<PROP_SEL>	Integer	Property Selected 0 Property not selected. 1 Property is selected.

AEC Reference:

+MQTTCONN

Description

Connection state.

AEC Syntax

AEC	Description
+MQTTCONN:<CONN_STATE>	Connection state

AEC Element Syntax

Element Name	Type	Description
<CONN_STATE>	Integer	MQTT connected state 0 Not connected. 1 Connected.

 ©2023-2026

+MQTTCONNACK

Description

Connection acknowledge.

AEC Syntax

AEC	Description
+MQTTCONNACK:<CONNACK_FLAGS>,<CONN_REASON_CODE>	Connection state

AEC Element Syntax

Element Name	Type	Description
<CONNACK_FLAGS>	Integer	Connect acknowledge flags 0 No session. 1 Session present.
<CONN_REASON_CODE>	Integer	Connect reason code 0 Success. 128 Unspecified error. 129 Malformed packet. 130 Protocol error. 131 Implementation specific error. 132 Unsupported protocol version. 133 Client identifier not valid. 134 Bad username or password. 135 Not authorized. 136 Server unavailable. 137 Server busy. 138 Banned. 140 Bad authentication method. 144 Topic name invalid. 149 Packet too large. 151 Quota exceeded. 153 Payload format invalid. 154 Retain not supported. 155 QoS not supported. 156 Use another server. 157 Server moved. 159 Connection rate exceeded.

 ©2023-2026

+MQTTPUBACK

Description

Publish acknowledge.

AEC Syntax

AEC	Description
+MQTTPUBACK:<PACKET_ID>	Publish acknowledge

AEC Element Syntax

Element Name	Type	Description
<PACKET_ID>	Integer	Packet ID

+MQTTPUBCOMP

Description

Publish complete.

AEC Syntax

AEC	Description
+MQTTPUBCOMP:<PACKET_ID>	Publish complete

AEC Element Syntax

Element Name	Type	Description
<PACKET_ID>	Integer	Packet ID

 ©2023-2026

+MQTTPUBERR

Description

Publish error.

AEC Syntax

AEC	Description
+MQTTPUBERR:<PACKET_ID>	Publish error

AEC Element Syntax

Element Name	Type	Description
<PACKET_ID>	Integer	Packet ID

+MQTTSUB

Description

Subscribe response.

AEC Syntax

AEC	Description
+MQTTSUB:<REASON_CODE>	Subscribe response

AEC Element Syntax

Element Name	Type	Description
<REASON_CODE>	Integer	SUBACK return code 0 Granted QoS 0. 1 Granted QoS 1. 2 Granted QoS 2. 128 Unspecified error. 131 Implementation specific error. 135 Not authorized. 143 Topic filter invalid. 145 Packet identifier in use. 151 Quota exceeded. 158 Shared Subscriptions not supported. 161 Subscription identifiers not supported. 162 Wildcard subscriptions not supported.

 ©2023-2026

+MQTTUNSUB

Description

Unsubscribe response.

AEC Syntax

AEC	Description
+MQTTUNSUB:<RESULT>	Unsubscribe response

AEC Element Syntax

Element Name	Type	Description
<RESULT>	Integer	Result of unsubscribe request 0 Success. 15 No subscription existed. 128 Unspecified error. 131 Implementation specific error. 135 Not authorized. 143 Topic filter invalid. 145 Packet identifier in use.

+MQTTSUBRX

Description

Receive subscribed data.

AEC Syntax

AEC	Description
+MQTTSUBRX:<DUP>,<QOS>,<RETAIN>,<TOPIC_NAME>,<TOPIC_PAYLOAD>	Received subscribed data
+MQTTSUBRX:<DUP>,<QOS>,<RETAIN>,<TOPIC_NAME>,<MSG_ID>,<MSG_LENGTH>	Received subscribed data notification

AEC Element Syntax

Element Name	Type	Description
<DUP>	Integer	Duplicate Message 0 New message. 1 Duplicate message.
<QOS>	Integer	QoS 0 QoS 0. 1 QoS 1. 2 QoS 2.
<RETAIN>	Integer	Retain Message 0 Not retained on the server. 1 Retained on the server.
<TOPIC_NAME>	String	Topic Name
<TOPIC_PAYLOAD>	String	Topic Payload
<MSG_ID>	Integer	Message ID
<MSG_LENGTH>	Integer	Length of message

 ©2023-2026

+MQTTPROPRX

Description

Indicates property has been updated.

AEC Syntax

AEC	Description
+MQTTPROPRX:<PROP_ID>	Property ID updated
+MQTTPROPRX:<PROP_ID>,<VAL>	Property ID updated, with value
+MQTTPROPRX:<PROP_ID>,<KEY>,<VALS>	Property ID updated, with key/value pair

AEC Element Syntax

Element Name	Type	Description
<PROP_ID>	Integer	Property Identifier
<VAL>	Any	Parameter value
<KEY>	String	Parameter key
<VALS>	String	Parameter value

+MQTTPUBSENT

Description

Publish sent (QoS 0).

AEC Syntax

AEC	Description
+MQTTPUBSENT	Publish sent (QoS 0)

 ©2023-2026

+MQTTSUBRXERR

Description

Received subscribed data was dropped.

AEC Syntax

AEC	Description
+MQTTSUBRXERR:<REASON>,<TOPIC_NAME>,<MSG_ID>,<MSG_LENGTH>	Received message dropped

AEC Element Syntax

Element Name	Type	Description
<REASON>	Integer	Reason the received message was dropped 1 Message exceeds the maximum packet size. 2 Insufficient memory to buffer the message. 3 No matching subscription. 4 Message length inconsistent with buffer.
<TOPIC_NAME>	String	Topic Name
<MSG_ID>	Integer	Message ID
<MSG_LENGTH>	Integer	Length of message

Examples:

Example Command Sequence 35: MQTT connection directly over TLS

←	`+TIME:3969510079`	Wait for time for correct TLS operation
→	`AT+TLSC=1,1,"ServerCACert"`	Set the broker CA certificate
←	`OK`	
→	`AT+TLSC=1,6,"mqtt.example.com"`	Set the broker domain
←	`OK`	
→	`AT+MQTTC=1,"mqtt.example.com"`	Set the broker hostname
←	`OK`	
→	`AT+MQTTC=2,8883`	Set the MQTTS port number
←	`OK`	
→	`AT+MQTTC=3,"clientid"`	Set client ID
←	`OK`	
→	`AT+MQTTC=4,"username"`	Set username
←	`OK`	
→	`AT+MQTTC=5,"password"`	Set password
←	`OK`	
→	`AT+MQTTC=7,1`	Use TLSC index 1
←	`OK`	
→	`AT+MQTTCONN=1`	Connect to broker
←	`OK`	
←	`+MQTTPROPRX:34,10`	Received property Topic Alias Max=10
←	`+MQTTPROPRX:33,20`	Received property Receive Max=20
←	`+MQTTCONNACK:0,0`	CONNACK response
←	`+MQTTCONN:1`	Connected

 ©2023-2026

Example Command Sequence 36: MQTT connection over TLS socket

←	`+TIME:3969510551`	Wait for time for correct TLS operation
→	`AT+SOCKO=2`	Open TCP socket
←	`+SOCKO:17604`	
←	`OK`	
→	`AT+TLSC=1,1,"ServerCACert"`	Set the broker CA certificate
←	`OK`	
→	`AT+TLSC=1,6,"mqtt.example.com"`	Set the broker domain
←	`OK`	
→	`AT+MQTTC=3,"clientid"`	Set client ID
←	`OK`	
→	`AT+MQTTC=4,"username"`	Set username
←	`OK`	
→	`AT+MQTTC=5,"password"`	Set password
←	`OK`	
→	`AT+MQTTC=10,17604`	Assign socket ID to MQTT configuration
←	`OK`	
→	`AT+SOCKTLS=17604,1`	Upgrade socket to TLS
←	`OK`	
→	`AT+SOCKBR=17604,"mqtt.example.com",8883`	Bind socket to server
←	`OK`	
←	`+SOCKIND:17604,"1.2.3.4",65333,"5.6.7.8",8883`	Socket connected
←	`+SOCKTLS:17604`	Socket upgraded to TLS
→	`AT+MQTTCONN=1`	Connect to broker
←	`OK`	
←	`+MQTTPROPRX:34,10`	Received property Topic Alias Max=10
←	`+MQTTPROPRX:33,20`	Received property Receive Max=20
←	`+MQTTCONNACK:0,0`	CONNACK response
←	`+MQTTCONN:1`	Connected

Example Command Sequence 37: Subscribe to MQTT topic and receive data (asynchronously)

→	`AT+MQTTSUB="Topic/name",1`	Subscribe to topic with QoS=1
←	`OK`	
←	`+MQTTSUB:1`	Subscription granted QoS=1
←	`+MQTTPROPRX:1,0`	Message property 1 received
←	`+MQTTPROPRX:3,"text/plain"`	Message property 3 received
←	`+MQTTSUBRX:0,1,0,"Topic/name","ExampleText1"`	Message received with data
←	`+MQTTPROPRX:1,0`	Message property 1 received
←	`+MQTTPROPRX:3,"text/plain"`	Message property 3 received
←	`+MQTTSUBRX:0,1,0,"Topic/name","ExampleText2"`	Message received with data

Example Command Sequence 38: Subscribe to MQTT topic and receive data (polled)

→	`AT+MQTTSUB="Topic/name",1`	Subscribe to topic with QoS=1
←	`OK`	
←	`+MQTTSUB:1`	Subscription granted QoS=1
→	`AT+MQTTC=9,0`	Configure receive threshold to zero bytes
←	`OK`	
←	`+MQTTPROPRX:1,0`	Message property 1 received
←	`+MQTTPROPRX:3,"text/plain"`	Message property 3 received
←	`+MQTTSUBRX:0,1,0,"Topic/name",1,12`	Message received, ID=1, length=12
←	`+MQTTPROPRX:1,0`	Message property 1 received
←	`+MQTTPROPRX:3,"text/plain"`	Message property 3 received
←	`+MQTTSUBRX:0,1,0,"Topic/name",2,12`	Message received, ID=2, length=12
→	`AT+MQTTSUBRD="Topic/name",1,12`	Read message ID=1
←	`+MQTTSUBRD:12,"ExampleText1"`	Message ID=1 data
←	`OK`	
→	`AT+MQTTSUBRD="Topic/name",2,12`	Read message ID=2
←	`+MQTTSUBRD:12,"ExampleText2"`	Message ID=2 data
←	`OK`	

 ©2023-2026

NETIF (Module ID = 9)

Introduction

The NETIF module configures and reports the settings of the network interfaces. Two interface configurations are provided, with indices 0 and 1. The WSTA, WAP and DHCP modules each select the interface they use with their NETIF_IDX setting, which defaults to 0. +NETIFC with no parameters reports the range of valid indices.

An interface's configuration can only be written while the interface is down, except L2_MONITOR_MODE and L2_MONITOR_MAX_LEN. Where an address has both a configured and a live value, the configured value is reported while the interface is down and the live value while it is up.

IPv4

The DHCP client (DHCPC_EN) is enabled by default and applies when the interface is used by the station. If DHCP is disabled and no static address is configured, a link-local (ZeroConf) address is used.

IPv6

Stateless address autoconfiguration (IPV6_SLAAC) is enabled by default. The link-local address, global addresses and the gateway learned from router advertisements are reported once the interface is up.

Layer 2 Access

If L2_ONLY is set the interface is brought up without the TCP/IP stack.

L2_MONITOR_MODE selects which received frames (unicast, broadcast, multicast) are reported by the +NETIFRX AEC, optionally truncated to L2_MONITOR_MAX_LEN bytes. Monitored frames are still passed to the TCP/IP stack.

+NETIFTX transmits a complete Ethernet frame, including its header, of up to 1596 bytes. The interface must be up.

Command Reference:

+NETIFC

Description

This command is used to read or set the network interface configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+NETIFC	Query interface list	GGGG
AT+NETIFC=<INTF>	Query all configuration elements	GGGG
AT+NETIFC=<INTF>,<ID>	Query a single element	GGGG
AT+NETIFC=<INTF>,<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<INTF>	Integer	Interface number Valid range is 0 to 1
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+NETIFC:<MIN_IDX>,<MAX_IDX>	Valid index range
+NETIFC:<ID>,<VAL>	Read response

 ©2023-2026

Response Element Syntax

Element Name	Type	Description
<MIN_IDX>	Integer	Lowest valid index
<MAX_IDX>	Integer	Highest valid index
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<NAME>	String (Read Only)	Network interface name Maximum length of string is 16	GGGG
2	<ETHER>	MAC Address (Read Only)	Ethernet MAC address of interface	GGGG
3	<HOSTNAME>	String	Host name of this interface (NetBIOS name), up to 16 characters Maximum length of string is 16	GGGG
10	<DHCPC_EN>	Bool	DHCP client enable 0 Disabled - use static configuration. 1 Enabled - use automatic settings.	GGGG
11	<DHCPC_LEASE_START>	UTC Time (Read Only)	DHCP lease start time, displayed in system time format	GGGG
12	<DHCPC_LEASE_ENDS>	UTC Time (Read Only)	DHCP lease end time, displayed in system time format	GGGG
13	<DHCPC_SRV_ADDR>	IPv4 Address (Read Only)	IP address of DHCP server Unsigned 32-bit value Format of IPv4 address is 'a.b.c.d'	GGGG
20	<L2_ONLY>	Bool	Layer 2 only flag 0 Layer 3 active. 1 Layer 3 not active.	GGGG

ID	Name	Type	Description	Sec
21	<L2_MONITOR_MODE>	Unsigned Integer	L2 monitor mode 0 Off. 1 Unicast. 2 Broadcast. 3 Unicast + Broadcast. 4 Multicast. 5 Unicast + Multicast. 6 Broadcast + Multicast. 7 Unicast + Broadcast + Multicast.	GGGG
22	<L2_MONITOR_MAX_LEN>	Unsigned Integer	Maximum number of bytes of each frame reported in NETIFRX AECs, 0 for the whole frame	GGGG
40	<IP_MASK>	IPv4 Address	IP address and net mask Format of IPv4 address is 'a.b.c.d/m'	GGGG
41	<GATEWAY>	IPv4 Address	IP address of gateway Format of IPv4 address is 'a.b.c.d'	GGGG
60	<IPV6_GLO_ADDR>	IPv6 Address	IPv6 global address Format of IPv6 address is 'a:b:c:d::e:f/m' This is a multiple value parameter with an ID range 60.0 to 60.1	GGGG
61	<IPV6_LL_ADDR>	IPv6 Address (Read Only)	IPv6 link-local address Format of IPv6 address is 'a:b:c:d::e:f/m'	GGGG
62	<IPV6_GATEWAY>	IPv6 Address	IPv6 gateway address Format of IPv6 address is 'a:b:c:d::e:f'	GGGG
63	<IPV6_SLAAC>	Bool	IPv6 SLAAC enable 0 Disabled - use static address only. 1 Enabled - allow autoconfiguration (default).	GGGG

 ©2023-2026

+NETIFTX

Description

This command is used to send L2 data frames.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+NETIFTX=<INTF>,<DATA>	Send an L2 data frame	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<INTF>	Integer	Interface number Valid range is 0 to 1
<DATA>	Byte Array	Data frame data

AEC Reference:

+NETIFRX

Description

Received L2 data frame.

AEC Syntax

AEC	Description
+NETIFRX:<INTF>,<DATA_LENGTH>,<DATA>	L2 data frame

AEC Element Syntax

Element Name	Type	Description
<INTF>	Integer	Interface number Valid range is 0 to 1
<DATA_LENGTH>	Integer	Length of data
<DATA>	Byte Array	Data frame

 ©2023-2026

OTA (Module ID = 10)

Introduction

Over-the-air (OTA) allows the firmware of the DCE to be upgraded. There are two OTA methods: an internal HTTP/HTTPS download based method using OTA (Module ID = 10) commands; or a host controlled flash write based method using NVM (Module ID = 29) commands.

Both methods make use of an alternate firmware image partition in non-volatile memory. The new image is placed in the alternate partition, then verified and activated. Once the new image has been activated, the current image continues to run, but the new image will have higher priority to run at the next boot.

After a successful OTA, activation and reboot, the alternate firmware image partition will contain the previously run image. It is possible to rollback to this previous image using the +OTAINV command. This command verifies the previous image, then invalidates the current running firmware image, so that it cannot be booted again. The current image continues to run, but the previous image will run at the next boot.

NOTE	If the OTA feature is not being utilised, the alternate firmware image partition on the DCE can be used for data storage. The partition supports up to 960kB of data. Data can be erased, written, read and checked via the NVM commands.

OTA Sequence

Each of +OTADL, +OTAVFY, +OTAACT and +OTAINV returns a new operation ID in its response, and every +OTA and +OTAERR AEC which follows carries that ID. Only one OTA operation can run at a time; a new operation is refused until the previous one has completed.

Download

+OTADL=1 downloads the image from the server configured with +OTAC into the alternate partition. It is refused if:

- +OTAINV has been used since the last reset: NVM_LOCKED;
- the STA is not connected: OTA_NO_STA_CONN;
- a download is already running, HOST is not set, or the HTTP client is in use by the HTTP module.

On success the +OTA AEC reports, in order:

1. Erase Done (10.0): the whole alternate partition has been erased.
2. Write Done (10.1): the whole image has been received and written.
3. Verify Done (10.2): the image signature has been verified.

The downloaded image is latent: it does not run until it is activated. A download cannot be cancelled.

Verify and Activate

+OTAVFY verifies the image in the alternate partition, reported by the +OTA AEC with Verify Done (10.2). It is only needed for the NVM method; a download is verified automatically.

+OTAACT verifies the image again and then activates it, reported with Activate Done (10.3). The current image continues to run; the activated image runs from the next reset.

If the image fails verification, +OTAVFY and +OTAACT fail with OTA_VERIFY_FAILED and no AEC is sent. +OTAACT fails with an error if the image has already been activated.

NOTE	A download started after +OTAACT and before the next reset erases the activated image.

Rollback

+OTAINV verifies the image in the alternate partition, which must be bootable: the previous (rollback) image, or an image activated since the last reset, but not a latent image. It then invalidates the current image, reported with Invalidate Done (10.4), and the alternate image runs from the next reset.

After +OTAINV, +NVMER, +NVMWR and +OTADL fail with NVM_LOCKED until the next reset.

After an Error

When the +OTAERR AEC is sent the download stops, and the alternate partition is left partly erased or written. Correct the cause and start the download again; the partition is erased again first. For the NVM method, erase and rewrite the partition.

- OTA_ERROR: an erase or flash write failed, or the connection closed before the whole image was received. The server must send a Content-Length header.
- OTA_PROTOCOL_ERROR: the server returned an HTTP status other than 200, which is included in the AEC (see Error Handling), or a DNS, connection or request error occurred.
- OTA_TLS_ERROR: the TLS session could not be created or the handshake failed, see TLS.
- OTA_IMG_TOO_LARGE: the image is larger than the partition.
- OTA_TIMEOUT: no progress was made within the timeout, see Timeout. Loss of the STA link, or the server closing the connection, during the download is also reported as a timeout.
- OTA_VERIFY_FAILED: the downloaded image failed verification.

After Reset

The +BOOT AEC reports the version of the running firmware. +DI lists the image in each partition; after a successful update the new image is CURRENT and the previous image is ROLLBACK. The RNWF02 does not roll back automatically; use +OTAINV to return to the previous image.

 ©2023-2026

Timeout

The internal OTA download has a timeout set by TIMEOUT, 20 seconds by default. The download is stopped with OTA_TIMEOUT if no progress is made within the timeout. The timer starts once the partition has been erased, and covers the DNS lookup, the connection, the TLS handshake and each gap between received packets. If the server sends a Keep-Alive header with a timeout, that value is used instead.

NOTE	It may be necessary to use a timeout greater than the default if the network conditions are poor, or the server has a high latency.

Error Handling

The OTA module reports errors via the +OTAERR asynchronous event. For protocol-specific errors (such as HTTP status codes indicating authentication failures, missing resources, or server errors), the extended error format includes the protocol type and the protocol-specific error code:

```
+OTAERR:<OTA_OP_ID>,<ERROR_CODE>,<PROTOCOL>,<PROTOCOL_STATUS>
```

For example, if the OTA server returns HTTP 401 (Unauthorized), the error will be reported as:

```
+OTAERR:1,10.7,"Protocol Error",1,401
```

Where:

- 1 is the operation ID
- 10.7 indicates Protocol Error
- 1 indicates the HTTP protocol
- 401 is the HTTP status code

Common HTTP status codes that may be reported:

- 401 - Unauthorized (authentication required)
- 403 - Forbidden (access denied)
- 404 - Not Found (firmware file does not exist)
- 500 - Internal Server Error
- 503 - Service Unavailable

Errors detected when an OTA command is issued, such as OTA_NO_STA_CONN, or a failed verification by +OTAVFY, +OTAACT or +OTAINV, are returned as the command's status rather than by an AEC.

Troubleshooting

Further firmware image information can be gathered with the +DI command which can help with debugging available images on the DCE.

Internal HTTP/S OTA Example

For OTA using the internal HTTP/HTTPS download based method, a HTTP server is required to serve the firmware image file. The DTE configures the download, then downloads and activates the new image. Image verification is done internally when the download completes.

HTTP Configuration

→	`AT+OTAC=2,"example.com"`	Set host
←	`OK`	
→	`AT+OTAC=3,80`	Set port
←	`OK`	
→	`AT+OTAC=4,"/firmware/"`	Set path
←	`OK`	
→	`AT+OTAC=5,"rnwf02_ota.bin"`	Set filename of the released OTA image
←	`OK`	

HTTPS Configuration

To use HTTPS a TLS configuration is required.

→	`AT+OTAC=7,1`	Set TLS configuration
←	`OK`	

Download and Activation

→	`AT+OTADL=1`	Request OTA download
←	`+OTADL:1`	
←	`OK`	
←	`+OTA:1,10.0,"Erase Done"`	Partition erased AEC
←	`+OTA:1,10.1,"Write Done"`	Firmware image written AEC
←	`+OTA:1,10.2,"Verify Done"`	Firmware image verified AEC
→	`AT+OTAACT`	Request to activate firmware
←	`+OTAACT:2`	
←	`OK`	
←	`+OTA:2,10.3,"Activate Done"`	Activation successful AEC

 ©2023-2026

NVM OTA Example

For OTA using the host controlled flash write based method, the DTE accesses the alternate firmware image partition of the DCE with flash read/write/erase commands. This feature allows extra flexibility with the data transport method, and ability to implement more advanced OTA logic if required - such as piecemeal updating with persistence between power cycles.

Once the DTE has written the image to the DCE using NVM (Module ID = 29) commands, the DTE should verify and activate the new image using OTA (Module ID = 10) commands.

→	`AT+NVMER=0,240`	Erase the alternate partition (240 sectors)
←	`OK`	
←	`+NVMER`	Erase complete AEC
→	`AT+NVMWR=0,512,[FFFFFFFF0101E0005048434D0300000000000300FFFFFFFF…]`	Write firmware image data
←	`OK`	
→	`AT+NVMWR=0,512,[…]`	
←	`OK`	
	`…`	Continue until writing finished
→	`AT+OTAVFY`	Request to verify firmware image
←	`+OTAVFY:1`	
←	`OK`	
←	`+OTA:1,10.2,"Verify Done"`	Firmware image verified AEC
→	`AT+OTAACT`	Request to activate firmware
←	`+OTAACT:2`	
←	`OK`	
←	`+OTA:2,10.3,"Activate Done"`	Activation successful AEC

Command Reference:

+OTAC

Description

This command is used to read or set the OTA configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+OTAC	Query all configuration elements	GGGG
AT+OTAC=<ID>	Query a single element	GGGG
AT+OTAC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+OTAC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<PROTOCOL>	Integer (Read Only)	Protocol type 1 HTTP.	GGGG
2	<HOST>	String IPv4 Address IPv6 Address	Host domain name or IP address Maximum length of string is 255 Format of IPv4 address is 'a.b.c.d' Format of IPv6 address is 'a:b:c:d::e:f'	GGGG
3	<PORT>	Unsigned Integer	Server port (e.g. 80 for HTTP, 443 for HTTPS); must be set	GGGG
4	<PATH>	String	Path to firmware Maximum length of string is 1024	GGGG
5	<FILE>	String	Firmware file Maximum length of string is 1024	GGGG
6	<TIMEOUT>	Unsigned Integer	Timeout in seconds Positive unsigned 8-bit value	GGGG
7	<TLS_CONF>	Integer	TLS configuration index (see +TLSC). 0 disables TLS Valid range is 0 to 4	GGGG
10	<SOCKET_ID>	Unsigned Integer	Socket ID	GGGG

+OTADL

Description

This command allows the downloading of firmware to the DCE.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+OTADL=<STATE>	Download OTA image	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<STATE>	Integer	Enable the OTA download feature 1 Use configuration from +OTAC command.

Response Syntax

Response	Description
+OTADL:<OTA_OP_ID>	OTA download response

Response Element Syntax

Element Name	Type	Description
<OTA_OP_ID>	Integer	Operation ID Positive unsigned 8-bit value

 ©2023-2026

+OTAVFY

Description

This command verifies the OTA firmware image.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+OTAVFY	Verify OTA image	GGGG

Response Syntax

Response	Description
+OTAVFY:<OTA_OP_ID>	OTA verify response

Response Element Syntax

Element Name	Type	Description
<OTA_OP_ID>	Integer	Operation ID Positive unsigned 8-bit value

+OTAACT

Description

This command activates the OTA firmware image.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+OTAACT	Activate OTA image	GGGG

Response Syntax

Response	Description
+OTAACT:<OTA_OP_ID>	OTA activate response

Response Element Syntax

Element Name	Type	Description
<OTA_OP_ID>	Integer	Operation ID Positive unsigned 8-bit value

 ©2023-2026

+OTAINV

Description

This command will invalidate the current running firmware image. Invalidation does not delete the current running firmware image.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+OTAINV	Invalidate current running image	GGGG

Response Syntax

Response	Description
+OTAINV:<OTA_OP_ID>	OTA invalidate response

Response Element Syntax

Element Name	Type	Description
<OTA_OP_ID>	Integer	Operation ID Positive unsigned 8-bit value

AEC Reference:

+OTA

Description

OTA state.

AEC Syntax

AEC	Description
+OTA:<OTA_OP_ID>,<OTA_STATE>	OTA status

AEC Element Syntax

Element Name	Type	Description
<OTA_OP_ID>	Integer	Operation ID Positive unsigned 8-bit value
<OTA_STATE>	Integer	OTA status code One of the OTA module status codes: ERASE_DONE WRITE_DONE VERIFY_DONE ACTIVATE_DONE INVALIDATE_DONE See Status Response Codes.

 ©2023-2026

+OTAERR

Description

OTA error.

AEC Syntax

AEC	Description
+OTAERR:<OTA_OP_ID>,<ERROR_CODE>	OTA error
+OTAERR:<OTA_OP_ID>,<ERROR_CODE>,<PROTOCOL>,<PROTOCOL_STATUS>	OTA protocol status

AEC Element Syntax

Element Name	Type	Description
<OTA_OP_ID>	Integer	Operation ID Positive unsigned 8-bit value
<ERROR_CODE>	Integer	Error code See Status Response Codes
<PROTOCOL>	Integer	Protocol type 1 HTTP protocol.
<PROTOCOL_STATUS>	Integer	Protocol-specific status code For HTTP protocol this is the server response code, valid range is 100 to 599

Examples:

Example Command Sequence 43: Internal OTA HTTP Configuration

→	`AT+OTAC=2,"example.com"`	Set host
←	`OK`	
→	`AT+OTAC=3,80`	Set port
←	`OK`	
→	`AT+OTAC=4,"/firmware/"`	Set path
←	`OK`	
→	`AT+OTAC=5,"rnwf02_ota.bin"`	Set filename of the released OTA image
←	`OK`	

Example Command Sequence 44: Internal OTA HTTPS Configuration

→	`AT+OTAC=7,1`	Set TLS configuration
←	`OK`	

Example Command Sequence 45: Internal OTA

→	`AT+OTADL=1`	Request OTA download
←	`+OTADL:1`	
←	`OK`	
←	`+OTA:1,10.0,"Erase Done"`	Partition erased AEC
←	`+OTA:1,10.1,"Write Done"`	Firmware image written AEC
←	`+OTA:1,10.2,"Verify Done"`	Firmware image verified AEC
→	`AT+OTAACT`	Request to activate firmware
←	`+OTAACT:2`	
←	`OK`	
←	`+OTA:2,10.3,"Activate Done"`	Activation successful AEC

 ©2023-2026

Example Command Sequence 46: NVM OTA

→	`AT+NVMER=0,240`	Erase the alternate partition (240 sectors)
←	`OK`	
←	`+NVMER`	Erase complete AEC
→	`AT+NVMWR=0,512,[FFFFFFFF0101E0005048434D0300000000000300FFFFFFFF…]`	Write firmware image data
←	`OK`	
→	`AT+NVMWR=0,512,[…]`	
←	`OK`	
	`…`	Continue until writing finished
→	`AT+OTAVFY`	Request to verify firmware image
←	`+OTAVFY:1`	
←	`OK`	
←	`+OTA:1,10.2,"Verify Done"`	Firmware image verified AEC
→	`AT+OTAACT`	Request to activate firmware
←	`+OTAACT:2`	
←	`OK`	
←	`+OTA:2,10.3,"Activate Done"`	Activation successful AEC

PING (Module ID = 11)

Introduction

The PING module sends a single ICMP echo request to a remote host and reports the round trip time.

The target may be an IPv4 or IPv6 address, or a host name which is first resolved using DNS. The +PING command completes once the request has been sent; the result is reported later by the +PING AEC, which carries the address and round trip time, or by the +PINGERR AEC if no reply is received within one second.

More than one request may be outstanding at a time.

 ©2023-2026

Command Reference:

+PING

Description

This command sends a ping (ICMP Echo Request) to the target address.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+PING=<TARGET_ADDR>	Ping a target address	GGGG
AT+PING=<TARGET_ADDR>,<PROTOCOL_VERSION>	Ping a target address (specifying IP protocol)	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<TARGET_ADDR>	Integer String Byte Array	IP address or host name of target
<PROTOCOL_VERSION>	Integer	IP protocol version 4 IPv4. 6 IPv6.

AEC Reference:

+PING

Description

Successful ping.

AEC Syntax

AEC	Description
+PING:<IP_ADDRESS>,<RTT>	Ping success response

AEC Element Syntax

Element Name	Type	Description
<IP_ADDRESS>	String	IP address of the target
<RTT>	Integer	Round trip time (in milliseconds)

 ©2023-2026

+PINGERR

Description

Error.

AEC Syntax

AEC	Description
+PINGERR:<ERROR_CODE>	Ping error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	Integer	Error code See Status Response Codes

RST (Module ID = 12)

Introduction

The +RST command performs a full software reset of the RNWF02. The OK response is sent before the reset takes place.

All runtime state and configuration is lost, including network connections, the system time and scan results. After the reset the configuration is restored from the configuration archive file "autoexec" if one has been stored, see CFG.

 ©2023-2026

Command Reference:

+RST

Description

This command is used to reset the DCE.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+RST	Reset target	GGGG

SNTP (Module ID = 13)

Introduction

The SNTP module configures the NTP client which sets the system time. The client is enabled by default.

The source of the NTP server is selected by NTP_STATIC:

- AUTO: the NTP server supplied by the DHCP server is used. If none is supplied the client does not start.
- MANUAL: the server configured by NTP_SVR is used, this may be a host name or an IPv4 or IPv6 address.

The client starts when the station interface obtains an IP address. Changes made to +SNTPC while connected are applied immediately.

Each successful synchronisation sets the system time and sends a +TIME AEC. Server timeouts and protocol errors are reported by the +SNTPERR AEC.

While the station is connected and the client is enabled, reading NTP_SVR returns the address of the server in use rather than the configured value.

 ©2023-2026

Command Reference:

+SNTPC

Description

This command is used to read or set the SNTP configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SNTPC	Query all configuration elements	GGGG
AT+SNTPC=<ID>	Query a single element	GGGG
AT+SNTPC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+SNTPC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<NTP_ENABLED>	Bool	Network Time (NTP) client function 0 Disabled. 1 Enabled.	GGGG
2	<NTP_STATIC>	Bool	NTP configuration mode 0 Auto-assigned by network. 1 Manually configured by user.	GGGG
3	<NTP_SVR>	String IPv4 Address IPv6 Address	The address/name of NTP server Maximum length of string is 128 Format of IPv4 address is 'a.b.c.d' Format of IPv6 address is 'a:b:c:d::e:f'	GGGG

 ©2023-2026

AEC Reference:

+SNTPERR

Description

SNTP error.

AEC Syntax

AEC	Description
+SNTPERR:<ERROR_CODE>	SNTP error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	Integer	Error code See Status Response Codes

SOCKET (Module ID = 14)

Introduction:

Socket Lifecycle

Opening

+SOCKO opens a UDP or TCP socket and returns its socket ID. The IP version (4 or 6) is optional; if it is not given IPv4 is used when the socket is bound. Socket IDs are allocated sequentially and are not reused until the 16-bit ID wraps.

A network connection is not needed to open a socket, but a TCP connection needs a local IP address. Up to 30 sockets can be open at once, after which +SOCKO fails with NO_FREE_SOCKETS. The TCP/IP stack provides 10 TCP and 10 UDP sockets, which are shared with the MQTT and HTTP modules.

TCP Client

1. +SOCKBR connects the socket to a remote IP address or host name. The OK response indicates the connection has started.
2. The +SOCKIND AEC reports that the connection is established, with the local and remote addresses and ports.

If the connection is not established within 10 seconds, including when it is refused by the remote host, the +SOCKERR AEC reports Socket Bind Failed. If the host name cannot be resolved, +SOCKERR reports the DNS error. The socket ID remains allocated in both cases and must be closed with +SOCKCL.

TCP Server

1. +SOCKBL listens on a local port. PEND_SKTS (default 1) sets the number of connections which can be open at once; each one reserves a TCP stack socket when +SOCKBL is issued.
2. Each accepted connection is given a new socket ID, reported by the +SOCKIND AEC. The new socket inherits the listening socket's +SOCKC and TLS settings. Data is read and written using the new socket ID.

Closing an accepted socket frees its connection slot for the next connection. Closing the listening socket stops new connections; sockets which are already connected stay open.

See example Opening a TCP server socket.

 ©2023-2026

UDP

- +SOCKBL binds the socket to a local port, and datagrams from any remote host are received.
- +SOCKBR sets the default destination used by +SOCKWR.
- +SOCKWRTO sends to the given address. It binds the socket if needed, and sets the default destination if none is set.
- +SOCKBM joins a multicast group.

TLS

+SOCKTLS applies a TLS configuration (1 to 4, see +TLSC) to a TCP socket:

- if it is issued before +SOCKBR or +SOCKBL, the handshake starts once the connection is established;
- if it is issued on a connected socket, the connection is upgraded to TLS.

The +SOCKTLS AEC reports that the handshake has completed; data cannot be written until then. On a server, each accepted socket performs its own handshake and reports +SOCKTLS with its own socket ID. If the handshake fails, the +SOCKERR AEC reports Socket TLS Failed.

Writing

- +SOCKWR writes to a TCP socket, or to a UDP socket with a default destination, up to 1460 bytes.
- +SOCKWRTO writes to a UDP socket, up to 1472 bytes.
- If there is not enough transmit buffer space the command fails with SOCKET_NOT_READY; retry later.

The optional SEQ_NUM parameter (SEQ_NUM, SEQ_NUM) detects lost or repeated writes:

- the first write which includes it sets the expected sequence number;
- every later ASCII or hex write must include the expected sequence number, which advances by the number of bytes written, otherwise it fails with SOCKET_SEQUENCE_ERROR.

In binary mode the data is sent as raw binary after the '#' prompt, see Raw Binary - From DTE to DCE. The DCE waits for the data with no timeout; send the +++ escape sequence to end the transfer early.

Reading

Received data is announced by the +SOCKRXT (TCP) or +SOCKRXU (UDP) AEC. The four methods of reading it are described in Reading Socket Data:

Method	Enabled by	Data delivered by	Flow control	UDP datagram larger than read
Basic	Default	+SOCKRD OUTPUT_MODE 1 or 2	The DTE reads when ready	The rest of the datagram is discarded
In-band	+SOCKRD OUTPUT_MODE 3	+SOCKRD, with the pending data information in the response	The DTE reads when ready; no AECs until all data is read	The rest of the datagram is discarded
Simple asynchronous	ASYNC_MAX_FRM_SZ	+SOCKRXT/+SOCKRXU with the data	None, data is sent as it arrives	The rest of the datagram is lost
Acknowledged asynchronous	ASYNC_MAX_FRM_SZ and ASYNC_WIN_SZ	+SOCKRXT/+SOCKRXU with the data and a sequence number	Window advanced by ASYNC_NEXT_SN	The rest of the datagram is lost

+SOCKRDBUF reads part of a UDP datagram without discarding the rest, see example Reading partial datagram.

Closing

+SOCKCL closes a socket and releases its socket ID.

The +SOCKCL AEC reports that a connected TCP socket was closed by the remote host or timed out. It is not sent when the DTE closes the socket with +SOCKCL. The socket ID remains allocated until +SOCKCL is issued. After the remote host closes the connection, data which has not been read can still be read; after a reset it is lost.

When the network interface goes down, for example on +WSTALD or +WSTA=0, sockets are not closed and no AEC is sent. The DTE should close them with +SOCKCL.

 ©2023-2026

Reading Socket Data

There are four methods of reading data from a socket, two polled and two asynchronous methods. Polling involves the +SOCKRD command and +SOCKRXT/+SOCKRXU AECs. Asynchronous methods involve the +SOCKRXT/+SOCKRXU AECs.

Basic SOCKRD/AEC method:

+SOCKRXT or +SOCKRXU AECs from the DCE notify the DTE that data is available to be read. The DTE then sends a +SOCKRD command with OUTPUT_MODE of 1 or 2 to read a number of bytes from the DCE. After the +SOCKRD response further +SOCKRXT or +SOCKRXU AECs will update the available pending data.

See example Basic socket read

←	`+SOCKRXT:6,15`	Indicates 15 bytes are present
→	`AT+SOCKRD=6,1,10`	Read 10 bytes in ASCII mode
←	`+SOCKRD:6,10,"1234567890"`	10 bytes returned
←	`OK`	
←	`+SOCKRXT:6,7`	An additional 2 bytes have been received

In-band SOCKRD/AEC method:

As with the basic +SOCKRD method, +SOCKRXT or +SOCKRXU announce the presence of data to be read from a SOCKET. If +SOCKRD OUTPUT_MODE 3 is used the resulting +SOCKRD response will include the pending data information in the response and no further AECs will be sent until all data has been consumed.

See example In-band socket read

←	`+SOCKRXT:6,20`	Indicates 20 bytes have been received
→	`AT+SOCKRD=6,3,10`	Read 10 bytes in ASCII mode with extended information
←	`+SOCKRD:6,10,10,"1234567890"`	10 bytes returned, indicates 10 bytes remain
←	`OK`	
→	`AT+SOCKRD=6,3,4`	Read 4 bytes in ASCII mode with extended information
←	`+SOCKRD:6,4,6,"abcd"`	4 bytes returned, indicates 6 bytes remain
←	`OK`	

Simple Asynchronous method:

By configuring the +SOCKC ASYNC_MAX_FRM_SZ parameter of a socket the DTE can set the maximum frame size of data to be received asynchronously from the DCE.

NOTE	For UDP sockets, if the ASYNC_MAX_FRM_SZ value is smaller than the size of a datagram, the remaining data from the datagram will be lost.

When the ASYNC_MAX_FRM_SZ parameter is set, the DCE will send received socket data to the DTE without polling from +SOCKRD, the data is included in an expanded form of +SOCKRXT or +SOCKRXU AEC.

See example Simple asynchronous socket read

→	`AT+SOCKC=6,102,1500`	Configure socket ASYNC_MAX_FRM_SZ to be 1500 bytes
←	`OK`	
←	`+SOCKRXT:6,10,"1234567890"`	10 bytes received
←	`+SOCKRXT:6,4,"abcd"`	4 bytes received

 ©2023-2026

Acknowledged Asynchronous method:

Extending the simple asynchronous method, if the ASYNC_WIN_SZ parameter of a socket is also set the DCE will use a sliding window protocol to send data to the DTE. In addition to the data included in the +SOCKRXT or +SOCKRXU AEC a sequence number is added, this represents the sequence number of the first byte of the data provided.

The DCE will continue to send data via AEC until ASYNC_WIN_SZ bytes have been transmitted, after which transmission will cease. To resume transmission the DTE must update the value of +SOCKC parameter ASYNC_NEXT_SN to acknowledge reception of data from the DCE, once this value has been updated the DCE will continue to send more data until ASYNC_NEXT_SN + ASYNC_WIN_SZ bytes have been sent.

The DTE does not need to wait until transmission has ceased before updating ASYNC_NEXT_SN, this can be done at any time and will ensure the transmission window remains open for subsequent +SOCKRXT or +SOCKRXU AECs.

Using this method the DTE can ensure that buffer space is available for receiving data transmission while still receiving data as it arrives at the DCE.

See example Acknowledged asynchronous socket read

→	`AT+SOCKC=6,102,1500`	Configure socket ASYNC_MAX_FRM_SZ to be 1500 bytes
←	`OK`	
→	`AT+SOCKC=6,101,8`	Configure socket ASYNC_WIN_SZ to be 8 bytes
←	`OK`	
←	`+SOCKRXT:6,8,0,"12345678"`	8 bytes received, sequence number 0 to 7
→	`AT+SOCKC=6,100,8`	Configure ASYNC_NEXT_SN to be 8
←	`OK`	
←	`+SOCKRXT:6,8,8,"90abcdef"`	8 bytes received, sequence number 8 to 15
→	`AT+SOCKC=6,100,12`	Configure ASYNC_NEXT_SN to be 12
←	`OK`	
←	`+SOCKRXT:6,4,16,"ghij"`	4 bytes received, sequence number 16 to 19

Socket Errors

Errors which occur after a command has completed are reported by the +SOCKERR AEC. The error code identifies the cause:

- Socket Bind Failed: a TCP connection was not established within 10 seconds of binding to the remote address, or the bind itself failed.
- Socket TLS Failed: the TLS handshake failed or could not be started.
- A DNS error code: the remote host name could not be resolved.

Command Reference:

+SOCKO

Description

This command is used to open a new socket.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKO=<PROTOCOL>	Open a socket	GGGG
AT+SOCKO=<PROTOCOL>,<PROTOCOL_VERSION>	Open a socket (specifying IP protocol)	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<PROTOCOL>	Integer	The protocol to use 1 UDP. 2 TCP.
<PROTOCOL_VERSION>	Integer	IP protocol version 4 IPv4. 6 IPv6.

Response Syntax

Response	Description
+SOCKO:<SOCK_ID>	Socket open response

Response Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value

 ©2023-2026

+SOCKBL

Description

This command is used to bind a socket to a local port.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKBL=<SOCK_ID>,<LCL_PORT>	Bind to a local port	GGGG
AT+SOCKBL=<SOCK_ID>,<LCL_PORT>,<PEND_SKTS>	Bind to a local port with number of connections	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<LCL_PORT>	Integer	The local port number to use Positive unsigned 16-bit value
<PEND_SKTS>	Integer	Maximum number of simultaneous connections, limited by the number of free TCP sockets Valid range is 1 to 10

+SOCKBR

Description

This command is used to bind a socket to a remote address.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKBR=<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>	Bind to a remote port	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<RMT_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	The address of the remote device
<RMT_PORT>	Integer	The port number on the remote device Positive unsigned 16-bit value

 ©2023-2026

+SOCKBM

Description

This command is used to bind a socket to a multicast group.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKBM=<SOCK_ID>,<MCAST_ADDR>,<MCAST_PORT>	Bind to a multicast port	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<MCAST_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	The address of the multicast group
<MCAST_PORT>	Integer	The port number of the multicast group Positive unsigned 16-bit value

+SOCKTLS

Description

This command is used to enable TLS on a socket.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKTLS=<SOCK_ID>,<TLS_CONF>	Apply TLS configuration	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<TLS_CONF>	Integer	TLS configuration index (see +TLSC) Valid range is 1 to 4

Response Syntax

Response	Description
+SOCKTLS:<SOCK_ID>	TLS success

Response Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value

 ©2023-2026

+SOCKWR

Description

This command is used to send data over a socket that is bound to a remote address and port number.

If no DATA parameter is supplied to the +SOCKWR command the DCE will enter raw binary mode and will remain in that mode until the specified length of binary data has been received from the DTE.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKWR=<SOCK_ID>,<LENGTH>,<SEQ_NUM>,<DATA>	Socket write with data and sequence number	GGGG
AT+SOCKWR=<SOCK_ID>,<LENGTH>,<DATA>	Socket write with data	GGGG
AT+SOCKWR=<SOCK_ID>,<LENGTH>	Socket write, data to follow	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<LENGTH>	Integer	The length of the data to send Valid range is 0 to 1460
<SEQ_NUM>	Integer	Sequence number of first byte. Once used, required on every subsequent ASCII or hex write Unsigned 16-bit value The first write that includes a sequence number sets the starting sequence number for the socket. After that, every write on the socket must include the sequence number, equal to the previous sequence number plus the previous data length (modulo 65536), otherwise the write fails with Socket Sequence Error. Sockets on which a sequence number has never been used are not affected.
<DATA>	String Byte Array	The data to send Maximum length of string is 1460

+SOCKWRTO

Description

This command is used to send data to an arbitrary destination using the connectionless UDP protocol.

If no DATA parameter is supplied to the +SOCKWRTO command the DCE will enter raw binary mode and will remain in that mode until the specified length of binary data has been received from the DTE.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKWRTO=<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<LENGTH>,<SEQ_NUM>,<DATA>	Socket write to socket with data and sequence number	GGGG
AT+SOCKWRTO=<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<LENGTH>,<DATA>	Socket write to socket with data	GGGG
AT+SOCKWRTO=<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<LENGTH>	Socket write to socket, data to follow	GGGG

 ©2023-2026

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<RMT_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	The address of the remote device
<RMT_PORT>	Integer	The port number on the remote device Positive unsigned 16-bit value
<LENGTH>	Integer	The length of the data to send Valid range is 0 to 1472
<SEQ_NUM>	Integer	Sequence number of first byte. Once used, required on every subsequent ASCII or hex write Unsigned 16-bit value The first write that includes a sequence number sets the starting sequence number for the socket. After that, every write on the socket must include the sequence number, equal to the previous sequence number plus the previous data length (modulo 65536), otherwise the write fails with Socket Sequence Error. Sockets on which a sequence number has never been used are not affected.
<DATA>	String Byte Array	The data to send Maximum length of string is 1472

+SOCKRD

Description

This command is used to read data from a socket.

Two AECs present notification of data received by the DCE:

- +SOCKRXU indicates UDP data has been received.
- +SOCKRXT indicates TCP data has been received.

The DTE is responsible for retrieving the datagram/stream data via the +SOCKRD command.

For TCP sockets the DCE will indicate, via +SOCKRXT, the number of bytes of data which are currently available for reading via the +SOCKRD command. The DCE may issue multiple +SOCKRXT AECs as data is received. When requesting data via the +SOCKRD command the DTE may receive less data than request, the number of bytes provided by the DCE will be declared in the +SOCKRD response before the data is presented. The DTE may request less data than that declared by the +SOCKRXT AEC as being available, the remaining data will be available for subsequent reading.

For UDP sockets the DCE will indicate, via +SOCKRXU, the number of bytes of data which were received in the oldest datagram received by the DCE. Only a single +SOCKRXU will be issued by the DCE even if subsequent UDP datagrams are received. Reading data from the UDP socket via the +SOCKRD command will read and discard the current datagram, if less data is requested than was indicated by the +SOCKRXU AEC the remaining unread data in the datagram will be discarded.

Output Modes

- **1**: the data is returned in the response as a string or hexadecimal string, see Strings.
- **2**: the DCE leaves terminal mode and sends the data as raw binary, see Raw Binary - From DCE to DTE. A <LENGTH> of 0 continues sending data until the escape sequence is received.
- **3**: as mode 1, with the remaining pending data (and for UDP, the number of pending datagrams and the sender's address and port) included in the response. No further receive AECs are sent until the pending data has been read.

The <LENGTH> is limited to the amount of data available. For TCP sockets a <LENGTH> of 0 returns no data; for UDP sockets it reads a zero-length datagram.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKRD=<SOCK_ID>,<OUTPUT_MODE>,<LENGTH>	Socket read	GGGG

 ©2023-2026

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<OUTPUT_MODE>	Integer	Output format of the data 1 Data returned in the response. 2 Raw binary (AT interface only). 3 Data returned in the response, with pending data information.
<LENGTH>	Integer	Maximum number of bytes to read

Response Syntax

Response	Description
+SOCKRD:<SOCK_ID>,<LENGTH>,<DATA>	Socket read response
+SOCKRD:<SOCK_ID>,<LENGTH>,<PEND_DATA>,<DATA>	Socket read response (TCP extended)
+SOCKRD:<SOCK_ID>,<LENGTH>,<PEND_DATA>,<PEND_DATAGRAM>,<RMT_ADDR>,<RMT_PORT>,<DATA>	Socket read response (UDP extended)

Response Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value
<LENGTH>	Integer	Number of bytes
<DATA>	Byte Array	Socket data
<PEND_DATA>	Integer	Number of pending bytes
<PEND_DATAGRAM>	Integer	Number of pending datagrams
<RMT_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	Remote address
<RMT_PORT>	Integer	Remote port Positive unsigned 16-bit value

+SOCKRDBUF

Description

This command is used to read data from a socket buffer (UDP only).

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKRDBUF=<SOCK_ID>,<OUTPUT_MODE>,<LENGTH>	Socket read	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<OUTPUT_MODE>	Integer	Output format of the data 1 Data returned in the response. 2 Raw binary (AT interface only). 3 Data returned in the response, with pending data information.
<LENGTH>	Integer	Maximum number of bytes to read

Response Syntax

Response	Description
+SOCKRDBUF:<SOCK_ID>,<LENGTH>,<DATA>	Socket read response
+SOCKRDBUF:<SOCK_ID>,<LENGTH>,<PEND_DATA>,<PEND_DATAGRAM>,<RMT_ADDR>,<RMT_PORT>,<DATA>	Socket read response (UDP extended)

 ©2023-2026

Response Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value
<LENGTH>	Integer	Number of bytes
<DATA>	Byte Array	Socket data
<PEND_DATA>	Integer	Number of pending bytes
<PEND_DATAGRAM>	Integer	Number of pending datagrams
<RMT_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	Remote address
<RMT_PORT>	Integer	Remote port Positive unsigned 16-bit value

+SOCKCL

Description

This command is used to close a socket.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKCL=<SOCK_ID>	Close socket	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value

Response Syntax

Response	Description
+SOCKCL:<SOCK_ID>	Socket close response

Response Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value

 ©2023-2026

+SOCKLST

Description

This command is used to present a list of the DCE's open sockets/connections.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKLST	List all sockets	GGGG
AT+SOCKLST=<SOCK_ID>	List socket	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value

Response Syntax

Response	Description
+SOCKLST:<SOCK_ID>,<PROTOCOL>,<RMT_ADDR>,<RMT_PORT>,<LCL_PORT>	Socket list response

Response Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value
<PROTOCOL>	Integer	The protocol in use 1 UDP. 2 TCP.
<RMT_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	Remote address
<RMT_PORT>	Integer	Remote port Positive unsigned 16-bit value
<LCL_PORT>	Integer	Local port Positive unsigned 16-bit value

+SOCKC

Description

This command is used to read or set the socket configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SOCKC	Query range of SOCK_ID	GGGG
AT+SOCKC=<SOCK_ID>	Query all configuration elements	GGGG
AT+SOCKC=<SOCK_ID>,<ID>	Query a single element	GGGG
AT+SOCKC=<SOCK_ID>,<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SOCK_ID>	Integer	Socket ID, as returned by SOCKO Positive unsigned 16-bit value
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+SOCKC:<MIN_IDX>,<MAX_IDX>	Valid index range
+SOCKC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<MIN_IDX>	Integer	Lowest valid index
<MAX_IDX>	Integer	Highest valid index
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

ID	Name	Type	Description	Sec
1	<SO_LINGER>	Bool	Socket linger enable/disable (TCP) 0 Disable. 1 Enable.	GGGG
2	<SO_NODELAY>	Bool	Socket no delay (TCP) 0 Disable. 1 Enable.	GGGG
3	<SO_KEEPALIVE>	Unsigned Integer	Socket keep alive timeout in milliseconds (TCP), 0 disables Unsigned 16-bit value	GGGG
4	<SO_RCVBUF>	Unsigned Integer	Socket receive buffer size (TCP) Valid range is 1460 to 32768	GGGG
5	<SO_SNDBUF>	Unsigned Integer	Socket transmit buffer size (TCP) Valid range is 1460 to 32768	GGGG
32	<IP_TOS>	Unsigned Integer	Socket Type of Service (TCP/UDP) 0x00 Normal. 0x04 Reliability. 0x08 Throughput. 0x10 Low delay.	GGGG
100	<ASYNC_NEXT_SN>	Unsigned Integer	Async next sequence number (acknowledgement) See Reading Socket Data.	GGGG
101	<ASYNC_WIN_SZ>	Unsigned Integer	Async window size in bytes, 0 for no window. Default is 0 See Reading Socket Data.	GGGG
102	<ASYNC_MAX_FRM_SZ>	Unsigned Integer	Async maximum frame size in bytes, 0 disables async delivery. Default is 0 See Reading Socket Data.	GGGG

AEC Reference:

+SOCKIND

Description

Socket established.

AEC Syntax

AEC	Description
+SOCKIND:<SOCK_ID>,<LCL_ADDR>,<LCL_PORT>,<RMT_ADDR>,<RMT_PORT>	TCP connection established

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value
<LCL_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	Local address
<LCL_PORT>	Integer	Local port Positive unsigned 16-bit value
<RMT_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	Remote address
<RMT_PORT>	Integer	Remote port Positive unsigned 16-bit value

 ©2023-2026

+SOCKRXT

Description

TCP receive.

AEC Syntax

AEC	Description
+SOCKRXT:<SOCK_ID>,<PEND_DATA>	TCP receive (polled)
+SOCKRXT:<SOCK_ID>,<LENGTH>,<DATA>	TCP receive (async)
+SOCKRXT:<SOCK_ID>,<LENGTH>,<SEQ_NUM>,<DATA>	TCP receive (async reliable)

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value
<PEND_DATA>	Integer	Number of pending bytes
<LENGTH>	Integer	Number of bytes
<DATA>	Byte Array	Socket data
<SEQ_NUM>	Integer	Sequence number of first byte Unsigned 16-bit value

+SOCKRXU

Description

UDP receive.

AEC Syntax

AEC	Description
+SOCKRXU:<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<PEND_DATA>,<PEND_DATAGRAM>	UDP receive (polled)
+SOCKRXU:<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<LENGTH>,<DATA>	UDP receive (async)
+SOCKRXU:<SOCK_ID>,<RMT_ADDR>,<RMT_PORT>,<LENGTH>,<SEQ_NUM>,<DATA>	UDP receive (async reliable)

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value
<RMT_ADDR>	Integer String Byte Array IPv4 or IPv6 Address	Remote address
<RMT_PORT>	Integer	Remote port Positive unsigned 16-bit value
<PEND_DATA>	Integer	Number of pending bytes
<PEND_DATAGRAM>	Integer	Number of pending datagrams
<LENGTH>	Integer	Number of bytes
<DATA>	Byte Array	Socket data
<SEQ_NUM>	Integer	Sequence number of first byte Unsigned 16-bit value

 ©2023-2026

+SOCKCL

Description

Socket close.

AEC Syntax

AEC	Description
+SOCKCL:<SOCK_ID>	Socket close response

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value

+SOCKTLS

Description

TLS success.

AEC Syntax

AEC	Description
+SOCKTLS:<SOCK_ID>	TLS success

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value

 ©2023-2026

+SOCKERR

Description

Socket error.

AEC Syntax

AEC	Description
+SOCKERR:<SOCK_ID>,<ERROR_CODE>	Socket error, see error code

AEC Element Syntax

Element Name	Type	Description
<SOCK_ID>	Integer	Socket ID Positive unsigned 16-bit value
<ERROR_CODE>	Integer	Error code See Status Response Codes

Examples:

Example Command Sequence 51: Opening a TCP server socket

→	`AT+SOCKO=2`	Open a TCP socket
←	`+SOCKO:5`	Socket ID 5 created
←	`OK`	
→	`AT+SOCKBL=5,5678`	Bind socket to local port 5678
←	`OK`	
←	`+SOCKIND:6,"1.2.3.4",5678,"11.22.33.44",12345`	New socket connection on port 5678

Example Command Sequence 52: Basic socket read

←	`+SOCKRXT:6,15`	Indicates 15 bytes are present
→	`AT+SOCKRD=6,1,10`	Read 10 bytes in ASCII mode
←	`+SOCKRD:6,10,"1234567890"`	10 bytes returned
←	`OK`	
←	`+SOCKRXT:6,7`	An additional 2 bytes have been received

Example Command Sequence 53: In-band socket read

←	`+SOCKRXT:6,20`	Indicates 20 bytes have been received
→	`AT+SOCKRD=6,3,10`	Read 10 bytes in ASCII mode with extended information
←	`+SOCKRD:6,10,10,"1234567890"`	10 bytes returned, indicates 10 bytes remain
←	`OK`	
→	`AT+SOCKRD=6,3,4`	Read 4 bytes in ASCII mode with extended information
←	`+SOCKRD:6,4,6,"abcd"`	4 bytes returned, indicates 6 bytes remain
←	`OK`	

Example Command Sequence 54: Simple asynchronous socket read

→	`AT+SOCKC=6,102,1500`	Configure socket ASYNC_MAX_FRM_SZ to be 1500 bytes
←	`OK`	
←	`+SOCKRXT:6,10,"1234567890"`	10 bytes received
←	`+SOCKRXT:6,4,"abcd"`	4 bytes received

 ©2023-2026

Example Command Sequence 55: Reliable asynchronous socket read

→	`AT+SOCKC=6,102,1500`	Configure socket ASYNC_MAX_FRM_SZ to be 1500 bytes
←	`OK`	
→	`AT+SOCKC=6,101,8`	Configure socket ASYNC_WIN_SZ to be 8 bytes
←	`OK`	
←	`+SOCKRXT:6,8,0,"12345678"`	8 bytes received, sequence number 0 to 7
→	`AT+SOCKC=6,100,8`	Configure ASYNC_NEXT_SN to be 8
←	`OK`	
←	`+SOCKRXT:6,8,8,"90abcdef"`	8 bytes received, sequence number 8 to 15
→	`AT+SOCKC=6,100,12`	Configure ASYNC_NEXT_SN to be 12
←	`OK`	
←	`+SOCKRXT:6,4,16,"ghij"`	4 bytes received, sequence number 16 to 19

Example Command Sequence 56: Set TLS configuration 1 for TLS client

→	`AT+TLSC=1,41,1`	Enable server domain verification (enabled by default)
←	`OK`	
→	`AT+TLSC=1,6,"ServerDomain"`	Specify server domain name
←	`OK`	
→	`AT+TLSC=1,40,1`	Enable server authentication (enabled by default)
←	`OK`	
→	`AT+TLSC=1,1,"ServerRootCACert"`	Specify file name of CA certificate
←	`OK`	
→	`AT+TLSC=1,2,"MyClientCert"`	Specify file name of client certificate (required if server authenticates the client)
←	`OK`	
→	`AT+TLSC=1,3,"MyClientPrivateKey"`	Specify file name of client private key (required if server authenticates the client)
←	`OK`	

Example Command Sequence 57: Open a TLS client socket

→	`AT+SOCKO=2`	Open a TCP socket
←	`+SOCKO:17603`	Socket created
←	`OK`	
→	`AT+SOCKTLS=17603,1`	Upgrade socket to TLS, using configuration 1
←	`OK`	
→	`AT+SOCKBR=17603,"example.com",443`	Bind socket to server
←	`OK`	
←	`+SOCKIND:17603,"1.2.3.4",65333,"5.6.7.8",443`	Socket connected
←	`+SOCKTLS:17603`	Socket upgraded to TLS

Example Command Sequence 58: Set TLS configuration 2 for TLS server

→	`AT+TLSC=2,2,"MyServerCertChain"`	Specify file name of server certificate
←	`OK`	
→	`AT+TLSC=2,3,"MyServerPrivateKey"`	Specify file name of server private key
←	`OK`	
→	`AT+TLSC=2,8,"MyDhParams"`	Specify file name of server DH parameters (DHE cipher suite only)
←	`OK`	
→	`AT+TLSC=2,40,0`	Disable client authentication
←	`OK`	
→	`AT+TLSC=2,41,0`	Disable client domain name verification
←	`OK`	

Example Command Sequence 59: Open a TLS server socket

→	`AT+SOCKO=2`	Open a TCP socket
←	`+SOCKO:16037`	Socket created
←	`OK`	
→	`AT+SOCKTLS=16037,2`	Upgrade socket to TLS, using configuration 2
←	`OK`	
→	`AT+SOCKBL=16037,443`	Bind socket to local port 443
←	`OK`	
←	`+SOCKIND:16038,"1.2.3.4",443,"11.22.33.44",12345`	New socket 16038 connected on port 443
←	`+SOCKTLS:16038`	Socket 16038 upgraded to TLS

Example Command Sequence 60: Reading partial datagram

→	`AT+SOCKO=1`	Open a UDP socket
←	`+SOCKO:7`	Socket ID 7 created
←	`OK`	
→	`AT+SOCKBL=7,1234`	Bind socket to port 1234
←	`OK`	
←	`+SOCKRXU:7,"1.2.3.4",1234,100,1`	100 bytes datagram received
→	`AT+SOCKRDBUF=7,1,10`	Read first 10 bytes of datagram
←	`+SOCKRDBUF:7,10,[11223344556677889900]`	
←	`OK`	
→	`AT+SOCKRDBUF=7,1,10`	Read next 10 bytes of the same datagram
←	`+SOCKRDBUF:7,10,[AABBCCDDEEFF11223344]`	
←	`OK`	
→	`AT+SOCKRD=7,1,10`	Discard rest of current datagram, read first 10 bytes of next datagram
←	`+SOCKRD:7,10,[1234567890ABCDEF0011]`	
←	`OK`	

 ©2023-2026

TIME (Module ID = 16)

Introduction

The TIME module sets and reads the system time. Three formats are supported:

- UNIX: seconds since 1970-01-01.
- NTP: seconds since 1900-01-01.
- RFC3339: a string of the form YYYY-MM-DDTHH:MM:SS.00Z.

A query with +TIME reports the time in a +TIME AEC followed by the OK response. Specifying a format in the query also sets the format used for all later time values reported on the interface, including those in +TIME AECs sent by the SNTP client. The default format is NTP.

Until the time is set it counts from 1970-01-01 at boot. When the SNTP client is enabled, each synchronisation overwrites a manually set time. The time is not retained across a reset.

Command Reference:

+TIME

Description

This command is used to set or query the system time.

Three formats of time are supported, two are in UTC seconds and one is a string based on RFC3339 / ISO-8601 format:

1. UNIX timestamp - UTC seconds since 1st January 1970
2. NTP time - UTC seconds since 1st January 1900
3. RFC3339 / ISO-8601 - String representation of date/time

RFC3339 / ISO-8601 format represents the time as:

```
YYYY-MM-DDTHH:MM:SS.00Z
```

The two fractional-second digits must be present but are ignored; times are reported with ".00".

For example:

```
2019-02-28T12:34:56.00Z - 28th February 2019, 12:34 PM 56 seconds (UTC+0)
```

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+TIME	Time query in current format	GGGG
AT+TIME=<FORMAT>	Time query in specified format	GGGG
AT+TIME=<FORMAT>,<UTC_SEC>	Time set in UTC seconds FORMAT must be 1 or 2	GGGG
AT+TIME=<FORMAT>,<DATE_TIME>	Time set in RFC3339/ISO-8601 format FORMAT must be 3	GGGG

 ©2023-2026

Command Parameter Syntax

Parameter Name	Type	Description
<FORMAT>	Integer	Format of time 1 UTC seconds (epoch 01/01/1970 - UNIX timestamp). 2 UTC seconds (epoch 01/01/1900 - NTP time). 3 RFC3339 / ISO-8601 format.
<UTC_SEC>	Integer	UTC seconds Unsigned 32-bit value
<DATE_TIME>	String	Date/time in format YYYY-MM-DDTHH:MM:SS.00Z

AEC Reference:

+TIME

Description

Time report.

AEC Syntax

AEC	Description
+TIME:<TIME>	Time report

AEC Element Syntax

Element Name	Type	Description
<TIME>	Integer String Byte Array	Current time

 ©2023-2026

TLS (Module ID = 17)

Introduction

Transport Layer Security (TLS) establishes an encrypted communication channel between a client and server, ensuring confidentiality and integrity of exchanged data.

The RNWF02 can establish a TLS session in either client mode or server mode. The TLS configuration parameters described in this module are used in conjunction with the SOCK, MQTT, HTTP, OTA, DNS and WSTA modules to enable secure connections.

Quick Start: TLS Client

1. Load the CA certificate which signed the server's certificate using the FS module, with file type 1 (certificate). The certificate may be in PEM or DER format; a PEM file may contain several certificates. File names are limited to 32 characters. The file system supplied with the RNWF02 includes a set of root CA certificates, list them with +FS=2,1.
2. Make sure the system time is valid. Certificates are checked against the system time, which counts from 1970-01-01 until it is set, so the CA certificate fails date validation until the time is set. Wait for the +TIME AEC from the SNTP client, or set the time with +TIME. By default the SNTP client only uses an NTP server supplied by DHCP; set NTP_STATIC and NTP_SVR to use a specific server.
3. Configure one of the four TLS configurations with +TLSC:
   - CA_CERT_NAME: the CA certificate file name, with or without its extension. The file must already be loaded.
   - DOMAIN_NAME: the server's domain name, used for domain name verification. It is not taken from the host name used to connect.
   - SERVER_NAME: the name sent in the Server Name Indication extension. It is not set automatically; many cloud services require it.
   - CERT_NAME and PRI_KEY_NAME: only needed if the server authenticates the client.
4. Use the configuration with +SOCKTLS, or with the TLS configuration element of MQTTC, HTTPC, OTAC, DNSC or WSTAC.

See example Basic TLS client configuration.

Only TLS 1.2 is supported. A TLS configuration cannot be changed while it is in use; +TLSC fails with STORE_UPDATE_BLOCKED.

Server Mode

A new TLS configuration has peer authentication and domain name verification enabled, in server mode as well as client mode. A server which does not authenticate its clients must disable both with PEER_AUTH and PEER_DOMAIN_VERIFY, otherwise the +TLSERR AEC reports CA Cert Missing or Peer Domain Missing. A server needs CERT_NAME and PRI_KEY_NAME, and DH_PARAM_NAME if DHE cipher suites are used.

See example Basic TLS server configuration.

Troubleshooting

Errors in a TLS configuration are reported by the +TLSERR AEC when a connection using it starts:

- CA Cert Missing: peer authentication is enabled and no CA certificate is set.
- CA Cert Validation: the CA certificate file could not be parsed.
- CA Cert Date Validation: the system time has not been set, or the CA certificate has expired.
- Key Pair Incomplete: only one of the certificate and private key is set.
- Peer Domain Missing: domain name verification is enabled and no domain name is set.

TLS handshake failures are reported by the module using the configuration:

- SOCK: +SOCKERR with Socket TLS Failed.
- HTTP: +HTTPERR with HTTP TLS Failed.
- OTA: +OTAERR with TLS Error.
- MQTT: +MQTTCONN reporting disconnected.
- WSTA (Enterprise): +WSTAERR with STA Connection Failed.

The wolfSSL error code is only reported through the SYSLOG module. Enable it with +SYSLOGC and look for a +SYSLOG AEC from the TLS module containing "wolfSSL error: <code>". Common codes, see TLS Error Codes for the full list:

- -188 ASN_NO_SIGNER_E: the CA certificate did not sign the server's certificate.
- -322 DOMAIN_NAME_MISMATCH: the domain name does not match the server's certificate.
- -150 ASN_BEFORE_DATE_E or -151 ASN_AFTER_DATE_E: the server's certificate is outside its validity period, or the system time has not been set.
- -155 ASN_SIG_CONFIRM_E: a certificate signature could not be verified.

 ©2023-2026

TLS configuration

Set TLS parameters using the +TLSC and +TLSCSC commands.

A new TLS configuration has the following security options enabled by default:

- Peer authentication (PEER_AUTH): a CA certificate must be provided via CA_CERT_NAME.
- Peer domain name verification (PEER_DOMAIN_VERIFY): the expected domain name must be provided via DOMAIN_NAME.
- Server renegotiation info required (SERVER_RENEG_INFO, client mode only).
- Session caching (SESSION_CACHING, client mode only).

If a required certificate or domain name is not configured, the TLS handshake fails. Peer authentication and domain name verification apply in both client and server mode; a server that does not authenticate its clients must disable them.

See examples Basic TLS client configuration and Basic TLS server configuration.

Certificate and Private Key

The X.509 certificate and the corresponding private key are required for the peer to authenticate the DCE.

The certificate file name can be specified via CERT_NAME and must be in PEM or DER format. If the file contains multiple certificates, it must be in PEM format. The private key file name can be specified via PRI_KEY_NAME and must be in PEM or DER format. If the private key is encrypted, the password must also be provided via PRI_KEY_PASSWORD.

Certificate and private key are always required for TLS server mode. For client mode, they are required if the server authenticates the client.

Server Name

The TLS Server Name Indication (SNI) extension allows a client to specify the hostname it is connecting to during the TLS handshake. This enables a server hosting multiple domains on the same IP address to present the correct certificate.

The server name is configured via SERVER_NAME. This parameter only applies to client mode.

Peer authentication

Peer authentication validates the peer's certificate chain against trusted CA certificates provided in the CA certificate file.

Peer authentication is enabled by default and is controlled by PEER_AUTH. When it is enabled, a valid CA certificate file name must be provided via CA_CERT_NAME. If the file contains multiple certificates, it must be in PEM format.

Peer authentication is recommended in client mode.

Domain Name Verification

Domain name verification confirms the peer's certificate was issued for the expected server by matching the configured domain name against the certificate's Subject Alternative Name (SAN), or Common Name (CN) if no SAN is present.

Domain name verification is enabled by default and is controlled by PEER_DOMAIN_VERIFY. When it is enabled, the expected domain name must be provided via DOMAIN_NAME.

Peer domain verification is recommended in client mode.

Cipher Suites

A cipher suite defines the algorithms used for key exchange, data encryption, and message authentication during a TLS session. The cipher suites index (CIPHER_SUITES_IDX) references a cipher suite list configured using the +TLSCSC command.

Diffie-Hellman Parameters

Diffie-Hellman (DH) parameters define the prime number and generator used to establish a shared secret between client and server without transmitting the secret itself. The file containing the Diffie-Hellman parameters must be specified via DH_PARAM_NAME if DHE key exchange cipher suites are enabled. Diffie-Hellman parameters are only required for server mode.

Certificate Pinning

Certificate pinning allows specifying a list of expected hash digest/hash algorithm pairs. During the TLS handshake, the peer's leaf certificate is hashed and compared against the configured digests. If none match, the connection is rejected.

Certificate pinning provides additional security beyond CA-based authentication by ensuring connections are only made to servers with specific known certificates, protecting against compromised CAs or misissued certificates.

See example configuring certificate digests

External crypto operations

TLS client authentication allows a server to verify the client's identity by requesting the client to sign a message with its private key, which is verified against the public key in the X.509 certificate. Signing operations can be delegated to an external crypto device via the EXTCRYPTO module.

When signing is enabled via EXTCRYPTO_OPS, only the certificate file must be provided via CERT_NAME. The private key is not required as it resides in the external crypto device.

See example TLS client configuration with external crypto device

 ©2023-2026

Server Renegotiation Extension

The TLS renegotiation info extension was introduced to address the TLS renegotiation vulnerability (CVE-2009-3555). Servers that are patched against this vulnerability include the renegotiation info extension during the TLS handshake.

When the server renegotiation info (SERVER_RENEG_INFO) is configured as "required", the client rejects connections to servers that do not provide it. Otherwise, the client will connect even if the server does not send the extension. "required" is the recommended setting unless connecting to legacy servers that do not support the security patch. This option is only applicable to client mode.

Session Caching

Session caching enables TLS session resumption using cached session data. When a TLS handshake completes, both client and server retain session state (including the master secret and negotiated cipher suite). On subsequent connections to the same server, this cached session can be reused to perform an abbreviated handshake that skips computationally expensive public-key operations. Session caching significantly reduces connection establishment time.

When a TLS configuration parameter is modified, the previously cached session is discarded.

TLS Cipher Suite codes

The supported cipher suites are listed below.

Code	Cipher Suite Name
0x002F	TLS_RSA_WITH_AES_128_CBC_SHA
0x0035	TLS_RSA_WITH_AES_256_CBC_SHA
0x0033	TLS_DHE_RSA_WITH_AES_128_CBC_SHA
0x0039	TLS_DHE_RSA_WITH_AES_256_CBC_SHA
0x003C	TLS_RSA_WITH_AES_128_CBC_SHA256
0x003D	TLS_RSA_WITH_AES_256_CBC_SHA256
0x0067	TLS_DHE_RSA_WITH_AES_128_CBC_SHA256
0x006B	TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
0x009C	TLS_RSA_WITH_AES_128_GCM_SHA256
0x009D	TLS_RSA_WITH_AES_256_GCM_SHA384
0x009E	TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
0x009F	TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
0xC009	TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA
0xC00A	TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA
0xC013	TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
0xC014	TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA
0xC023	TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256
0xC024	TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384
0xC027	TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
0xC028	TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
0xC02B	TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
0xC02C	TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
0xC02F	TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
0xC030	TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

 ©2023-2026

TLS Error Codes

To assist with debugging TLS issues, error codes from wolfSSL can be obtained by enabling logging via +SYSLOGC.

Error Code Enum	Error Code	Error Description
OPEN_RAN_E	-101	opening random device error
READ_RAN_E	-102	reading random device error
WINCRYPT_E	-103	windows crypt init error
CRYPTGEN_E	-104	windows crypt generation error
RAN_BLOCK_E	-105	reading random device would block
BAD_MUTEX_E	-106	Bad mutex operation
MP_INIT_E	-110	mp_init error state
MP_READ_E	-111	mp_read error state
MP_EXPTMOD_E	-112	mp_exptmod error state
MP_TO_E	-113	mp_to_xxx error state, can't convert
MP_SUB_E	-114	mp_sub error state, can't subtract
MP_ADD_E	-115	mp_add error state, can't add
MP_MUL_E	-116	mp_mul error state, can't multiply
MP_MULMOD_E	-117	mp_mulmod error state, can't multiply mod
MP_MOD_E	-118	mp_mod error state, can't mod
MP_INVMOD_E	-119	mp_invmod error state, can't inv mod
MP_CMP_E	-120	mp_cmp error state
MP_ZERO_E	-121	got a mp zero result, not expected
MEMORY_E	-125	out of memory error
RSA_WRONG_TYPE_E	-130	RSA wrong block type for RSA function
RSA_BUFFER_E	-131	RSA buffer error, output too small or input too large
BUFFER_E	-132	output buffer too small or input too large
ALGO_ID_E	-133	setting algo id error
PUBLIC_KEY_E	-134	setting public key error
DATE_E	-135	setting date validity error
SUBJECT_E	-136	setting subject name error
ISSUER_E	-137	setting issuer name error
CA_TRUE_E	-138	setting CA basic constraint true error
EXTENSIONS_E	-139	setting extensions error
ASN_PARSE_E	-140	ASN parsing error, invalid input
ASN_VERSION_E	-141	ASN version error, invalid number
ASN_GETINT_E	-142	ASN get big int error, invalid data
ASN_RSA_KEY_E	-143	ASN key init error, invalid input
ASN_OBJECT_ID_E	-144	ASN object id error, invalid id
ASN_TAG_NULL_E	-145	ASN tag error, not null
ASN_EXPECT_0_E	-146	ASN expect error, not zero

ASN_BITSTR_E	-147	ASN bit string error, wrong id
ASN_UNKNOWN_OID_E	-148	ASN oid error, unknown sum id
ASN_DATE_SZ_E	-149	ASN date error, bad size
ASN_BEFORE_DATE_E	-150	ASN date error, current date before
ASN_AFTER_DATE_E	-151	ASN date error, current date after
ASN_SIG_OID_E	-152	ASN signature error, mismatched oid
ASN_TIME_E	-153	ASN time error, unknown time type
ASN_INPUT_E	-154	ASN input error, not enough data
ASN_SIG_CONFIRM_E	-155	ASN sig error, confirm failure
ASN_SIG_HASH_E	-156	ASN sig error, unsupported hash type
ASN_SIG_KEY_E	-157	ASN sig error, unsupported key type
ASN_DH_KEY_E	-158	ASN key init error, invalid input
ASN_CRIT_EXT_E	-160	ASN unsupported critical extension
ECC_BAD_ARG_E	-170	ECC input argument of wrong type
ASN_ECC_KEY_E	-171	ASN ECC bad input
ECC_CURVE_OID_E	-172	Unsupported ECC OID curve type
BAD_FUNC_ARG	-173	Bad function argument provided
NOT_COMPILED_IN	-174	Feature not compiled in
UNICODE_SIZE_E	-175	Unicode password too big
NO_PASSWORD	-176	no password provided by user
ALT_NAME_E	-177	alt name size problem, too big
AES_GCM_AUTH_E	-180	AES-GCM Authentication check failure
AES_CCM_AUTH_E	-181	AES-CCM Authentication check failure
CAVIUM_INIT_E	-182	Cavium Init type error
COMPRESS_INIT_E	-183	Compress init error
COMPRESS_E	-184	Compress error
DECOMPRESS_INIT_E	-185	DeCompress init error
DECOMPRESS_E	-186	DeCompress error
BAD_ALIGN_E	-187	Bad alignment for operation, no alloc
ASN_NO_SIGNER_E	-188	ASN sig error, no CA signer to verify certificate
ASN_CRL_CONFIRM_E	-189	ASN CRL no signer to confirm failure
ASN_CRL_NO_SIGNER_E	-190	ASN CRL no signer to confirm failure
ASN_OCSP_CONFIRM_E	-191	ASN OCSP signature confirm failure
BAD_ENC_STATE_E	-192	Bad ecc enc state operation
BAD_PADDING_E	-193	Bad padding, msg not correct length
REQ_ATTRIBUTE_E	-194	Setting cert request attributes error
PKCS7_OID_E	-195	PKCS#7, mismatched OID error
PKCS7_RECIP_E	-196	PKCS#7, recipient error
FIPS_NOT_ALLOWED_E	-197	FIPS not allowed error
ASN_NAME_INVALID_E	-198	ASN name constraint error
RNG_FAILURE_E	-199	RNG Failed, Reinitialize
HMAC_MIN_KEYLEN_E	-200	FIPS Mode HMAC Minimum Key Length error

 ©2023-2026

RSA_PAD_E	-201	RSA Padding Error
LENGTH_ONLY_E	-202	Returning output length only
IN_CORE_FIPS_E	-203	In Core Integrity check failure
AES_KAT_FIPS_E	-204	AES KAT failure
DES3_KAT_FIPS_E	-205	DES3 KAT failure
HMAC_KAT_FIPS_E	-206	HMAC KAT failure
RSA_KAT_FIPS_E	-207	RSA KAT failure
DRBG_KAT_FIPS_E	-208	HASH DRBG KAT failure
DRBG_CONT_FIPS_E	-209	HASH DRBG Continuous test failure
AESGCM_KAT_FIPS_E	-210	AESGCM KAT failure
THREAD_STORE_KEY_E	-211	Thread local storage key create failure
THREAD_STORE_SET_E	-212	Thread local storage key set failure
MAC_CMP_FAILED_E	-213	MAC comparison failed
IS_POINT_E	-214	ECC is point on curve failed
ECC_INF_E	-215	ECC point infinity error
ECC_PRIV_KEY_E	-216	ECC private key not valid error
SRP_CALL_ORDER_E	-217	SRP function called in the wrong order
SRP_VERIFY_E	-218	SRP proof verification failed
SRP_BAD_KEY_E	-219	SRP bad ephemeral values
ASN_NO_SKID	-220	ASN no Subject Key Identifier found
ASN_NO_AKID	-221	ASN no Authority Key Identifier found
ASN_NO_KEYUSAGE	-223	ASN no Key Usage found
SKID_E	-224	Setting Subject Key Identifier error
AKID_E	-225	Setting Authority Key Identifier error
KEYUSAGE_E	-226	Bad Key Usage value
CERTPOLICIES_E	-227	Setting Certificate Policies error
WC_INIT_E	-228	wolfCrypt failed to initialize
SIG_VERIFY_E	-229	wolfCrypt signature verify error
BAD_PKCS7_SIGNEEDS_CHECKCOND_E	-230	Bad condition variable operation
SIG_TYPE_E	-231	Signature Type not enabled/available
HASH_TYPE_E	-232	Hash Type not enabled/available
WC_KEY_SIZE_E	-234	Key size error, either too small or large
ASN_COUNTRY_SIZE_E	-235	ASN Cert Gen, invalid country code size
MISSING_RNG_E	-236	RNG required but not provided
ASN_PATHLEN_SIZE_E	-237	ASN CA path length too large error
ASN_PATHLEN_INV_E	-238	ASN CA path length inversion error
BAD_KEYWRAP_ALG_E	-239	Algorithm error with keywrap
BAD_KEYWRAP_IV_E	-240	Decrypted AES key wrap IV incorrect
WC_CLEANUP_E	-241	wolfCrypt cleanup failed
ECC_CDH_KAT_FIPS_E	-242	ECC CDH known answer test failure
DH_CHECK_PUB_E	-243	DH check public key error
BAD_PATH_ERROR	-244	Bad path for opendir

ASYNC_OP_E	-245	Async operation error
ECC_PRIVATEONLY_E	-246	Invalid use of private only ECC key
EXTKEYUSAGE_E	-247	Bad extended key usage value
WC_HW_E	-248	Error with hardware crypto use
WC_HW_WAIT_E	-249	Hardware waiting on resource
PSS_SALTLEN_E	-250	PSS length of salt is too long for hash
PRIME_GEN_E	-251	Failure finding a prime
BER_INDEF_E	-252	Cannot decode indefinite length BER
RSA_OUT_OF_RANGE_E	-253	Ciphertext to decrypt out of range
RSAPSS_PAT_FIPS_E	-254	RSA-PSS PAT failure
ECDSA_PAT_FIPS_E	-255	ECDSA PAT failure
DH_KAT_FIPS_E	-256	DH KAT failure
AESCCM_KAT_FIPS_E	-257	AESCCM KAT failure
SHA3_KAT_FIPS_E	-258	SHA-3 KAT failure
ECDHE_KAT_FIPS_E	-259	ECDHE KAT failure
AES_GCM_OVERFLOW_E	-260	AES-GCM invocation counter overflow
AES_CCM_OVERFLOW_E	-261	AES-CCM invocation counter overflow
RSA_KEY_PAIR_E	-262	RSA Key Pair-Wise consistency check fail
DH_CHECK_PRIVE_E	-263	DH check private key error
WC_AFALG_SOCK_E	-264	AF_ALG socket error
WC_DEVCRYPTO_E	-265	/dev/crypto error
ZLIB_INIT_ERROR	-266	Zlib init error
ZLIB_COMPRESS_ERROR	-267	Zlib compression error
ZLIB_DECOMPRESS_ERROR	-268	Zlib decompression error
PKCS7_NO_SIGNER_E	-269	No signer in PKCS7 signed data msg
WC_PKCS7_WANT_READ_E	-270	PKCS7 stream operation wants more input
CRYPTOCB_UNAVAILABLE	-271	Crypto callback unavailable
PKCS7_SIGNEEDS_CHECK	-272	Signature needs verified by caller
ASN_SELF_SIGNED_E	-275	ASN self-signed certificate error
MIN_CODE_E	-300	errors -101 - -299
INPUT_CASE_ERROR	-301	process input state error
PREFIX_ERROR	-302	bad index to key rounds
MEMORY_ERROR	-303	out of memory
VERIFY_FINISHED_ERROR	-304	verify problem on finished
VERIFY_MAC_ERROR	-305	verify mac problem
PARSE_ERROR	-306	parse error on header
UNKNOWN_HANDSHAKE_TYPE	-307	weird handshake type
SOCKET_ERROR_E	-308	error state on socket
SOCKET_NODATA	-309	expected data, not there
INCOMPLETE_DATA	-310	don’t have enough data to complete task
UNKNOWN_RECORD_TYPE	-311	unknown type in record hdr
DECRYPT_ERROR	-312	error during decryption

 ©2023-2026

FATAL_ERROR	-313	revcd alert fatal error
ENCRYPT_ERROR	-314	error during encryption
FREAD_ERROR	-315	fread problem
NO_PEER_KEY	-316	need peer’s key
NO_PRIVATE_KEY	-317	need the private key
RSA_PRIVATE_ERROR	-318	error during rsa priv op
NO_DH_PARAMS	-319	server missing DH params
BUILD_MSG_ERROR	-320	build message failure
BAD_HELLO	-321	client hello malformed
DOMAIN_NAME_MISMATCH	-322	peer subject name mismatch
WANT_READ	-323	want read, call again
NOT_READY_ERROR	-324	handshake layer not ready
VERSION_ERROR	-326	record layer version error
WANT_WRITE	-327	want write, call again
BUFFER_ERROR	-328	malformed buffer input
VERIFY_CERT_ERROR	-329	verify cert error
VERIFY_SIGN_ERROR	-330	verify sign error
CLIENT_ID_ERROR	-331	psk client identity error
SERVER_HINT_ERROR	-332	psk server hint error
PSK_KEY_ERROR	-333	psk key error
GETTIME_ERROR	-337	gettimeofday() error
GETITIMER_ERROR	-338	getitimer() error
SIGACT_ERROR	-339	sigaction() error
SETITIMER_ERROR	-340	setitimer() error
LENGTH_ERROR	-341	record layer length error
PEER_KEY_ERROR	-342	cant decode peer key
ZERO_RETURN	-343	peer sent close notify
SIDE_ERROR	-344	wrong client/server type
NO_PEER_CERT	-345	peer didn’t send key
ECC_CURVETYPE_ERROR	-350	Bad ECC Curve Type
ECC_CURVE_ERROR	-351	Bad ECC Curve
ECC_PEERKEY_ERROR	-352	Bad Peer ECC Key
ECC_MAKEKEY_ERROR	-353	Bad Make ECC Key
ECC_EXPORT_ERROR	-354	Bad ECC Export Key
ECC_SHARED_ERROR	-355	Bad ECC Shared Secret
NOT_CA_ERROR	-357	Not CA cert error
BAD_CERT_MANAGER_ERROR	-359	Bad Cert Manager
OCSP_CERT_REVOKED	-360	OCSP Certificate revoked
CRL_CERT_REVOKED	-361	CRL Certificate revoked
CRL_MISSING	-362	CRL Not loaded
MONITOR_SETUP_E	-363	CRL Monitor setup error
THREAD_CREATE_E	-364	Thread Create Error

OCSP_NEED_URL	-365	OCSP need an URL for lookup
OCSP_CERT_UNKNOWN	-366	OCSP responder doesn't know
OCSP_LOOKUP_FAIL	-367	OCSP lookup not successful
MAX_CHAIN_ERROR	-368	max chain depth exceeded
COOKIE_ERROR	-369	dtls cookie error
SEQUENCE_ERROR	-370	dtls sequence error
SUITES_ERROR	-371	suites pointer error
OUT_OF_ORDER_E	-373	out of order message
BAD_KEA_TYPE_E	-374	bad KEA type found
SANITY_CIPHER_E	-375	sanity check on cipher error
RECV_OVERFLOW_E	-376	RXCB returned more than rqed
GEN_COOKIE_E	-377	Generate Cookie Error
NO_PEER_VERIFY	-378	Need peer cert verify Error
FWRITE_ERROR	-379	fwrite problem
CACHE_MATCH_ERROR	-380	cache hrd match error
UNKNOWN_SNI_HOST_NAME_E	-381	Unrecognized host name Error
UNKNOWN_MAX_FRAG_LEN_E	-382	Unrecognized max frag len Error
KEYUSE_SIGNATURE_E	-383	KeyUse digSignature error
KEYUSE_ENCIPHER_E	-385	KeyUse KeyEncipher error
EXTKEYUSE_AUTH_E	-386	Extended Key Usage server/client auth not set
SEND_OOB_READ_E	-387	Send Cb out of bounds read
SECURE_RENEGOTIATION_E	-388	Invalid renegotiation info
SESSION_TICKET_LEN_E	-389	Session Ticket too large
SESSION_TICKET_EXPECT_E	-390	Session Ticket missing
SCR_DIFFERENT_CERT_E	-391	SCR Different cert error
SESSION_SECRET_CB_E	-392	Session secret CB fcn failure
NO_CHANGE_CIPHER_E	-393	Finished before change cipher
SANITY_MSG_E	-394	Sanity check on msg order error
DUPLICATE_MST_E	-395	Duplicate message error
SNI_UNSUPPORTED	-396	SSL 3.0 does not support SNI
SOCKET_PEER_CLOSED_E	-397	Underlying transport closed
BAD_TICKET_KEY_CB_SZ	-398	Bad session ticket key cb size
BAD_TICKET_MSG_SZ	-399	Bad session ticket msg size
BAD_TICKET_ENCRYPT	-400	Bad user ticket encrypt
DH_KEY_SIZE_E	-401	DH key too small
SNI_ABSENT_ERROR	-402	No SNI request
RSA_SIGN_FAULT	-403	RSA sign fault
HANDSHAKE_SIZE_ERROR	-404	Handshake message too large
UNKNOWN_ALPN_PROTOCOL_NAME_E	-405	Unrecognized protocol name error
BAD_CERTIFICATE_STATUS_ERROR	-406	Bad certificate status message
OCSP_INVALID_STATUS	-407	Invalid OCSP status
OCSP_WANT_READ	-408	OCSP callback response

 ©2023-2026

RSA_KEY_SIZE_E	-409	RSA key too small
ECC_KEY_SIZE_E	-410	ECC key too small
DTLS_EXPORT_VER_E	-411	Export version error
INPUT_SIZE_E	-412	Input size too big error
CTX_INIT_MUTEX_E	-413	Initialize ctx mutex error
EXT_MASTER_SECRET_NEEDED_E	-414	Need EMS enabled to resume
DTLS_POOL_SZ_E	-415	Exceeded DTLS pool size
DECODE_E	-416	Decode handshake message error
HTTP_TIMEOUT	-417	HTTP timeout for OCSP or CRL req
WRITE_DUP_READ_E	-418	Write dup write side can’t read
WRITE_DUP_WRITE_E	-419	Write dup read side can’t write
INVALID_CERT_CTX_E	-420	TLS cert ctx not matching
BAD_KEY_SHARE_DATA	-421	Key share data invalid
MISSING_HANDSHAKE_DATA	-422	Handshake message missing data
BAD_BINDER	-423	Binder does not match
EXT_NOT_ALLOWED	-424	Extension not allowed in msg
INVALID_PARAMETER	-425	Security parameter invalid
MCAST_HIGHWATER_CB_E	-426	Multicast highwater cb error
ALERT_COUNT_E	-427	Alert count exceeded error
EXT_MISSING	-428	Required extension not found
UNSUPPORTED_EXTENSION	-429	TLS extension not requested by the client
PRF_MISSING	-430	PRF not compiled in
DTLS_RETX_OVER_TX	-431	Retransmit DTLS flight over
DH_PARAMS_NOT_FFDHE_E	-432	DH params from server not FFDHE
TCA_INVALID_ID_TYPE	-433	TLS extension Trusted CA ID type invalid
TCA_ABSENT_ERROR	-434	TLS extension Trusted CA ID response absent
UNSUPPORTED_SUITE	-500	Unsupported cipher suite
MATCH_SUITE_ERROR	-501	Can’t match cipher suite
COMPRESSION_ERROR	-502	Compression mismatch
KEY_SHARE_ERROR	-503	Key share mismatch
POST_HAND_AUTH_ERROR	-504	Client won’t do post-hand auth
HRR_COOKIE_ERROR	-505	HRR msg cookie mismatch

Command Reference:

+TLSC

Description

This command is used to read or set the TLS configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+TLSC	Query configuration list	GGGG
AT+TLSC=<CONF>	Query all configuration elements	GGGG
AT+TLSC=<CONF>,<ID>	Query a single element	GGGG
AT+TLSC=<CONF>,<ID>,<VAL>	Set a single element ID must not be 9	GGGG
AT+TLSC=<CONF>,<ID>,<DIGEST>,<ALGO>	Set a hash digest/algorithm pair ID must be 9	GGGG
AT+TLSC=<CONF>,<ID>,<DIGEST>	Clear all digests (DIGEST must be empty) ID must be 9	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<CONF>	Integer	Configuration number Valid range is 1 to 4
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value
<DIGEST>	Byte Array	Hash digest
<ALGO>	Integer	Hash algorithm 1 SHA1. 2 SHA2-256. 3 SHA2-224. 4 SHA2-512. 5 SHA2-384. Valid range is 1 to 5

 ©2023-2026

Response Syntax

Response	Description
+TLSC:<MIN_IDX>,<MAX_IDX>	Valid index range
+TLSC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<MIN_IDX>	Integer	Lowest valid index
<MAX_IDX>	Integer	Highest valid index
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<CA_CERT_NAME>	String	CA certificate name Maximum length of string is 32	GGGG
2	<CERT_NAME>	String	Certificate name Maximum length of string is 32	GGGG
3	<PRI_KEY_NAME>	String	Private key name Maximum length of string is 32	GGGG
4	<PRI_KEY_PASSWORD>	String	Private key password Maximum length of string is 64	GGGG
5	<SERVER_NAME>	String	Server name Maximum length of string is 256	GGGG
6	<DOMAIN_NAME>	String	Domain name Maximum length of string is 256	GGGG
7	<CIPHER_SUITES_IDX>	Integer	Cipher suite index Valid range is 1 to 2	GGGG
8	<DH_PARAM_NAME>	String	Server Diffie-Hellman parameters identifier Maximum length of string is 32	GGGG
9	<PEER_CERT_DIGESTS>	Complex Value[1]	List of hash digest/algorithm pairs This is a multiple value parameter with an ID range 9.0 to 9.3 See Certificate Digests below.	GGGG
20	<EXTCRYPTO_OPS>	Integer	Crypto operations to be done externally 0 None. 1 Signing.	GGGG

ID	Name	Type	Description	Sec
21	<SIGN_TYPES>	Integer (Read Only)	Types of signing to be done externally 1 ECDSA.	GGGG
40	<PEER_AUTH>	Bool	Peer authentication. Default is enabled 0 Peer authentication is disabled. 1 Peer authentication is enabled.	GGGG
41	<PEER_DOMAIN_VERIFY>	Bool	Peer domain verification. Default is enabled 0 Peer domain name verification is disabled. 1 Peer domain name verification is enabled.	GGGG
42	<SERVER_RENEG_INFO>	Bool	Client mode: require renegotiation info from server. Default is required 0 Do not require renegotiation info from server. 1 Require renegotiation info from server.	GGGG
43	<SESSION_CACHING>	Bool	Client mode: cache and reuse session IDs. Default is enabled 0 Do not reuse session IDs. 1 Attempt to reuse session IDs.	GGGG

Complex Value Syntax

No.	Complex Value	Description
1	<DIGEST>,<ALGO>	Hash digest/algorithm pair

Complex Value Element Syntax

Element Name	Type	Description
<DIGEST>	Byte Array	Hash digest
<ALGO>	Integer	Hash algorithm

 ©2023-2026

Certificate Digests

Up to four digest/algorithm pairs can be stored. Setting a pair without an index adds it after the existing entries; this fails if four are already stored. Setting a pair with an index (9.<n>) replaces that entry.

The digest length must match the algorithm: SHA1 20 bytes, SHA2-224 28 bytes, SHA2-256 32 bytes, SHA2-384 48 bytes, SHA2-512 64 bytes.

Setting an empty digest ([]) without an index removes all pairs; with an index (9.<n>) it removes that entry only, and later entries move down one place. A pair cannot be removed by specifying its digest.

+TLSCSC

Description

This command is used to read or set the TLS cipher suite configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+TLSCSC	Query configuration list	GGGG
AT+TLSCSC=<CSL_IDX>	Query all configuration elements	GGGG
AT+TLSCSC=<CSL_IDX>,<ID>	Query a single element	GGGG
AT+TLSCSC=<CSL_IDX>,<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<CSL_IDX>	Integer	Cipher suite list index Valid range is 1 to 2
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+TLSCSC:<MIN_IDX>,<MAX_IDX>	Valid index range
+TLSCSC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<MIN_IDX>	Integer	Lowest valid index
<MAX_IDX>	Integer	Highest valid index
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<CIPHER_SUITES>	Unsigned Integer	Cipher suites Positive unsigned 16-bit value This is a multiple value parameter with an ID range 1.0 to 1.31	GGGG
2	<CIPHER_SUITES_AVAIL>	Unsigned Integer (Read Only)	Cipher suites available Positive unsigned 16-bit value This is a multiple value parameter with an ID range 2.0 to 2.31	GGGG

AEC Reference:

+TLSERR

Description

TLS error.

AEC Syntax

AEC	Description
+TLSERR:<CONF>,<ERROR_CODE>	TLS error

AEC Element Syntax

Element Name	Type	Description
<CONF>	Integer	Configuration number Valid range is 1 to 4
<ERROR_CODE>	Integer	Error code See Status Response Codes

 ©2023-2026

Examples:

Example Command Sequence 61: Peer certificate digest pinning with +TLSC

→	`AT+TLSC=1,9,[000102030405060708090A0B0C0D0E0F00010203],1`	Add SHA1 000102... certificate digest
←	`OK`	
→	`AT+TLSC=1,9,[101112131415161718191A1B1C1D1E1F10111213],1`	Add SHA1 101112... certificate digest
←	`OK`	
→	`AT+TLSC=1,9`	List certificate digests
←	`+TLSC:9.0,[000102030405060708090A0B0C0D0E0F00010203],1`	
←	`+TLSC:9.1,[101112131415161718191A1B1C1D1E1F10111213],1`	
←	`OK`	
→	`AT+TLSC=1,9,[]`	Clear certificate digest list
←	`OK`	
→	`AT+TLSC=1,9`	List certificate digests
←	`OK`	

WAP (Module ID = 18)

Introduction

The WAP module operates the RNWF02 as a soft access point.

Configure the soft AP with +WAPC and then start it with +WAP=1. While the soft AP is running only MAC_BLOCKLIST may be changed.

The soft AP cannot be started while the station is connected, or while a scan, periodic scan or the beacon service is running.

Unless the network interface is configured as layer 2 only, starting the soft AP sets the interface address and starts the DHCP server (see DHCP) and the DNS server.

Clients are reported by the +WAPSC AEC when they connect, +WAPAIP when they are assigned an address and +WAPSD when they disconnect. Up to 10 clients are supported; the association IDs are the same as those used by the ASSOC module.

Stopping the soft AP with +WAP=0 completes asynchronously, the state continues to read as 1 until the soft AP has fully stopped.

 ©2023-2026

Command Reference:

+WAPC

Description

This command is used to read or set the DCE's hotspot access point configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WAPC	Query all configuration elements	GGGG
AT+WAPC=<ID>	Query a single element	GGGG
AT+WAPC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+WAPC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<SSID>	String	Network Name Maximum length of string is 32	GGGG
2	<SEC_TYPE>	Integer	Security Type 0 Open. 2 WPA2 Personal Mixed Mode. 3 WPA2 Personal. 4 WPA3 Personal Transition Mode. 5 WPA3 Personal.	GGGG
3	<CREDENTIALS>	String	Credentials required for connecting to the network of the security type specified Maximum length of string is 128	GGGG
4	<CHANNEL>	Integer	The channel on which to set up the network Valid range is 1 to 13	GGGG
5	<HIDDEN>	Bool	Visibility of the network 0 Not hidden, SSID is broadcast in beacons. 1 Hidden, SSID is not broadcast.	GGGG
8	<NETIF_IDX>	Unsigned Integer	Network interface index Valid range is 0 to 1	GGGG
9	<REKEY_INTERVAL>	Unsigned Integer	Specify the time in seconds that must pass before each re-key attempt Valid range is 60 to 86400	GGGG
20	<MAC_BLOCKLIST>	MAC Address	MAC address blocklist entry Valid range is 0 to 3 This is a multiple value parameter with an ID range 20.0 to 20.3	GGGG
50	<MFP_TYPE>	Integer	Management Frame Protection configuration 0 MFP Disabled. 1 MFP Enabled. 2 MFP Required.	GGGG

 ©2023-2026

+WAP

Description

This command is used to enable the DCE's hotspot access point functionality.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WAP	Read status of AP function	GGGG
AT+WAP=<STATE>	Set state of AP function	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<STATE>	Integer	State of the hotspot feature 0 Disable. 1 Enable.

Response Syntax

Response	Description
+WAP:<STATE>	Query state

Response Element Syntax

Element Name	Type	Description
<STATE>	Integer	Connected state 0 Disabled. 1 Enabled.

AEC Reference:

+WAPAIP

Description

STA IP address assignment.

AEC Syntax

AEC	Description
+WAPAIP:<ASSOC_ID>,<IP_ADDRESS>	Assignment of STA IP address

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	Integer	Association ID
<IP_ADDRESS>	String	IP address assigned

 ©2023-2026

+WAPSC

Description

STA connected.

AEC Syntax

AEC	Description
+WAPSC:<ASSOC_ID>,<MAC_ADDRESS>	Station connect

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	Integer	Association ID
<MAC_ADDRESS>	String	MAC address of the STA

+WAPSD

Description

STA disconnected.

AEC Syntax

AEC	Description
+WAPSD:<ASSOC_ID>,<MAC_ADDRESS>	Station disconnect

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	Integer	Association ID
<MAC_ADDRESS>	String	MAC address of the STA

 ©2023-2026

+WAPERR

Description

AP error.

AEC Syntax

AEC	Description
+WAPERR:<ERROR_CODE>	AP error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	Integer	Error code See Status Response Codes

WSCN (Module ID = 19)

Introduction

The WSCN module scans for access points. Two independent scan types are provided, each with its own configuration:

- One-shot active or passive scans, configured with +WSCNC and started with +WSCN.
- Periodic passive scans, configured with +WSCNPC and started and stopped with +WSCNP.

Each access point found is reported by a +WSCNIND AEC if ASYNC_NOTIFY is set, followed by a +WSCNDONE AEC with the number of results. The 8 results with the strongest RSSI may be read from SCAN_RESULTS after the scan.

Results may be filtered by minimum RSSI (RSSI_THRESH), by security type (SEC_FILTER) and by up to 4 SSIDs (FILT_LIST), an SSID ending in * matches any SSID beginning with the preceding characters.

Scanning is allowed while the station is connected, but not while it is connecting or disconnecting.

Periodic Scanning

A periodic scan sweeps the configured channels once every INTERVAL milliseconds. The duration of a sweep is the passive slot time multiplied by the number of slots and the number of channels; the interval must be at least twice the sweep duration and at least the sweep duration plus 50 ms, otherwise the scan will not start.

Periodic scanning requires the soft AP to be stopped. The +WSCNPC configuration cannot be changed while a periodic scan is running.

+WSCNP=0 allows the current sweep to complete and then sends the +WSCNPDONE AEC. If a sweep fails to start the periodic scan ends with the +WSCNPERR AEC.

 ©2023-2026

Command Reference:

+WSCNC

Description

This command is used to modify or query the behavior of the active scanning function.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WSCNC	Query all configuration elements	GGGG
AT+WSCNC=<ID>	Query a single element	GGGG
AT+WSCNC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+WSCNC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<CHANNEL>	Unsigned Integer	Channel to scan (or 0 for all channels) Valid range is 0 to 13	GGGG
2	<ACT_SLOT_TIME>	Unsigned Integer	The time in milliseconds to wait for probe responses Valid range is 10 to 1500	GGGG
3	<PASV_SLOT_TIME>	Unsigned Integer	The time in milliseconds to wait for beacons Valid range is 10 to 1500	GGGG
4	<NUM_SLOTS>	Unsigned Integer	Number of scan slots Positive unsigned 8-bit value	GGGG
5	<PROBES_PER_SLOT>	Unsigned Integer	Number of probes sent per active slot Valid range is 1 to 2	GGGG
6	<RSSI_THRESH>	Integer	RSSI threshold (or 0 for none) Signed 8-bit value	GGGG
7	<SEC_FILTER>	Unsigned Integer	Security filter, bitmask of: 0x01 Personal. 0x02 Enterprise. 0x04 Open. 0x08 Unknown or unsupported. 0x10 WPA or WPA2 Mixed Mode. 0x20 WPA2. 0x40 WPA3 or WPA3 Transition Mode.	GGGG
8	<ASYNC_NOTIFY>	Bool	Asynchronous notification 0 No scan indication AECs are generated. 1 Indication AECs are generated.	GGGG
10	<FILT_LIST>	Byte Array	SSID filter list, an entry ending with '*' matches any SSID with the preceding prefix This is a multiple value parameter with an ID range 10.0 to 10.3	GGGG

 ©2023-2026

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th><th>Sec</th></tr>
<tr><td>11</td><td><CHANMASK24></td><td>Unsigned Integer</td><td>Scan-all channel mask (2.4 GHz)
<table><tr><td>15
-</td><td>14
-</td><td>13
CH14</td><td>12
CH13</td><td>11
CH12</td><td>10
CH11</td><td>9
CH10</td><td>8
CH9</td></tr><tr><td>7
CH8</td><td>6
CH7</td><td>5
CH6</td><td>4
CH5</td><td>3
CH4</td><td>2
CH3</td><td>1
CH2</td><td>0
CH1</td></tr></table><table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0x0001</td><td>CH1</td><td>Channel 1 - 2412 MHz.</td></tr><tr><td>0x0002</td><td>CH2</td><td>Channel 2 - 2417 MHz.</td></tr><tr><td>0x0004</td><td>CH3</td><td>Channel 3 - 2422 MHz.</td></tr><tr><td>0x0008</td><td>CH4</td><td>Channel 4 - 2427 MHz.</td></tr><tr><td>0x0010</td><td>CH5</td><td>Channel 5 - 2432 MHz.</td></tr><tr><td>0x0020</td><td>CH6</td><td>Channel 6 - 2437 MHz.</td></tr><tr><td>0x0040</td><td>CH7</td><td>Channel 7 - 2442 MHz.</td></tr><tr><td>0x0080</td><td>CH8</td><td>Channel 8 - 2447 MHz.</td></tr><tr><td>0x0100</td><td>CH9</td><td>Channel 9 - 2452 MHz.</td></tr><tr><td>0x0200</td><td>CH10</td><td>Channel 10 - 2457 MHz.</td></tr><tr><td>0x0400</td><td>CH11</td><td>Channel 11 - 2462 MHz.</td></tr><tr><td>0x0800</td><td>CH12</td><td>Channel 12 - 2467 MHz.</td></tr><tr><td>0x1000</td><td>CH13</td><td>Channel 13 - 2472 MHz.</td></tr><tr><td>0x2000</td><td>CH14</td><td>Channel 14 - 2485 MHz.</td></tr></table></td><td>GGGG</td></tr>
<tr><td>20</td><td><SCAN_RESULTS></td><td>Complex Value[1]
(Read Only)</td><td>Top scan results from previous scan, ordered by RSSI
This is a multiple value parameter
with an ID range 20.0 to 20.7</td><td>GGGG</td></tr>
</table>

Complex Value Syntax

No.	Complex Value	Description
1	<RSSI>,<SEC_TYPE>,<CHANNEL>,<BSSID>,<SSID>	Scan indicator

Complex Value Element Syntax

Element Name	Type	Description
<RSSI>	Integer	Received signal strength
<SEC_TYPE>	Integer	Recommended security type to use when connecting to this AP -1 Unknown or unsupported. 0 Open. 2 WPA2-Personal Mixed Mode. 3 WPA2-Personal. 4 WPA3-Personal Transition Mode. 5 WPA3-Personal. 6 WPA2-Enterprise Mixed Mode. 7 WPA2-Enterprise. 8 WPA3-Enterprise Transition Mode. 9 WPA3-Enterprise.
<CHANNEL>	Integer	The channel number of the detected device
<BSSID>	String	The BSSID of detected device
<SSID>	String	SSID of detected device

 ©2023-2026

+WSCN

Description

This command is used to scan for infrastructure networks in range of the DCE.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WSCN=<ACT_PASV>	Scan	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ACT_PASV>	Integer	Flag indicating active or passive scanning 0 Passive scanning. 1 Active scanning.

+WSCNPC

Description

This command is used to modify or query the behavior of the periodic passive scanning function.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WSCNPC	Query all configuration elements	GGGG
AT+WSCNPC=<ID>	Query a single element	GGGG
AT+WSCNPC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+WSCNPC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG

 ©2023-2026

ID	Name	Type	Description	Sec
1	<CHANNEL>	Unsigned Integer	Channel to scan (or 0 for all channels) Valid range is 0 to 13	GGGG
3	<PASV_SLOT_TIME>	Unsigned Integer	The time in milliseconds to wait for beacons Valid range is 10 to 1500	GGGG
4	<NUM_SLOTS>	Unsigned Integer	Number of scan slots Positive unsigned 8-bit value	GGGG
6	<RSSI_THRESH>	Integer	RSSI threshold (or 0 for none) Signed 8-bit value	GGGG
7	<SEC_FILTER>	Unsigned Integer	Security filter, bitmask of: 0x01 Personal. 0x02 Enterprise. 0x04 Open. 0x08 Unknown or unsupported. 0x10 WPA or WPA2 Mixed Mode. 0x20 WPA2. 0x40 WPA3 or WPA3 Transition Mode.	GGGG
8	<ASYNC_NOTIFY>	Bool	Asynchronous notification 0 No scan indication AECs are generated. 1 Indication AECs are generated.	GGGG
10	<FILT_LIST>	Byte Array	SSID filter list, an entry ending with '*' matches any SSID with the preceding prefix This is a multiple value parameter with an ID range 10.0 to 10.3	GGGG

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th><th>Sec</th></tr>
<tr><td>11</td><td><CHANMASK24></td><td>Unsigned Integer</td><td>Scan-all channel mask (2.4 GHz)

<table><tr><td>15
-</td><td>14
-</td><td>13
CH14</td><td>12
CH13</td><td>11
CH12</td><td>10
CH11</td><td>9
CH10</td><td>8
CH9</td></tr><tr><td>7
CH8</td><td>6
CH7</td><td>5
CH6</td><td>4
CH5</td><td>3
CH4</td><td>2
CH3</td><td>1
CH2</td><td>0
CH1</td></tr></table>
<table><tr><th>Value</th><th>Label</th><th>Description</th></tr><tr><td>0x0001</td><td>CH1</td><td>Channel 1 - 2412 MHz.</td></tr><tr><td>0x0002</td><td>CH2</td><td>Channel 2 - 2417 MHz.</td></tr><tr><td>0x0004</td><td>CH3</td><td>Channel 3 - 2422 MHz.</td></tr><tr><td>0x0008</td><td>CH4</td><td>Channel 4 - 2427 MHz.</td></tr><tr><td>0x0010</td><td>CH5</td><td>Channel 5 - 2432 MHz.</td></tr><tr><td>0x0020</td><td>CH6</td><td>Channel 6 - 2437 MHz.</td></tr><tr><td>0x0040</td><td>CH7</td><td>Channel 7 - 2442 MHz.</td></tr><tr><td>0x0080</td><td>CH8</td><td>Channel 8 - 2447 MHz.</td></tr><tr><td>0x0100</td><td>CH9</td><td>Channel 9 - 2452 MHz.</td></tr><tr><td>0x0200</td><td>CH10</td><td>Channel 10 - 2457 MHz.</td></tr><tr><td>0x0400</td><td>CH11</td><td>Channel 11 - 2462 MHz.</td></tr><tr><td>0x0800</td><td>CH12</td><td>Channel 12 - 2467 MHz.</td></tr><tr><td>0x1000</td><td>CH13</td><td>Channel 13 - 2472 MHz.</td></tr><tr><td>0x2000</td><td>CH14</td><td>Channel 14 - 2485 MHz.</td></tr></table></td><td>GGGG</td></tr>
<tr><td>12</td><td><INTERVAL></td><td>Unsigned Integer</td><td>Interval in milliseconds between the start of one periodic sweep and the start of the next. A sweep must fit within half of it, with a further small margin for result retrieval

Valid range is 500 to 600000</td><td>GGGG</td></tr>
<tr><td>20</td><td><SCAN_RESULTS></td><td>Complex Value[1]
(Read Only)</td><td>Top scan results from previous scan, ordered by RSSI
This is a multiple value parameter
with an ID range 20.0 to 20.7</td><td>GGGG</td></tr>
</table>

Complex Value Syntax

No.	Complex Value	Description
1	<RSSI>,<SEC_TYPE>,<CHANNEL>,<BSSID>,<SSID>	Scan indicator

 ©2023-2026

Complex Value Element Syntax

Element Name	Type	Description
<RSSI>	Integer	Received signal strength
<SEC_TYPE>	Integer	Recommended security type to use when connecting to this AP -1 Unknown or unsupported. 0 Open. 2 WPA2-Personal Mixed Mode. 3 WPA2-Personal. 4 WPA3-Personal Transition Mode. 5 WPA3-Personal. 6 WPA2-Enterprise Mixed Mode. 7 WPA2-Enterprise. 8 WPA3-Enterprise Transition Mode. 9 WPA3-Enterprise.
<CHANNEL>	Integer	The channel number of the detected device
<BSSID>	String	The BSSID of detected device
<SSID>	String	SSID of detected device

+WSCNP

Description

This command is used to start or stop the periodic passive scanning function.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WSCNP	Query state of the periodic scan function	GGGG
AT+WSCNP=<STATE>	Set state of the periodic scan function	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<STATE>	Integer	State of the periodic scan function 0 Disable. 1 Enable.

Response Syntax

Response	Description
+WSCNP:<STATE>	Query state

Response Element Syntax

Element Name	Type	Description
<STATE>	Integer	State of the periodic scan function 0 Disabled. 1 Enabled.

 ©2023-2026

AEC Reference:

+WSCNIND

Description

Scan results.

AEC Syntax

AEC	Description
+WSCNIND:<RSSI>,<SEC_TYPE>,<CHANNEL>,<BSSID>,<SSID>	Scan indicator

AEC Element Syntax

Element Name	Type	Description
<RSSI>	Integer	Received signal strength
<SEC_TYPE>	Integer	Recommended security type to use when connecting to this AP -1 Unknown or unsupported. 0 Open. 2 WPA2-Personal Mixed Mode. 3 WPA2-Personal. 4 WPA3-Personal Transition Mode. 5 WPA3-Personal. 6 WPA2-Enterprise Mixed Mode. 7 WPA2-Enterprise. 8 WPA3-Enterprise Transition Mode. 9 WPA3-Enterprise.
<CHANNEL>	Integer	The channel number of the detected device
<BSSID>	String	The BSSID of detected device
<SSID>	String	SSID of detected device

+WSCNDONE

Description

Scan completed.

AEC Syntax

AEC	Description
+WSCNDONE:<NUM_RESULTS>	Scan complete

AEC Element Syntax

Element Name	Type	Description
<NUM_RESULTS>	Integer	Number of results returned in this scan operation

 ©2023-2026

+WSCNPDONE

Description

Periodic scanning stopped on request. Sent once when the run ends, after the final sweep has reported its results, and is the only report of the run-wide counters.

AEC Syntax

AEC	Description
+WSCNPDONE:<NUM_SWEEPS>	Periodic scanning stopped

AEC Element Syntax

Element Name	Type	Description
<NUM_SWEEPS>	Integer	Number of sweeps started over the periodic scanning run which has just ended

+WSCNPERR

Description

A periodic scan ended on a fault rather than by completing.

AEC Syntax

AEC	Description
+WSCNPERR:<ERROR_CODE>	Periodic scan error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	Integer	Error code See Status Response Codes

 ©2023-2026

Examples:

Example Command Sequence 62: Scan for Access Points and filter by security type

→	`AT+WSCNC=7`	Query security filter
←	`+WSCNC=7,0x77`	Security filter 0x77 (all supported security types)
←	`OK`	
→	`AT+WSCN=1`	Scan for Access Points
←	`OK`	
←	`+WSCNIND:-38,6,11,"00:01:02:03:04:05","SSID1"`	AP on network "SSID1" found on channel 11. For connecting, the recommended security type is WPA2-Enterprise Mixed Mode
←	`+WSCNIND:-38,6,6,"00:02:04:06:08:0A","SSID1"`	AP on network "SSID1" found on channel 6. For connecting, the recommended security type is WPA2-Enterprise Mixed Mode
←	`+WSCNIND:-40,4,1,"00:03:06:09:0C:0F","SSID2"`	AP on network "SSID2" found on channel 1. For connecting, the recommended security type is WPA3-Personal Transition Mode
←	`+WSCNIND:-35,5,6,"00:04:08:0C:10:14","SSID3"`	AP on network "SSID3" found on channel 6. For connecting, the recommended security type is WPA3-Personal
←	`+WSCNDONE:4`	Scan completed, 4 results found
→	`AT+WSCNC=7,0x21`	Set security filter to WPA2-Personal
→	`AT+WSCN=1`	Scan for Access Points
←	`OK`	
←	`+WSCNIND:-41,4,1,"00:03:06:09:0C:0F","SSID2"`	AP on network "SSID2" found on channel 1. For connecting, the recommended security type is WPA3-Personal Transition Mode
←	`+WSCNIND:-36,5,6,"00:04:08:0C:10:14","SSID3"`	AP on network "SSID3" found on channel 6. For connecting, the recommended security type is WPA3-Personal
←	`+WSCNDONE:2`	Scan completed, 2 results found
→	`AT+WSCNC=7,0x41`	Set security filter to WPA3-Personal or WPA3-Personal Transition Mode
→	`AT+WSCN=1`	Scan for Access Points
←	`OK`	
←	`+WSCNIND:-35,5,6,"00:04:08:0C:10:14","SSID3"`	AP on network "SSID3" found on channel 6. For connecting, the recommended security type is WPA3-Personal
←	`+WSCNDONE:1`	Scan completed, 1 result found
→	`AT+WSCNC=7,0x22`	Set security filter to WPA2-Enterprise
→	`AT+WSCN=1`	Scan for Access Points
←	`OK`	
←	`+WSCNDONE:0`	Scan completed, 0 results found
→	`AT+WSCNC=7`	Query security filter
←	`+WSCNC=7,0x22`	Security filter 0x22 (WPA2-Enterprise)
←	`OK`	

Example Command Sequence 63: Setting SSID filter with +WSCNC

→	`AT+WSCNC=10,"SSID1"`	Add SSID1 to filter list
←	`OK`	
→	`AT+WSCNC=10,"SSID2"`	Add SSID2 to filter list
←	`OK`	
→	`AT+WSCNC=10,"SSID3"`	Add SSID3 to filter list
←	`OK`	
→	`AT+WSCNC=10`	List all filters
←	`+WSCNC:10.0,"SSID1"`	
←	`+WSCNC:10.1,"SSID2"`	
←	`+WSCNC:10.2,"SSID3"`	
←	`OK`	
→	`AT+WSCNC=10.1,""`	Remove second filter entry (SSID2)
←	`OK`	
→	`AT+WSCNC=10`	List all filters
←	`+WSCNC:10.0,"SSID1"`	
←	`+WSCNC:10.1,"SSID3"`	
←	`OK`	
→	`AT+WSCNC=10,""`	Remove all filters
←	`OK`	

Example Command Sequence 64: Setting SSID prefix filters with +WSCNC

→	`AT+WSCN=1`	Scan for Access Points, no SSID filter set
←	`OK`	
←	`+WSCNIND:-38,6,11,"00:01:02:03:04:05","SSID01"`	AP on network "SSID01" found on channel 11
←	`+WSCNIND:-40,4,1,"00:03:06:09:0C:0F","SSID02"`	AP on network "SSID02" found on channel 1
←	`+WSCNIND:-35,5,6,"00:04:08:0C:10:14","SSID11"`	AP on network "SSID11" found on channel 6
←	`+WSCNDONE:3`	Scan completed, 3 results found
→	`AT+WSCNC=10,"SSID0*"`	Add a filter matching every SSID starting with "SSID0"
←	`OK`	
→	`AT+WSCN=1`	Scan for Access Points
←	`OK`	
←	`+WSCNIND:-38,6,11,"00:01:02:03:04:05","SSID01"`	"SSID01" matches the filter prefix
←	`+WSCNIND:-40,4,1,"00:03:06:09:0C:0F","SSID02"`	"SSID02" matches the filter prefix
←	`+WSCNDONE:2`	Scan completed, 2 results found, "SSID11" does not match the filter prefix

 ©2023-2026

Example Command Sequence 65: Periodic passive scanning with an SSID prefix filter

→	`AT+WSCNPC=1,6`	Set the periodic scan channel to 6
←	`OK`	
→	`AT+WSCNPC=3,500`	Listen for 500 ms in each scan slot
←	`OK`	
→	`AT+WSCNPC=4,2`	Two slots per sweep, so each sweep listens on the channel for a total of 1000 ms
←	`OK`	
→	`AT+WSCNPC=12,5000`	Start a sweep every 5000 ms
←	`OK`	
→	`AT+WSCNPC=6,-45`	Discard anything weaker than -45 dBm
←	`OK`	
→	`AT+WSCNPC=10,"ASSET*"`	Report only matching SSIDs, so beacons from ordinary access points are dropped
←	`OK`	
→	`AT+WSCNP=1`	Enable periodic scanning, the first sweep starts immediately. All of the scan configuration must be in place by now: it is locked while scanning is enabled
←	`OK`	
←	`+WSCNIND:-30,0,6,"F8:F0:05:12:34:56","ASSET35"`	A matching beacon is detected on channel 6
←	`+WSCNDONE:1`	Sweep completed, 1 result found
	`..`	
←	`+WSCNDONE:0`	A later sweep found no matching beacons
	`..`	
→	`AT+WSCNP=0`	Stop periodic scanning
←	`OK`	
←	`+WSCNPDONE:12`	The run is over: 12 sweeps were carried out

Example Command Sequence 66: Periodic passive scanning with an SSID prefix filter

→	`AT+WSCNPC=1,6`	Set the periodic scan channel to 6
←	`OK`	
→	`AT+WSCNPC=3,500`	Listen for 500 ms in each scan slot
←	`OK`	
→	`AT+WSCNPC=4,2`	Two slots per sweep, so each sweep listens on the channel for a total of 1000 ms
←	`OK`	
→	`AT+WSCNPC=12,5000`	Start a sweep every 5000 ms
←	`OK`	
→	`AT+WSCNPC=6,-45`	Discard anything weaker than -45 dBm
←	`OK`	
→	`AT+WSCNPC=10,"CMD*"`	Report only SSIDs carrying command payloads
←	`OK`	
→	`AT+WSCNP=1`	Enable periodic scanning, the first sweep starts immediately. All of the scan configuration must be in place by now: it is locked while scanning is enabled
←	`OK`	
←	`+WSCNDONE:0`	A sweep which found no matching beacon
	`..`	
←	`+WSCNIND:-30,0,6,"F8:F0:05:AB:CD:EF","CMD3240138012398502495190247597"`	Matching SSID payload received
←	`+WSCNDONE:1`	Sweep completed, 1 result found
	`..`	
→	`AT+WSCNP=0`	Stop periodic scanning
←	`OK`	
←	`+WSCNPDONE:8`	The run is over: 8 sweeps were carried out

 ©2023-2026

WSTA (Module ID = 20)

Introduction

The RNWF02 supports a wide range of features when operating as a Wi-Fi station (STA). It is designed for robust connectivity, security, and seamless network integration in a variety of applications.

The WSTA module is used to configure the STA, control STA connection and get information about the STA.

Quick Start: Connecting to an Access Point

1. Configure the Access Point's SSID, security type and credentials with +WSTAC, see example WPA Personal. The configuration can only be changed while the STA is disconnected, except ROAMING.
2. Select how the STA's network interface (index 0 by default, see NETIF) obtains its IP address with +NETIFC: the DHCP client (DHCPC_EN, enabled by default), or a static address set with IP_MASK and GATEWAY. If DHCP is disabled and no address is set, a link-local (ZeroConf) address is used.
3. Start the connection with +WSTA=1. The OK response indicates the connection attempt has started, not that the STA is connected.
4. The +WSTALU AEC reports that the link is up, with the association ID, the BSSID of the Access Point and the channel.
5. The +WSTAAIP AEC reports each address assigned automatically, by DHCP or ZeroConf for IPv4 and by SLAAC for IPv6. It is sent again when the DHCP lease is renewed or the address changes. A static IPv4 address is not reported.

See example Connect at Wi-Fi and IP layers.

Connection Refused

+WSTA=1 returns an error, and no AEC is sent, if:

- the STA is already connecting or connected, or the soft AP is running: STA Connection Not Permitted;
- a scan or periodic scan is in progress: Scan In Progress or Periodic Scan is Running;
- the security configuration is not valid, for example an invalid passphrase, an unsupported security type or a missing Enterprise TLS context: Invalid Parameter.

Connection Failed

If the Access Point refuses or does not respond to the connection, or the connection has not completed within CONN_TIMEOUT (10 seconds by default), the +WSTAERR AEC is sent with the status STA Connection Failed. The cause is not reported; an incorrect password, an Access Point which cannot be found and an authentication failure all appear the same.

After +WSTAERR the STA is disconnected and the RNWF02 does not retry. To retry, send +WSTA=1 again.

Link Loss and Disconnection

The +WSTALD AEC reports that the link has been lost. The RNWF02 does not reconnect automatically, send +WSTA=1 to reconnect. Roaming between Access Points within the same ESSID is handled automatically while connected and is reported by the +WSTAROAM AEC, followed by +WSTAAIP when roaming is configured for layer 3.

+WSTA=0 disconnects the STA asynchronously: if the STA was connected the +WSTALD AEC follows, if it was still connecting no AEC follows. Querying +WSTA returns 1 from the start of the connection attempt until the disconnection has completed.

Configuring the STA

Set STA parameters using the +WSTAC command.

In order to connect the STA to an Access Point (AP), the following parameters must be set:

- The AP's network name (SSID): SSID
- The security type (Open, WPA Personal or WPA Enterprise): SEC_TYPE
- The security credentials (depending on security type)

Other parameters are optional, with default behaviour if they are not set:

- The AP's Wi-Fi channel (1-13, or 0 for any; defaults to 0): CHANNEL
- The AP's BSSID: BSSID
- Management Frame Protection configuration (disabled, enabled or required; defaults to enabled): MFP_TYPE
- Disabling of WPA3 Transition modes (defaults to "not modified"): WPA3_TD
- Connection attempt timeout in milliseconds (defaults to 10000 ms; 0 disables the timeout): CONN_TIMEOUT
- Roaming configuration (disabled, Wi-Fi only, or layer 3; defaults to layer 3): ROAMING

Security types and credentials

Open security provides no authentication or encryption. There are no security credentials to set.

WPA Personal security provides authentication and encryption via a password. WPA2 and WPA3 are supported, including transition and mixed modes. The credentials parameter must be set to the AP's WPA Personal password: CREDENTIALS

WPA Enterprise security provides encryption and various methods for authentication. WPA2 and WPA3 are supported, including transition and mixed modes. See the WPA Enterprise section below for further information about how to set authentication methods and security credentials.

 ©2023-2026

Roaming

The RNWF02 STA supports Simple Roaming, which enables faster handover between Access Points within an ESSID for maintained connectivity.

Roaming can be configured as:

- Wi-Fi only - the STA moves to another AP within the same ESSID; the IP configuration (layer 3 and above) is unchanged
- Layer 3 - as above, and the DHCP lease is also renewed after each roam

Controlling STA connection

Once the STA has been configured appropriately, start and stop the STA connection using the +WSTA command.

STA information

Query the STA parameters using the +WSTAC command.

Query the STA status using the +WSTA command.

During connection:

- The +WSTALU and +WSTALD AECs indicate the Wi-Fi link being established ("up") or lost ("down") respectively. The +WSTAROAM AEC indicates a layer 2 roam event (connection lost and regained, possibly to a different AP).
- The +WSTAAIP AEC indicates the IP address assigned to the STA. Layer 3 roam events may cause this AEC to be received multiple times, whenever the IP address changes.

WPA Enterprise

WPA Enterprise significantly enhances Wi-Fi security by using an external authentication server (typically a RADIUS server) to perform user or device authentication dynamically. This process utilises the Extensible Authentication Protocol (EAP), enabling flexible authentication mechanisms beyond entering a simple shared password.

WPA Enterprise typically uses a **two-step process**:

1. **Establish a secure encrypted tunnel between the Wi-Fi client (STA) and the authentication server.** A TLS (Transport Layer Security) handshake allows the client to authenticate the server. The resulting TLS session provides an encrypted tunnel to protect subsequent communications.
2. **Authenticate the client securely inside the encrypted tunnel.** After the tunnel is established, a second method (such as MSCHAPv2 or another TLS handshake) allows the server to authenticate the client via the client's credentials (such as an MSCHAPv2 username and password or a TLS certificate and private key). This client authentication is protected by the encrypted tunnel.

Upon successful authentication, the client and server generate a Pairwise Master Key (PMK). This is used to establish a secure Wi-Fi association between the STA and the Access Point.

Supported WPA Enterprise methods

To maximise compatibility with a broad range of Enterprise networks, the RNWF02 supports the following tunnel and credential methods:

Method	Description
EAP-TTLSv0/MSCHAPv2	TLS tunnel method is TTLSv0; client authentication is done using MSCHAPv2
EAP-PEAPv0/MSCHAPv2	TLS tunnel method is PEAPv0; client authentication method is MSCHAPv2
EAP-PEAPv1/MSCHAPv2	TLS tunnel method is PEAPv1; client authentication method is MSCHAPv2
EAP-TLS	Single-step process with mutual authentication in the TLS handshake
EAP-PEAPv0/TLS	TLS tunnel method is PEAPv0; client authentication method is another TLS handshake
EAP-PEAPv1/TLS	TLS tunnel method is PEAPv1; client authentication method is another TLS handshake

 ©2023-2026

Configuring WPA Enterprise methods and security credentials

In order to use WPA Enterprise to connect the STA to an AP, via an authentication server, the following parameters must be set:

- The AP's network name (SSID): SSID
- The security type (WPA Enterprise): SEC_TYPE
- The unprotected user identity (many servers accept an anonymous identity): ENT_IDENT_PLAIN
- The protected user identity: ENT_IDENT_PROT

NOTE The protected user identity parameter may be left unset if the tunnel method parameter is set to "None" or "TTLSv0".

- The client credential type (TLS or MSCHAPv2): ENT_CRED_TYPE
- The TLS configuration index for the tunnel method: ENT_TUNN_TLS_CONF

NOTE The TLS configuration index for the tunnel method may be left unset if the tunnel method parameter is set to "None".

In addition:

- If the client credential type is TLS, the TLS configuration index to use for client credentials must be set: ENT_CRED_TLS_CONF
- If the client credential type is MSCHAPv2, the MSCHAPv2 username and password parameters must be set: ENT_MSCHV2_UN and ENT_MSCHV2_PW

Other WPA Enterprise credentials are optional, with default behaviours if they are not set:

- The WPA Enterprise tunnel method (TTLSv0, PEAPv0, PEAPv1, "none" or "any"; defaults to "any"): ENT_TUNN_METHOD

NOTE

TLS configurations must be configured using the TLSC module. In particular:

- If the TLS configuration is to be used for client credentials, the client's certificate and private key must be specified via CERT_NAME and PRI_KEY_NAME
- For EAP-PEAPv0 tunnel method, TLS session caching is not supported and SESSION_CACHING is ignored
- For TLS credential method, TLS session caching is not used and SESSION_CACHING is ignored
- For WPA3 (and WPA2/WPA3 transition mode) Enterprise, server domain name verification must be enabled via PEER_DOMAIN_VERIFY and DOMAIN_NAME

NOTE For EAP-PEAP/TLS methods, it may be appropriate and convenient to use the same TLS configuration for both the tunnel method and the client credentials.

Example WPA Enterprise Configurations

Example 1: WPA3 Enterprise with MSCHAPv2 credentials

First configure a TLS context for establishing a secure encrypted tunnel:

→	`AT+TLSC=1,1,"radius_cert"`	Specify file name of CA certificate
←	`OK`	
→	`AT+TLSC=1,6,"wifi.alpha.net"`	Specify authentication server's domain name
←	`OK`	
→	`AT+TIME=3`	Query to ensure time is valid (knowledge of current time is needed for TLS)
←	`OK`	
←	`+TIME:"2025-10-24T18:03:43.00Z"`	

Then configure the STA connection parameters, making use of the above TLS context:

→	`AT+WSTAC=1,"AUTO-WPA3-ENTERPRISE"`	Specify SSID of Access Point
←	`OK`	
→	`AT+WSTAC=2,9`	Set security type to WPA3-Enterprise
←	`OK`	
→	`AT+WSTAC=20,"Anonymous@wifi.alpha.net"`	Set Enterprise identity, to be sent unprotected (anonymous username)
←	`OK`	
→	`AT+WSTAC=21,"my_username@wifi.alpha.net"`	Set Enterprise identity, to be sent protected inside secure tunnel (real username)
←	`OK`	
→	`AT+WSTAC=26,1`	Set TLSC config index to use for establishing the tunnel
←	`OK`	
→	`AT+WSTAC=30,1`	Set credential type to MSCHAPv2
←	`OK`	
→	`AT+WSTAC=32,"my_username"`	Set MSCHAPv2 username
←	`OK`	
→	`AT+WSTAC=33,"my_password"`	Set MSCHAPv2 password
←	`OK`	

Note that the tunnel method has not been specified here. Either EAP-TTLS or EAP-PEAP may be used, depending on the server's request.

Now connect:

 ©2023-2026

→	`AT+NETIFC=0,10,1`	Enable DHCP client
←	`OK`	
→	`AT+WSTA=1`	Start connection attempt
←	`OK`	
←	`+WSTALU:1,"00:01:02:03:04:05",6`	Link up: connected on channel 6 to an AP with BSSID 00:01:02:03:04:05
←	`+WSTAAIP:1,"192.168.0.20"`	IP address received via auto configuration (DHCP/SLAAC)

Example 2: WPA3 Enterprise with EAP-PEAPv1/TLS method

First configure a TLS context, including the user's certificate and private key:

→	`AT+TLSC=2,1,"radius_cert"`	Specify file name of CA certificate
←	`OK`	
→	`AT+TLSC=2,2,"my_cert"`	Specify file name of user's certificate
←	`OK`	
→	`AT+TLSC=2,3,"my_key"`	Specify file name of user's private key
←	`OK`	
→	`AT+TLSC=2,6,"wifi.alpha.net"`	Specify authentication server's domain name
←	`OK`	
→	`AT+TIME=3`	Query to ensure time is valid (knowledge of current time is needed for TLS)
←	`OK`	
←	`+TIME:"2025-10-24T18:03:43.00Z"`	

Then configure the STA connection parameters, making use of the above TLS context for both establishing the tunnel and authenticating the client:

→	`AT+WSTAC=1,"AUTO-WPA3-ENTERPRISE"`	Specify SSID of Access Point
←	`OK`	
→	`AT+WSTAC=2,9`	Set security type to WPA3-Enterprise
←	`OK`	
→	`AT+WSTAC=20,"Anonymous@wifi.alpha.net"`	Set EAP identity to use during tunnel establishment (anonymous username)
←	`OK`	
→	`AT+WSTAC=21,"my_username@wifi.alpha.net"`	Set EAP identity to use inside secure tunnel (real username)
←	`OK`	
→	`AT+WSTAC=25,4`	Restrict tunnel method to PEAPv1
←	`OK`	
→	`AT+WSTAC=26,2`	Set TLSC config index to use for establishing the tunnel
←	`OK`	
→	`AT+WSTAC=30,0`	Set credential type to TLS
←	`OK`	
→	`AT+WSTAC=31,2`	Set TLSC config index to use for authenticating the client
←	`OK`	

Now connect:

 ©2023-2026

→	`AT+NETIFC=0,10,1`	Enable DHCP client
←	`OK`	
→	`AT+WSTA=1`	Start connection attempt
←	`OK`	
←	`+WSTALU:1,"00:01:02:03:04:05",6`	Link up: connected on channel 6 to an AP with BSSID 00:01:02:03:04:05
←	`+WSTAAIP:1,"192.168.0.20"`	IP address received via auto configuration (DHCP/SLAAC)

Command Reference:

+WSTAC

Description

This command is used to read or set the DCE's Wi-Fi station mode configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Enterprise Parameter Visibility

When the configuration is read, WPA Enterprise parameters are only reported when they apply to the current configuration. A parameter that is not reported can still be set.

- No Enterprise parameters are reported until ENT_TUNN_TLS_CONF or ENT_CRED_TLS_CONF is set to a TLS configuration index (see +TLSC).
- ENT_IDENT_PROT is not reported when ENT_TUNN_METHOD is NONE or TTLSv0.
- ENT_TUNN_TLS_CONF is not reported when ENT_TUNN_METHOD is NONE.
- ENT_CRED_TLS_CONF is not reported when ENT_CRED_TYPE is MSCHAPv2.
- ENT_MSCHV2_UN and ENT_MSCHV2_PW are not reported when ENT_CRED_TYPE is TLS.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WSTAC	Query all configuration elements	GGGG
AT+WSTAC=<ID>	Query a single element	GGGG
AT+WSTAC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

 ©2023-2026

Response Syntax

Response	Description
+WSTAC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<SSID>	String	Network name Maximum length of string is 32	GGGG
2	<SEC_TYPE>	Integer	Security type 0 Open. 2 WPA2-Personal Mixed Mode. 3 WPA2-Personal. 4 WPA3-Personal Transition Mode. 5 WPA3-Personal. 6 WPA2-Enterprise Mixed Mode. 7 WPA2-Enterprise. 8 WPA3-Enterprise Transition Mode. 9 WPA3-Enterprise.	GGGG
3	<CREDENTIALS>	String	Credentials for connecting to the network Maximum length of string is 128	GGGG
4	<CHANNEL>	Integer	Current channel of BSS (or 0 for any) Valid range is 0 to 13	GGGG
5	<BSSID>	MAC Address	The BSSID of the network to connect to	GGGG
7	<CONN_TIMEOUT>	Integer	Connection timeout in milliseconds. 0 disables the timeout. Default is 10000 Valid range is 0 to 0x7FFFFFFF	GGGG
8	<NETIF_IDX>	Unsigned Integer	Network interface index Valid range is 0 to 1	GGGG

ID	Name	Type	Description	Sec
9	<ROAMING>	Integer	Roaming state 0 Roaming is disabled. 1 Roaming is enabled on WiFi only. 2 Roaming is enabled on layer 3.	GGGG
20	<ENT_IDENT_PLAIN>	String	Identity for Enterprise, to be sent unprotected Maximum length of string is 255	GGGG
21	<ENT_IDENT_PROT>	String	Identity for Enterprise, to be sent protected (PEAP tunnels only) Maximum length of string is 255	GGGG
25	<ENT_TUNN_METHOD>	Integer	Tunnel method for Enterprise 0 Any. 1 None. 2 TTLSv0. 3 PEAPv0. 4 PEAPv1.	GGGG
26	<ENT_TUNN_TLS_CONF>	Integer	TLS configuration index (see +TLSC) to use for Enterprise tunnel. 0 if not configured Valid range is 0 to 4	GGGG
30	<ENT_CRED_TYPE>	Integer	Type of credentials for Enterprise 0 TLS. 1 MSCHAPv2.	GGGG
31	<ENT_CRED_TLS_CONF>	Integer	TLS configuration index (see +TLSC) to use for Enterprise TLS credentials. 0 if not configured Valid range is 0 to 4	GGGG
32	<ENT_MSCHV2_UN>	String	Username element of Enterprise MSCHAPv2 credentials Maximum length of string is 255	GGGG
33	<ENT_MSCHV2_PW>	String	Password element of Enterprise MSCHAPv2 credentials Maximum length of string is 255	GGGG
50	<MFP_TYPE>	Integer	Management Frame Protection configuration 0 MFP Disabled. 1 MFP Enabled. 2 MFP Required.	GGGG
51	<WPA3_TD>	Bool	WPA3 Transition disable modifier 0 WPA3 Transition algorithms not modified. 1 WPA3 Transition algorithms are disabled.	GGGG

 ©2023-2026

+WSTA

Description

This command is used to control or query the DCE's station mode functionality.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WSTA	Read status of STA function	GGGG
AT+WSTA=<STATE>	Set state of STA function	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<STATE>	Integer	State of the Wi-Fi station feature 0 Disable. 1 Use configuration from +WSTAC.

Response Syntax

Response	Description
+WSTA:<STATE>	State

Response Element Syntax

Element Name	Type	Description
<STATE>	Integer	Station state 0 Not connected. 1 Connecting, connected or disconnecting.

AEC Reference:

+WSTAAIP

Description

Indication of STA automatic address assignment.

AEC Syntax

AEC	Description
+WSTAAIP:<ASSOC_ID>,<IP_ADDRESS>	Assignment of IP address to association

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	Integer	Association ID
<IP_ADDRESS>	String	IP address assigned

 ©2023-2026

+WSTALD

Description

Link lost.

AEC Syntax

AEC	Description
+WSTALD:<ASSOC_ID>	Link down

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	Integer	Association ID

+WSTAERR

Description

Connection error.

AEC Syntax

AEC	Description
+WSTAERR:<ERROR_CODE>	Link error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	Integer	Error code See Status Response Codes

 ©2023-2026

+WSTALU

Description

Link established.

AEC Syntax

AEC	Description
+WSTALU:<ASSOC_ID>,<BSSID>,<CHANNEL>	Link up

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	Integer	Association ID
<BSSID>	String	The BSSID of the Access Point the DCE has connected to
<CHANNEL>	Integer	The channel number of network

+WSTAROAM

Description

Roaming occurred.

AEC Syntax

AEC	Description
+WSTAROAM:<ASSOC_ID>	Roaming occurred

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	Integer	Association ID

 ©2023-2026

Examples:

Example Command Sequence 73: WPA Personal: Configure STA connection parameters

→	`AT+WSTAC=1,"MyAP"`	Specify SSID of Access Point
←	`OK`	
→	`AT+WSTAC=2,3`	Set security type to WPA2-Personal
←	`OK`	
→	`AT+WSTAC=3,"MyAPPSK"`	Set WPA2 Personal passphrase for connecting to the Access Point
←	`OK`	

Example Command Sequence 74: Connect at Wi-Fi and IP layers

→	`AT+NETIFC=0,10,1`	Enable DHCP client
←	`OK`	
→	`AT+WSTA=1`	Start connection attempt
←	`OK`	
←	`+WSTALU:1,"00:01:02:03:04:05",6`	Link up: connected on channel 6 to an AP with BSSID 00:01:02:03:04:05
←	`+WSTAAIP:1,"192.168.0.20"`	IP address received via auto configuration (DHCP/SLAAC)

Example Command Sequence 75: Some disconnections, reconnections and roaming

→	`AT+WSTA=1`	Start connection attempt
←	`OK`	
←	`+WSTALU:1,"00:01:02:03:04:05",11`	Link up: connected on channel 11
←	`+WSTAAIP:1,"192.168.0.20"`	IP address received via auto configuration (DHCP/SLAAC)
→	`AT+WSTA=0`	Disconnect
←	`OK`	
→	`AT+WSTA`	Query STA status
←	`+WSTA:0`	STA is not connected
←	`OK`	
→	`AT+WSTA=1`	Start connection attempt
←	`OK`	
←	`+WSTALU:2,"00:01:02:03:04:05",11`	Link up: each connection is given a new association ID
←	`+WSTAAIP:2,"192.168.0.20"`	IP address received via auto configuration (DHCP/SLAAC)
←	`+WSTAROAM:2`	STA has roamed
←	`+WSTAAIP:2,"10.1.0.5"`	Change in IP address due to layer 3 roaming
→	`AT+WSTA`	Query STA status
←	`+WSTA:1`	STA is connected
←	`OK`	
←	`+WSTALD:2`	Link down

Example Command Sequence 76: +WSTA refused while a scan or periodic

	`scan is active`	
→	`AT+WSCN=1`	Scan for Access Points
←	`OK`	
→	`AT+WSTA=1`	Refused: a scan is in progress
←	`ERROR:<n>,"Scan In Progress"`	
←	`+WSCNDONE:2`	The scan in flight completes on its own
→	`AT+WSTA=1`	Now accepted
←	`OK`	
←	`+WSTALU:1,"00:01:02:03:04:05",6`	
←	`+WSTAAIP:1,"192.168.0.20"`	
→	`AT+WSTA=0`	Disconnect
←	`OK`	
←	`+WSTALD:1`	Link down
→	`AT+WSCNP=1`	Enable periodic scanning
←	`OK`	
→	`AT+WSTA=1`	Refused: periodic scanning is running
←	`ERROR:<n>,"Periodic Scan is Running"`	
→	`AT+WSCNP=0`	Disable periodic scanning
←	`OK`	
←	`+WSCNPDONE:1`	Periodic scanning stopped after 1 sweep
→	`AT+WSTA=1`	Now accepted
←	`OK`	
←	`+WSTALU:2,"00:01:02:03:04:05",6`	
←	`+WSTAAIP:2,"192.168.0.20"`	

Example Command Sequence 77: WPA Enterprise: Configure a TLS context for establishing a tunnel

→	`AT+TLSC=1,1,"radius_cert"`	Specify file name of CA certificate
←	`OK`	
→	`AT+TLSC=1,6,"wifi.alpha.net"`	Specify authentication server’s domain name
←	`OK`	
→	`AT+TIME=3`	Query to ensure time is valid (knowledge of current time is needed for TLS)
←	`OK`	
←	`+TIME:"2025-10-24T18:03:43.00Z"`	

 ©2023-2026

Example Command Sequence 78: WPA Enterprise: Configure STA connection parameters for MSCHAPv2

→	`AT+WSTAC=1,"AUTO-WPA3-ENTERPRISE"`	Specify SSID of Access Point
←	`OK`	
→	`AT+WSTAC=2,9`	Set security type to WPA3-Enterprise
←	`OK`	
→	`AT+WSTAC=20,"Anonymous@wifi.alpha.net"`	Set Enterprise identity, to be sent unprotected (anonymous username)
←	`OK`	
→	`AT+WSTAC=21,"my_username@wifi.alpha.net"`	Set Enterprise identity, to be sent protected inside secure tunnel (real username)
←	`OK`	
→	`AT+WSTAC=26,1`	Set TLSC config index to use for establishing the tunnel
←	`OK`	
→	`AT+WSTAC=30,1`	Set credential type to MSCHAPv2
←	`OK`	
→	`AT+WSTAC=32,"my_username"`	Set MSCHAPv2 username
←	`OK`	
→	`AT+WSTAC=33,"my_password"`	Set MSCHAPv2 password
←	`OK`	

Example Command Sequence 79: WPA Enterprise: Configure a TLS context for EAP-PEAPv1/TLS

→	`AT+TLSC=2,1,"radius_cert"`	Specify file name of CA certificate
←	`OK`	
→	`AT+TLSC=2,2,"my_cert"`	Specify file name of user's certificate
←	`OK`	
→	`AT+TLSC=2,3,"my_key"`	Specify file name of user's private key
←	`OK`	
→	`AT+TLSC=2,6,"wifi.alpha.net"`	Specify authentication server's domain name
←	`OK`	
→	`AT+TIME=3`	Query to ensure time is valid (knowledge of current time is needed for TLS)
←	`OK`	
←	`+TIME:"2025-10-24T18:03:43.00Z"`	

Example Command Sequence 80: WPA Enterprise: Configure STA connection parameters for EAP-PEAPv1/TLS

→	`AT+WSTAC=1,"AUTO-WPA3-ENTERPRISE"`	Specify SSID of Access Point
←	`OK`	
→	`AT+WSTAC=2,9`	Set security type to WPA3-Enterprise
←	`OK`	
→	`AT+WSTAC=20,"Anonymous@wifi.alpha.net"`	Set EAP identity to use during tunnel establishment (anonymous username)
←	`OK`	
→	`AT+WSTAC=21,"my_username@wifi.alpha.net"`	Set EAP identity to use inside secure tunnel (real username)
←	`OK`	
→	`AT+WSTAC=25,4`	Restrict tunnel method to PEAPv1
←	`OK`	
→	`AT+WSTAC=26,2`	Set TLSC config index to use for establishing the tunnel
←	`OK`	
→	`AT+WSTAC=30,0`	Set credential type to TLS
←	`OK`	
→	`AT+WSTAC=31,2`	Set TLSC config index to use for authenticating the client
←	`OK`	

 ©2023-2026

ASSOC (Module ID = 22)

Introduction

The ASSOC module lists the current Wi-Fi associations of the RNWF02. These are the station's association with its access point, reported with AP_STA set to 0, and the soft AP's connected clients, reported with AP_STA set to 1.

Each association is reported by an +ASSOC AEC which includes its current RSSI. +ASSOC=<ID>,0 disconnects an association.

The association IDs are the same as those reported by the WSTA and WAP modules.

Command Reference:

+ASSOC

Description

This command is used to query or disconnect WiFi associations.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+ASSOC	Query all associations	GGGG
AT+ASSOC=<ASSOC_ID>	Query specific association ID	GGGG
AT+ASSOC=<ASSOC_ID>,<STATE>	Disconnect association	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ASSOC_ID>	Integer	Association ID Positive unsigned 16-bit value
<STATE>	Integer	State to set 0 Disconnect the association.

 ©2023-2026

AEC Reference:

+ASSOC

Description

WiFi association report.

AEC Syntax

AEC	Description
+ASSOC:<ASSOC_ID>,<AP_STA>,<BSSID>,<RSSI>	WiFi association report

AEC Element Syntax

Element Name	Type	Description
<ASSOC_ID>	Integer	Association ID Positive unsigned 16-bit value
<AP_STA>	Integer String Byte Array	AP or STA 0 AP. 1 STA.
<BSSID>	String	The BSSID of association peer
<RSSI>	Integer	RSSI

SI (Module ID = 23)

Introduction

The SI module reports system information. +SI returns the system uptime in its response. The FILTER parameter selects additional statistics which are reported as AECs:

- Wi-Fi memory pool usage, +SIMERR and +SIMSTAT.
- Wi-Fi packet counts, +SIWPKTS.
- Heap usage, +SIHERR and +SIHEAP.
- Stack usage, +SISTACK.

If no filter is given all statistics are reported, a filter of 0 reports only the uptime.

 ©2023-2026

Command Reference:

+SI

Description

This command is used to query system information.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SI	Request system information	GGGG
AT+SI=<FILTER>	Request filtered system information	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<FILTER>	Integer	System information filter bitmask 0x01 WiFi memory allocation stats. 0x02 WiFi packet stats. 0x04 Heap allocation stats. 0x08 Stack usage stats.

Response Syntax

Response	Description
+SI:<UPTIME>	Response

Response Element Syntax

Element Name	Type	Description
<UPTIME>	Integer	System uptime in seconds

AEC Reference:

+SIMSTAT

Description

WiFi memory statistics for each type of memory pool.

AEC Syntax

AEC	Description
+SIMSTAT:<TYPE>,<ALLOC>,<FREE>,<ALLOC_SZ>,<FREE_SZ>	WiFi memory statistic

AEC Element Syntax

Element Name	Type	Description
<TYPE>	Integer	Type 0 Memory. 1 Config. 2 High priority TX. 3 High priority RX. 4 Normal priority TX. 5 Normal priority RX.
<ALLOC>	Integer	Number of memory allocations made
<FREE>	Integer	Number of memory frees
<ALLOC_SZ>	Integer	Total bytes allocated
<FREE_SZ>	Integer	Total bytes freed

 ©2023-2026

+SIMERR

Description

WiFi Memory errors.

AEC Syntax

AEC	Description
+SIMERR:<ALLOC_ERR>	WiFi memory errors

AEC Element Syntax

Element Name	Type	Description
<ALLOC_ERR>	Integer	Memory allocation errors

+SIWPKTS

Description

WiFi packets statistics.

AEC Syntax

AEC	Description
+SIWPKTS:<TX>,<RX>	WiFi packets

AEC Element Syntax

Element Name	Type	Description
<TX>	Integer	WiFi packets transmitted
<RX>	Integer	WiFi packets received

 ©2023-2026

+SIHEAP

Description

Heap allocation statistics.

AEC Syntax

AEC	Description
+SIHEAP:<ALLOC>,<FREE>,<ALLOC_SZ>,<FREE_SZ>,<PEAK_ALLOC_SZ>	Heap statistics

AEC Element Syntax

Element Name	Type	Description
<ALLOC>	Integer	Number of memory allocations made
<FREE>	Integer	Number of memory frees
<ALLOC_SZ>	Integer	Total bytes allocated
<FREE_SZ>	Integer	Total bytes freed
<PEAK_ALLOC_SZ>	Integer	Peak bytes allocated

+SIHERR

Description

Heap allocation errors.

AEC Syntax

AEC	Description
+SIHERR:<ALLOC_ERR>	Heap errors

AEC Element Syntax

Element Name	Type	Description
<ALLOC_ERR>	Integer	Memory allocation errors

 ©2023-2026

+SISTACK

Description

Stack usage.

AEC Syntax

AEC	Description
+SISTACK:<STACK_MIN_RESERVED>,<STACK_RESERVED>,<STACK_USAGE>	Stack statistics

AEC Element Syntax

Element Name	Type	Description
<STACK_MIN_RESERVED>	Integer	Minimum stack reserved memory
<STACK_RESERVED>	Integer	Stack reserved memory
<STACK_USAGE>	Integer	Stack usage

WPROV (Module ID = 25)

Introduction

The WPROV module provides a TCP server which allows a remote client to send commands to the RNWF02 over the network, using either the ASCII or binary command format. It may be used, for example, to configure the station from a phone connected to the soft AP.

1. Bring up a network interface with WAP or WSTA.
2. Configure the service with +WPROVC. The configuration cannot be changed while the service is running.
3. Start the service with +WPROV=1.
4. When a client connects the +WPROVAT AEC is sent; the client can then send commands, and receives their responses, over the connection.
5. When the client disconnects the +WPROVDT AEC is sent.

+WPROV=0 stops the service. It cannot be sent over the provisioning connection itself.

 ©2023-2026

Command Reference:

+WPROVC

Description

This command is used to read or set the provisioning service configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WPROVC	Query all configuration elements	GGGG
AT+WPROVC=<ID>	Query a single element	GGGG
AT+WPROVC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+WPROVC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
1	<PORT>	Unsigned Integer	Service listening port Positive unsigned 16-bit value	GGGG
2	<ASCII>	Bool	ASCII or binary protocol 0 Binary protocol. 1 ASCII protocol.	GGGG
3	<PROTOCOL_VERSION>	Unsigned Integer	IP protocol version 4 IPv4. 6 IPv6.	GGGG

 ©2023-2026

+WPROV

Description

This command is used to control or query the provisioning service.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WPROV	Read status of provisioning service	GGGG
AT+WPROV=<STATE>	Set state of provisioning service	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<STATE>	Integer	State of the provisioning service 0 Disable. 1 Use configuration from +WPROVC.

AEC Reference:

+WPROVAT

Description

Indication of client attach to provisioning service.

AEC Syntax

AEC	Description
+WPROVAT:<ID>,<IP_ADDRESS>,<PORT>	Attachment of client to provisioning service

AEC Element Syntax

Element Name	Type	Description
<ID>	Integer	Client ID
<IP_ADDRESS>	String	IP address assigned
<PORT>	Integer	Port

 ©2023-2026

+WPROVDT

Description

Indication of client detach from provisioning service.

AEC Syntax

AEC	Description
+WPROVDT:<ID>	Detachment of client from provisioning service

AEC Element Syntax

Element Name	Type	Description
<ID>	Integer	Client ID

DI (Module ID = 26)

Introduction:

Firmware image sequence numbers

Contained within each IMAGE_INFO is a SEQ_NUM value, this represents the image sequence number. The boot process prefers the image with the lower sequence number.

NOTE

The following sequence numbers are invalid:

- 0
- 0xFFFFFFFF

 ©2023-2026

Command Reference:

+DI

Description

This command is used to query the device information.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+DI	Query all configuration elements	GGGG
AT+DI=<ID>	Query a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value

Response Syntax

Response	Description
+DI:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
1	<DEVICE_ID>	Unsigned Integer (Read Only)	Device ID	GGGG
14	<NUM_IMAGES>	Integer (Read Only)	Number of images	GGGG
15	<IMAGE_INFO>	Complex Value[1] (Read Only)	Image information This is a multiple value parameter with an ID range 15.0 to 15.1	GGGG

Complex Value Syntax

No.	Complex Value	Description
1	<SEQ_NUM>,<VERSION>,<SRC_ADDR>,<IMG_STATE>	Image information

Complex Value Element Syntax

Element Name	Type	Description
<SEQ_NUM>	Integer	Sequence number
<VERSION>	Integer	Version information
<SRC_ADDR>	Integer	Source address
<IMG_STATE>	Integer	Image state 0 The image has not yet been activated. 1 The image will run on next boot (subject to verification). 2 The image is currently running. 3 The image will run on next boot (subject to verification) if the current image is invalidated.

 ©2023-2026

Examples:

Example Command Sequence 81: Query device information with +DI

→	`AT+DI`	Query all device information
←	`+DI:1,0x29C70053`	Device ID
←	`+DI:14,2`	2 images
←	`+DI:15.0,0xFFEEDDC0,0x00010200,0x60000000,2`	Image info for currently running image
←	`+DI:15.1,0xFFFFFFFF,0x00010300,0xFFFFFFFF,0`	Image info for latent image
←	`OK`	

EXTCRYPTO (Module ID = 27)

Introduction

Cryptographic operations can be delegated to an external crypto device controlled by the host application, providing secure key storage and hardware-accelerated public key operations. This can be a Microchip ATECC608 Trust&GO (TNGTLS) secure element, which is fitted on some module variants (see the module data sheet), or an equivalent third-party device.

The ATECC608 Trust&GO is pre-provisioned, featuring ECDSA signing, ECC private key secure storage and X.509 compressed certificate storage. The host application accesses the secure element directly through its I2C or SWI interface; the RNWF02 does not communicate with it.

The +EXTCRYPTO AEC allows the DTE to receive a crypto operation request from the DCE. The +EXTCRYPTO command is used to send the operation result back to the DCE.

Crypto operation requests are enabled via the EXTCRYPTO_OPS parameter of the +TLSC command. The only supported operation is ECDSA signing.

Using External Crypto for Secure Signing

To leverage the crypto device for secure signing, the application should follow these steps:

1. Retrieve from the crypto device the X.509 certificate that includes the public key corresponding to the securely stored private key.
2. Send the X.509 certificate to the DCE using the +FS command.
3. Enable secure signing via the external crypto device and specify the certificate in the TLS configuration.

   See example configuring TLS with external crypto.

4. Respond to signing requests from the +EXTCRYPTO AEC with the +EXTCRYPTO command.

   See example handling signing request and sending response.

NOTE	The ECC curve used for signing is selected according to the certificate public key. Refer to the CURVE element in the +EXTCRYPTO AEC for the list of supported curves.

NOTE	The application code is responsible for interacting with the external crypto device to retrieve the certificate and forward signing requests and responses.

 ©2023-2026

Command Reference:

+EXTCRYPTO

Description

This command is used to provide the result of an external crypto operation.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+EXTCRYPTO=<OP_ID>,<STATUS>	Provide the result of a failed external signing operation	GGGG
AT+EXTCRYPTO=<OP_ID>,<STATUS>,<SIGNATURE>	Provide the result of a successful external signing operation	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<OP_ID>	Integer	Operation identifier, matching <OP_ID> in corresponding AEC Positive unsigned 16-bit value
<STATUS>	Integer	Operation success or failure 0 Operation succeeded. 1 Operation failed.
<SIGNATURE>	Byte Array	Signature (big endian). For ECDSA signatures: R then S, each the size of the curve.

AEC Reference:

+EXTCRYPTO

Description

This event is used to request an external crypto operation.

AEC Syntax

AEC	Description
+EXTCRYPTO:<OP_ID>,<OP_TYPE>,<OP_SOURCE_TYPE>,<OP_SOURCE_ID>,<SIGN_TYPE>,<CURVE>,<SIGN_VALUE>	Request an external signing operation

AEC Element Syntax

Element Name	Type	Description
<OP_ID>	Integer	Operation identifier, to be included in EXTCRYPTO CMD Positive unsigned 16-bit value
<OP_TYPE>	Integer	The type of operation 1 Signing.
<OP_SOURCE_TYPE>	Integer	The type of source requesting the operation 1 A TLSC configuration.
<OP_SOURCE_ID>	Integer	The ID of the source requesting the operation (the TLS configuration index for TLSC) Valid range is 1 to 4
<SIGN_TYPE>	Integer	The type of signing operation 1 ECDSA.
<CURVE>	Integer	The ECDSA curve 1 Curve secp256r1. 2 Curve secp384r1. 3 Curve secp521r1.
<SIGN_VALUE>	Byte Array	The value to be signed (big endian)

 ©2023-2026

Examples:

Example Command Sequence 82: Example of TLS configuration for secure signing via external crypto operations

→	`AT+TLSC=2,41,1`	Enable server domain verification
←	`OK`	
→	`AT+TLSC=2,6,"ServerDomain"`	Specify server domain name
←	`OK`	
→	`AT+TLSC=2,40,1`	Enable server authentication
←	`OK`	
→	`AT+TLSC=2,1,"ServerRootCACert"`	Specify file name of CA certificate
←	`OK`	
→	`AT+TLSC=2,2,"EccClientCert"`	Specify file name of ECC client certificate
←	`OK`	
→	`AT+TLSC=2,20,1`	Enable signing via external crypto
←	`OK`	

Example Command Sequence 83: Example of signing request/response using +EXTCRYPTO

←	`+EXTCRYPTO:123,1,1,2,1,1,[0123…cdef]`	Crypto request ID 123: ECDSA signing using curve secp256r1, for TLS config 2
→	`AT+EXTCRYPTO=123,0,[0123…cdef]`	Response to crypto request ID 123
←	`OK`	

WIFI (Module ID = 28)

Introduction

The Wi-Fi module provides configuration settings for:

- Wi-Fi power save
- Bluetooth/Wi-Fi coexistence
- Regulatory domain
- Vendor-specific information elements (IEs)

Configure these settings with the `AT+WIFIC` command.

Vendor-specific IEs are supported in management frames in both station (STA) and SoftAP modes. The transmit and receive settings are independent.

Transmitting vendor-specific IEs

Configure transmission with VSIE_TX_MASK, VSIE_TX_DATA, and VSIE_TX_DATA_COMMIT.

- VSIE_TX_MASK selects the frame types in which the IEs are transmitted. The IEs are added to frames of the selected types that the RNWF02 transmits. In SoftAP mode these are beacon and probe response frames; in STA mode, probe request frames.
- VSIE_TX_DATA holds up to four IE payloads. Each payload starts with a three-byte organizationally unique identifier (OUI), followed by up to 252 bytes of vendor-specific data.
- VSIE_TX_DATA_COMMIT activates the staged IE payloads.

Changes to VSIE_TX_DATA are applied to a staged set and have no effect until VSIE_TX_DATA_COMMIT is set. Any change to the staged set clears the commit state; the previously committed IEs remain active until the new set is committed. Reading VSIE_TX_DATA returns the active entries when VSIE_TX_DATA_COMMIT is set and the staged entries otherwise.

Clearing VSIE_TX_MASK or committing an empty VSIE_TX_DATA set disables vendor-specific IE transmission.

Both VSIE_TX_MASK and the IE payloads can be updated at run time.

See example configuring vendor-specific IEs for transmission.

 ©2023-2026

Receiving vendor-specific IEs

Configure reception with VSIE_RX_MASK and VSIE_RX_OUI.

- VSIE_RX_MASK selects the frame types from which IEs are received. Set it to `0` to stop receiving IEs.
- VSIE_RX_OUI is the three-byte OUI used to filter received IEs. Matching IEs from the selected frame types are reported by the +WIFIVSIERX AEC.

In SoftAP mode, IEs can be received from all three supported frame types: beacons, probe requests, and probe responses. In STA mode, reception additionally depends on the connection and scan state:

- While connected, beacon IEs are received only from the associated AP. Beacons from other APs are ignored.
- While disconnected and not scanning, all beacons are ignored.
- During a passive scan, beacon IEs are received from any AP that transmits a beacon within the scan interval.
- During an active scan, IEs are received in probe responses (scan responses).

See example configuring vendor-specific IE reception.

Command Reference:

+WIFIC

Description

This command is used to read or set the device's Wi-Fi configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WIFIC	Query all configuration elements	GGGG
AT+WIFIC=<ID>	Query a single element	GGGG
AT+WIFIC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+WIFIC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th><th>Sec</th></tr>
<tr><td>0</td><td><PRESET></td><td>Integer</td><td>Configuration preset

0 Default.</td><td>GGGG</td></tr>
<tr><td>10</td><td><REGDOMAIN_SELECTED></td><td>String</td><td>Regulatory domain selected
Maximum length of string is 6</td><td>GGGG</td></tr>
<tr><td>11</td><td><REGDOMAIN_AVAILABLE></td><td>String
(Read Only)</td><td>Regulatory domains available
Maximum length of string is 6
This is a multiple value parameter
with an ID range 11.0 to 11.5</td><td>GGGG</td></tr>
<tr><td>12</td><td><REGDOMAIN_CHANMASK24></td><td>Unsigned Integer
(Read Only)</td><td>Regulatory channel mask (2.4 GHz)

<table>
<tr><td>15
-</td><td>14
-</td><td>13
CH14</td><td>12
CH13</td><td>11
CH12</td><td>10
CH11</td><td>9
CH10</td><td>8
CH9</td></tr>
<tr><td>7
CH8</td><td>6
CH7</td><td>5
CH6</td><td>4
CH5</td><td>3
CH4</td><td>2
CH3</td><td>1
CH2</td><td>0
CH1</td></tr>
</table>

<table>
<tr><th>Value</th><th>Label</th><th>Description</th></tr>
<tr><td>0x0001</td><td>CH1</td><td>Channel 1 - 2412 MHz.</td></tr>
<tr><td>0x0002</td><td>CH2</td><td>Channel 2 - 2417 MHz.</td></tr>
<tr><td>0x0004</td><td>CH3</td><td>Channel 3 - 2422 MHz.</td></tr>
<tr><td>0x0008</td><td>CH4</td><td>Channel 4 - 2427 MHz.</td></tr>
<tr><td>0x0010</td><td>CH5</td><td>Channel 5 - 2432 MHz.</td></tr>
<tr><td>0x0020</td><td>CH6</td><td>Channel 6 - 2437 MHz.</td></tr>
<tr><td>0x0040</td><td>CH7</td><td>Channel 7 - 2442 MHz.</td></tr>
<tr><td>0x0080</td><td>CH8</td><td>Channel 8 - 2447 MHz.</td></tr>
<tr><td>0x0100</td><td>CH9</td><td>Channel 9 - 2452 MHz.</td></tr>
<tr><td>0x0200</td><td>CH10</td><td>Channel 10 - 2457 MHz.</td></tr>
<tr><td>0x0400</td><td>CH11</td><td>Channel 11 - 2462 MHz.</td></tr>
<tr><td>0x0800</td><td>CH12</td><td>Channel 12 - 2467 MHz.</td></tr>
<tr><td>0x1000</td><td>CH13</td><td>Channel 13 - 2472 MHz.</td></tr>
<tr><td>0x2000</td><td>CH14</td><td>Channel 14 - 2485 MHz.</td></tr>
</table>
</td><td>GGGG</td></tr>
<tr><td>20</td><td><POWERSAVE></td><td>Bool</td><td>Power Save Mode

0 Off.
1 On.</td><td>GGGG</td></tr>
<tr><td>21</td><td><POWERSAVE_LISTEN_INTERVAL></td><td>Unsigned Integer</td><td>Listen Interval in units of beacon period

Valid range is 3 to 100</td><td>GGGG</td></tr>
</table>

ID	Name	Type	Description	Sec
22	<POWERSAVE_INACT_LIMIT>	Unsigned Integer	Inactivity Time Limit - the sleep inactivity threshold/limit for power-save operation in seconds Valid range is 10 to 6000	GGGG
30	<COEX_ENABLED>	Bool	BT/Wi-Fi coexistence arbiter 0 Disabled. 1 Enabled.	GGGG
31	<COEX_INTERFACE_TYPE>	Bool	BT/Wi-Fi coexistence arbiter interface type 0 3-wire interface (BT_Act, BT_Prio, WLAN_Act). 1 2-wire interface (BT_Prio, WLAN_Act).	GGGG
32	<COEX_WLAN_RX_VS_BT>	Bool	BT/Wi-Fi coexistence arbiter WLAN Rx priority over BT Low Priority 0 WLAN Rx priority lower than BT Low Priority. 1 WLAN Rx priority higher than BT Low Priority.	GGGG
33	<COEX_WLAN_TX_VS_BT>	Bool	BT/Wi-Fi coexistence arbiter WLAN Tx priority over BT Low Priority 0 WLAN Tx priority lower than BT Low Priority. 1 WLAN Tx priority higher than BT Low Priority.	GGGG
34	<COEX_ANTENNA_MODE>	Bool	BT/Wi-Fi coexistence arbiter antenna mode 0 Dedicated antenna. 1 Shared antenna.	GGGG
40	<AMPDU_TX_ENABLED>	Bool	Wi-Fi A-MPDU Tx Enable 0 Disabled. 1 Enabled.	GGGG
50	<VSIE_TX_MASK>	Unsigned Integer	Frame types carrying the transmitted vendor-specific IEs, bitmask of: 0x01 Beacon. 0x02 Probe request. 0x04 Probe response.	GGGG
51	<VSIE_TX_DATA>	Byte Array	Vendor-specific IE data This is a multiple value parameter with an ID range 51.0 to 51.3	GGGG

 ©2023-2026

ID	Name	Type	Description	Sec
52	<VSIE_TX_DATA_COMMIT>	Bool	Commit the vendor-specific IE transmit data to the device. Modifying the transmit data clears this element, it must be set again for the new data to be transmitted 0 Transmit IE data has changed since it was last committed. 1 Transmit IE data is live.	GGGG
55	<VSIE_RX_MASK>	Unsigned Integer	Frame types filtered for vendor-specific IE reception, bitmask of: 0x01 Beacon. 0x02 Probe request. 0x04 Probe response.	GGGG
56	<VSIE_RX_OUI>	Byte Array	Receive vendor-specific IE OUI filter	GGGG

AEC Reference:

+WIFIVSIERX

Description

Received vendor-specific IE.

AEC Syntax

AEC	Description
+WIFIVSIERX:<FRAME_TYPE>,<SOURCE_MAC>,<RSSI>,<DATA>	Received vendor-specific IE

AEC Element Syntax

Element Name	Type	Description
<FRAME_TYPE>	Integer	Management frame type 1 Beacon. 2 Probe request. 4 Probe response.
<SOURCE_MAC>	String	Source MAC address
<RSSI>	Integer	Received signal strength in dBm
<DATA>	Byte Array	Vendor-specific IE data

 ©2023-2026

Examples:

Example Command Sequence 84: Settings with +WIFIC

→	`AT+WIFIC=10,"USA"`	Configure regulatory domain to USA
←	`OK`	
→	`AT+WIFIC=10,"JPN"`	Configure regulatory domain to JPN
←	`OK`	
→	`AT+WIFIC=10`	Query regulatory domain
←	`+WIFIC:10,"JPN"`	
←	`OK`	
→	`AT+WIFIC=20`	Query power save mode
←	`+WIFIC:20,0`	
←	`OK`	
→	`AT+WIFIC=20,1`	Configure power save on
←	`OK`	

Example Command Sequence 85: +WIFIC coexistence settings for 3-wire PTA interface

→	`AT+WIFIC=31,0`	Configure 3-wire interface
←	`OK`	
→	`AT+WIFIC=32,1`	WLAN RX priority higher than BT low priority
←	`OK`	
→	`AT+WIFIC=33,1`	WLAN TX priority higher than BT low priority
←	`OK`	
→	`AT+WIFIC=34,1`	Shared antenna
←	`OK`	
→	`AT+WIFIC=30,1`	Enable coex
←	`OK`	

Example Command Sequence 86: Configure vendor-specific information elements for transmission

→	`AT+WIFIC=50,0x1`	Select beacon frames for VSIE transmission
←	`OK`	
→	`AT+WIFIC=51,[01020301AABBCC]`	Stage OUI 01-02-03, type 01, and data AABBCC
←	`OK`	
→	`AT+WIFIC=51,[01020302DDEEFF]`	Stage OUI 01-02-03, type 02, and data DDEEFF
←	`OK`	
→	`AT+WIFIC=51`	Read the staged VSIE data
←	`+WIFIC:51.0,[01020301AABBCC]`	
←	`+WIFIC:51.1,[01020302DDEEFF]`	
←	`OK`	
→	`AT+WIFIC=52,1`	Commit the staged data, the entries become live
←	`OK`	
→	`AT+WIFIC=51.0,[01020301010101]`	Amend the data following the OUI of entry 0
←	`OK`	
→	`AT+WIFIC=51`	Read the staged VSIE data
←	`+WIFIC:51.0,[01020301010101]`	
←	`+WIFIC:51.1,[01020302DDEEFF]`	
←	`OK`	
→	`AT+WIFIC=52,1`	Commit again to make the amended data live
←	`OK`	

Example Command Sequence 87: Configure vendor-specific information element reception

→	`AT+WIFIC=55,0x1`	Receive from beacon frames
←	`OK`	
→	`AT+WIFIC=56,[001122]`	Set RX OUI filter
←	`OK`	
←	`+WIFIVSIERX:1,"00:11:22:33:44:55",-55,[001122AABB]`	

 ©2023-2026

NVM (Module ID = 29)

Introduction

The NVM module provides access to the alternate firmware partition, the partition which does not contain the running firmware image. All offsets are relative to the start of the partition, its size is reported by the NUM_SECTORS and SECTOR_SZ configuration values.

- +NVMER erases whole 4 KB sectors. The erase runs in the background and completes with the +NVMER AEC, or +NVMERR on failure.
- +NVMWR writes up to 1024 bytes and completes immediately. A write must not cross a sector boundary, and the area must have been erased first. The data is read back and verified after writing.
- +NVMRD reads up to 1024 bytes, which are returned in the command response.
- +NVMCHK calculates a CRC16 or SHA digest over an area of the partition, as selected by CHECK_MODE. The calculation runs in the background and the result is reported by the +NVMCHK AEC, or +NVMERR on failure.

Only one erase, write, read or check operation may run at a time, a new operation fails while an erase or check is in progress. After +OTAINV erase and write are locked until the next reset.

Command Reference:

+NVMC

Description

This command is used to read or set the NVM configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+NVMC	Query all configuration elements	GGGG
AT+NVMC=<ID>	Query a single element	GGGG
AT+NVMC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+NVMC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<START_OFFSET>	Unsigned Integer (Read Only)	Starting address offset	GGGG
2	<NUM_SECTORS>	Unsigned Integer (Read Only)	Number of sectors	GGGG
3	<SECTOR_SZ>	Unsigned Integer (Read Only)	Sector size	GGGG
4	<CHECK_MODE>	Unsigned Integer	Checking mode 2 CRC16. 10 SHA. 11 SHA256. 12 SHA224. 13 SHA512. 14 SHA384.	GGGG

+NVMER

Description

This command allows sector erase within the alternate firmware partition.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+NVMER=<SECTOR_OFFSET>,<SECTORS>	NVM erase sectors	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<SECTOR_OFFSET>	Integer	Sector offset to be used for the NVM operation Valid range is 0 to 239
<SECTORS>	Integer	The number of sectors to erase during NVM operation Valid range is 1 to 240

 ©2023-2026

+NVMWR

Description

This command allows data to be written to the alternate firmware partition. Writes are performed as normal for flash devices, therefore it is recommended to erase the sector of data before performing a write. Write operations cannot cross a sector boundary.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+NVMWR=<OFFSET>,<LENGTH>,<DATA>	NVM write with offset, length and data	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<OFFSET>	Integer	Byte offset to be used for the NVM operation Valid range is 0 to 0x000EFFFF
<LENGTH>	Integer	The number of bytes for the NVM operation Valid range is 1 to 1024
<DATA>	String Byte Array	The data to write in hexadecimal string format Valid range is 1 to 1024

+NVMCHK

Description

This command allows checking of the alternate firmware partition.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+NVMCHK=<OFFSET>,<LENGTH>	NVM check	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<OFFSET>	Integer	Byte offset to be used for the NVM operation Valid range is 0 to 0x000EFFFF
<LENGTH>	Integer	The number of bytes for the NVM operation Valid range is 1 to 0x000F0000

 ©2023-2026

+NVMRD

Description

This command allows reading of the alternate firmware partition.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+NVMRD=<OFFSET>,<LENGTH>	NVM read	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<OFFSET>	Integer	Byte offset to be used for the NVM operation Valid range is 0 to 0x000EFFFF
<LENGTH>	Integer	The number of bytes for the NVM operation Valid range is 1 to 1024

Response Syntax

Response	Description
+NVMRD:<OFFSET>,<LENGTH>,<DATA>	Read response

Response Element Syntax

Element Name	Type	Description
<OFFSET>	Integer	Byte offset Valid range is 0 to 0x000EFFFF
<LENGTH>	Integer	The number of bytes Valid range is 1 to 1024
<DATA>	String Byte Array	Data bytes Valid range is 1 to 1024

AEC Reference:

+NVMER

Description

NVM Erase.

AEC Syntax

AEC	Description
+NVMER	NVM erase success

 ©2023-2026

+NVMCHK

Description

NVM Check.

AEC Syntax

AEC	Description
+NVMCHK:<OFFSET>,<LENGTH>,<CHECK_MODE>,<HASH>	Hash response

AEC Element Syntax

Element Name	Type	Description
<OFFSET>	Integer	Byte offset Valid range is 0 to 0x000EFFFF
<LENGTH>	Integer	The number of bytes Valid range is 1 to 0x000F0000
<CHECK_MODE>	Integer	Checking mode used 2 CRC16. 10 SHA. 11 SHA256. 12 SHA224. 13 SHA512. 14 SHA384.
<HASH>	String Byte Array	Hash response

+NVMERR

Description

NVM error.

AEC Syntax

AEC	Description
+NVMERR:<ERROR_CODE>	NVM error

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	Integer	Error code See Status Response Codes

 ©2023-2026

Examples:

Example Command Sequence 88: Simple example of NVM usage

→	`AT+NVMER=0,1`	Erase first sector
←	`OK`	
←	`+NVMER`	Erase complete AEC
→	`AT+NVMWR=0,4,[4D434850]`	Write 4 bytes of data to offset 0
←	`OK`	
→	`AT+NVMCHK=0,4`	Request CRC16 checksum, 4 bytes from offset 0
←	`OK`	
←	`+NVMCHK:0x00000000,4,2,0xCD77`	CRC16 checksum AEC
→	`AT+NVMC=4,11`	Set check mode to SHA256
←	`OK`	
→	`AT+NVMCHK=0,4`	Request SHA256 hash, 4 bytes from offset 0
←	`OK`	
←	`+NVMCHK:0x00000000,4,11,[7BE8FA11F537A1CA4612D6E30CAF94A20F3C37999012119672BE21DAD2791D5C]`	SHA256 AEC
→	`AT+NVMRD=0,4`	Read 4 bytes of data from offset 0
←	`+NVMRD:0x00000000,4,"MCHP"`	Read data response
←	`OK`	

DFU (Module ID = 30)

Introduction

Secure Device Firmware Update (DFU) is a process for upgrading the firmware of a secured device.

The RNWF02 device has non-volatile memory (NVM) with two partitions which may be used to store firmware images.

The partitions can be referred to as "low" and "high" based on their physical addresses. They can also be referred to as "current" and "alternate" based on their usage at a given time; in normal operation, the "current" partition contains a copy of the currently running firmware image, while the "alternate" partition may be blank or may contain a rollback image, a latent image or any other data.

Region	Address	Size
Low partition	0x60000000	960 KB (0xF0000)
High partition	0x600F0000	960 KB (0xF0000)
File system	0x601E0000	Not affected by DFU

The DFU process should involve the following steps:

1. Determine the NVM partition in which to program the firmware image.
2. Prepare the firmware image.
3. Boot the device in DFU mode.
4. Erase and program the NVM partition.
5. Reset the device.

Determine the NVM partition in which to program the firmware image

The appropriate NVM partition ("alternate", "current", "low" or "high") depends on the user's upgrade policy. Two common upgrade policies are:

- A "flip-flop" policy in which the new firmware image is always programmed to the "alternate" partition.
- A "golden image" policy in which the initial firmware image is programmed to one partition (based on physical address), and subsequent firmware images are always programmed to the other partition.

 ©2023-2026

Prepare the firmware image

Locate the released rnwf02_dfu_high.bin file for the firmware version to which you want to upgrade.

Read the firmware image from the file, then update two values in the image header. Both are 32-bit little endian values:

Offset	Length	Field
0	4	Sequence Number
20	4	Firmware Image Source Address

- Sequence Number: The Sequence Number is the boot priority of the firmware image. Use the +DFUSEQ command to get the offset, length and value for updating it.
- Firmware Image Source Address: The Firmware Image Source Address must match the physical address of the NVM partition in which the firmware image is to be programmed. Use the +DFUADR command, with the desired partition, to get the offset, length and value for updating it.

Neither field is covered by the image signatures, so updating them does not invalidate the image. No other part of the image may be changed.

The image with the lowest Sequence Number has the highest boot priority. The Sequence Number of a latent image is 0xFFFFFFFF; a latent image is never booted until it is activated. For an activated image, the value returned is 0x10 lower than the lowest Sequence Number of any valid image already in the NVM, so the new image boots on the next reset. The lowest four bits of the Sequence Number are used by the boot ROM and are always returned as 0.

NOTE A status code of DFU_ADDRESS_WARNING is a warning that the other partition does not contain a bootable firmware image. The offset, length and value in the response are still valid and should be used. If the user proceeds in spite of this warning and step 4 does not complete for any reason, the device will reach a state in which it contains no bootable firmware images. In that situation, repeating step 4 should recover the device, so long as the user still has the firmware image available.

NOTE Save the value of the NVM partition address for use during step 4.

Boot the device in DFU mode

Boot the device in DFU mode using the following steps:

1. Assert RNWF02 pin NMCLR (low).
2. Write to RNWF02 pins PB0 (Clock) and PB1 (Data) to send the DFU Test Mode Entry pattern "MCHP".
3. Deassert RNWF02 pin NMCLR.

In DFU mode the device runs the Programming Executive (PE), which communicates over a UART at 230400 baud, 8 data bits, no parity, 1 stop bit.

Erase and program the NVM partition

Configure pins PB0 and PB1 for UART Rx and Tx respectively, from the RNWF02's point of view: the host transmits to PB0 and receives from PB1.

Use Programming Executive (PE) commands to erase the NVM partition, at the address obtained during step 2. Erase only the number of pages needed for the image.

WARNING	Do not use the PE chip erase command. It erases both partitions and the file system.

Use Programming Executive (PE) commands to program the new firmware image into the NVM partition, at the address obtained during step 2.

Reset the device

Reset the device to leave DFU mode. If the image was prepared as activated it is booted; if it was prepared as latent, the previous image continues to run.

The state of the image in each partition can be read with +DI. A latent image in the alternate partition can be activated later with +OTAACT.

 ©2023-2026

Command Reference:

+DFUADR

Description

This command is used to get the parameters for modifying the little endian address field of a DFU image.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+DFUADR=<PARTITION>	Get parameters appropriate for the specified partition	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<PARTITION>	Integer	The partition into which the DFU image will be programmed 0 Alternate partition (recommended). 1 Current partition. 2 Low partition. 3 High partition.

Response Syntax

Response	Description
+DFUADR:<OFFSET>,<LENGTH>,<VALUE>	Parameters for modifying the little endian address field of a DFU image

Response Element Syntax

Element Name	Type	Description
<OFFSET>	Integer	Offset (in bytes) of field in DFU image
<LENGTH>	Integer	Length (in bytes) of field in DFU image
<VALUE>	Integer	Value to write to field in DFU image

+DFUSEQ

Description

This command is used to get the parameters for modifying the little endian sequence number field of a DFU image.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+DFUSEQ=<IMAGE_STATE>	Get parameters appropriate for the specified image state	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<IMAGE_STATE>	Integer	The desired state of the DFU image after programming 0 Latent image (will not run until activated). 1 Activated image.

Response Syntax

Response	Description
+DFUSEQ:<OFFSET>,<LENGTH>,<VALUE>	Parameters for modifying the little endian sequence number field of a DFU image

Response Element Syntax

Element Name	Type	Description
<OFFSET>	Integer	Offset (in bytes) of field in DFU image
<LENGTH>	Integer	Length (in bytes) of field in DFU image
<VALUE>	Integer	Value to write to field in DFU image

 ©2023-2026

Examples:

Example Command Sequence 89: Example of using +DFUADR and +DFUSEQ

→	AT+DFUADR=0	Get parameters for making an image for the alternate partition
←	+DFUADR:20,4,0x600F0000	Offset 20, length 4, value 0x600F0000
←	OK	
	**	
→	AT+DFUADR=1	Get parameters for making an image for the current partition
←	+DFUADR:20,4,0x60000000	Offset 20, length 4, value 0x60000000
←	ERROR:30.0,"No Bootable Image In Other Partition"	
	**	
→	AT+DFUSEQ=0	Get parameters for making a latent image
←	+DFUSEQ:0,4,0xFFFFFFFF	Offset 0, length 4, value 0xFFFFFFFF
←	OK	
	**	
→	AT+DFUSEQ=1	Get parameters for making an activated image
←	+DFUSEQ:0,4,0xFFEEDDB0	Offset 0, length 4, value 0xFFEEDDB0 (lower than running image 0xFFEEDDC0)
←	OK	

PPS (Module ID = 31)

Introduction

The RNWF02 device supports two advanced power-saving modes designed to reduce current consumption in typical IoT and embedded applications:

- **Platform Power-Save (PPS)** mode places the CPU and hardware peripherals into sleep modes - most clocks and peripherals are powered down, leaving only essential hardware blocks active. This drastically reduces power consumption while preserving Wi-Fi connectivity and system context, enabling rapid wake-up and seamless operation.
- **EXtreme Deep Sleep (XDS)** enters the device's lowest power state, with all peripherals powered down and system context lost. This mode achieves maximum power savings for long standby periods where Wi-Fi or network connections are not required. Waking from XDS triggers a full system reboot.

PPS Operation Details

When **PPS** is enabled, the system automatically transitions into PPS mode during idle periods, minimising power while maintaining system and Wi-Fi state. This includes active protocol connections, Wi-Fi association, and buffered data queues.

Wake-Up Triggers

There are a number of events that will cause the RNWF02 to wake when PPS is active:

- Internal timers or internal scheduled events
  - These will be accounted for when PPS calculates its sleep duration for the next period, ensuring it wakes up in time to process the next event/timer
- Host Data Terminal Equipment (DTE) activity over the UART
  - The UART wake protocol (see Wake-up Via Uart) will wake the RNWF02 immediately
- In the absence of internal timers or external triggers, PPS will wake the RNWF02 after 10 seconds

Once awake, the RNWF02 will run until the next idle period, whereby it will transition into PPS mode again.

Wi-Fi Power Save Mode during PPS

When PPS is enabled and Wi-Fi is connected, the RNWF02 operates Wi-Fi Legacy Powersave to minimise power consumption while maintaining network connectivity.

If Wi-Fi power save (POWERSAVE) is not already enabled, enabling PPS enables it. It remains enabled after PPS is disabled. If Wi-Fi power save cannot be enabled, enabling PPS fails with PPS_WIFI_PS_NOT_ENABLED.

 ©2023-2026

Listen Interval

The RNWF02 wakes periodically based on the configured Wi-Fi Listen Interval, which determines how often it wakes to listen for buffered traffic from the AP. By default, this interval is 10 beacon periods, roughly equivalent to 1 second (typical beacon interval is 102.4 ms). The range is 3 to 100 beacon periods.

The listen interval is configured with POWERSAVE_LISTEN_INTERVAL.

Traffic buffered by the AP may be delayed by up to one listen interval. A longer interval saves more power but increases this delay.

Each sleep period lasts between 40 ms and 20 seconds; the RNWF02 does not sleep if the next event is less than 40 ms away.

DTIM (Delivery Traffic Indication Message) Awareness

To efficiently receive broadcast and multicast traffic buffered by the AP, the device aims to wake on the closest DTIM period aligned with the configured listen interval. The DTIM is a special beacon that signals buffered broadcast/multicast data transmission by the AP. For example, if the Listen Interval is set to 10 beacon periods (default) and the AP's DTIM period is 3, the RNWF02 will wake after 9 beacon periods (i.e. on the 3rd DTIM) instead of exactly after 10 beacons. This alignment maximises power savings by minimising unnecessary wake events while ensuring timely reception of important broadcast traffic.

Sleep Timing Accuracy

To optimise power saving and accuracy of wake intervals, it is strongly recommended to connect and enable a secondary oscillator (SOSC) clocked at 32.768 kHz. The SOSC provides a high-accuracy timing source during sleep, allowing for more precise wake-up timing when using longer sleep durations. The crystal is connected to pins PB3 and PB4.

The SOSC shares pin PB4 with the BT_ACT input of 3-wire Wi-Fi/BT coexistence. If SEC_OSC_ENABLED is set while coexistence is enabled in 3-wire mode (COEX_ENABLED and COEX_INTERFACE_TYPE), enabling PPS fails with CONFIG_CONFLICT. Use 2-wire coexistence, or do not enable the SOSC.

PPS Pause

PPS Pause allows temporarily suspending Platform Power-Save operation without fully disabling it. This is useful during periods where continuous device responsiveness is required, such as during critical command sequences, firmware updates or other high-activity times when entering sleep state would be undesirable.

How PPS Pause Works

- When a pause is active, the system remains fully awake, bypassing PPS sleep transitions.
- Multiple pause commands can be issued sequentially, and the pause durations extend the total pause period.
- After all active pause timers expire, the RNWF02 automatically sends the +PPS AEC to indicate that PPS will now resume.
- Once the pause period has expired, the system will return to normal PPS operation, entering low-power sleep during idle periods.

Hardware Connections

Signal	Pin	Direction	Function
WAKE_REQ	PB2 (INT0)	Input	Wake request from the host, active on the falling edge. The RNWF02 enables an internal pull-up while PPS or XDS is active.
WAKE_RDY	PA4	Output	Low when the RNWF02 is ready to receive commands, high while it may be asleep.
SOSC	PB3, PB4	-	Optional 32.768 kHz crystal, see Sleep Timing Accuracy. PB4 is also the BT_ACT input of 3-wire coexistence.
MCLR	MCLR	Input	Reset. Also wakes the RNWF02 from XDS.

Wake-Up via UART

- The RNWF02 UART interface uses the dedicated flow control pins to wake the device:
  - **WAKE_REQ (PB2):** The host DTE drives this pin **low** (falling edge) to request the RNWF02 to wake from PPS sleep.
  - **WAKE_RDY (PA4):** The RNWF02 drives this pin **low** once it is ready to receive AT commands.

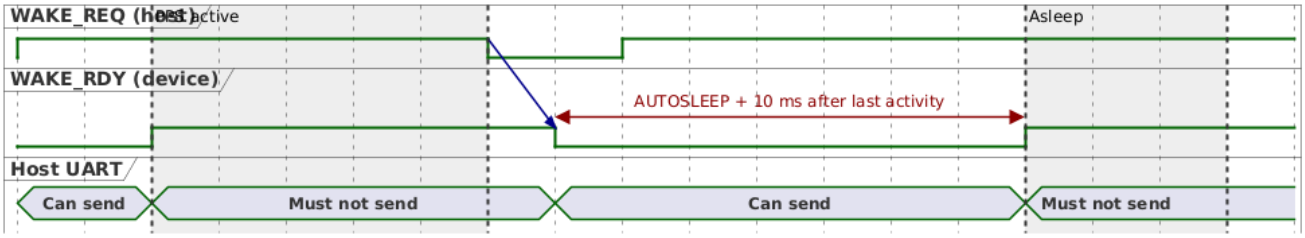
NOTE

- RTS/CTS hardware flow control is **not supported** under PPS operation on RNWF02
- Wake-up latency from PPS mode is approximately **4 ms**.

 ©2023-2026

Wake Sequence

1. Enable PPS with `AT+PPS=1`. After the OK response WAKE_RDY goes high, and the RNWF02 may sleep at any time.
2. While WAKE_RDY is high, do not send on the UART. The UART is powered down while the RNWF02 sleeps, so data sent then is lost; UART activity does not wake the RNWF02.
3. Drive WAKE_REQ low. The RNWF02 wakes and drives WAKE_RDY low. WAKE_REQ can be released once WAKE_RDY is low.
4. Send AT commands. The RNWF02 stays awake for at least the AUTOSLEEP time plus 10 ms after waking, and for as long as commands are being received or processed. AUTOSLEEP is configured with `AT+PPSC=2,<delayms>` (default 10 ms, range 10 to 3000 ms).
5. When the RNWF02 returns to sleep WAKE_RDY goes high. Repeat from step 3 to send more commands.



The RNWF02 also wakes on its own timers, for example at each listen interval. These wakes do not drive WAKE_RDY low, and the host must still use the wake sequence before sending. AECs can be sent whenever the RNWF02 is awake, so the host must always be ready to receive.

During a PPS pause (`AT+PPS=2`) the RNWF02 stays awake and WAKE_RDY stays low until the pause expires.

When PPS ends, because of `AT+PPS=0` or the PPS duration expiring, WAKE_RDY goes low and stays low. `AT+PPS=0` must itself be sent using the wake sequence.

PPS Duration

If the host does not have the dedicated pins WAKE_REQ (PB2) and WAKE_RDY (PA4) connected, PPS can be limited to a fixed duration so that the RNWF02 returns to normal operation without being woken.

The duration, in seconds, is configured with `AT+PPSC=3,<duration>` and must be set before PPS is enabled with `AT+PPS=1`. When the duration expires, PPS is disabled, the RNWF02 remains awake, and the `+PPS` AEC is sent with the status PPS Timeout. A duration of 0 (the default) means PPS remains active until disabled.

Configuration and Usage

See example Configuring and enabling PPS

→	`AT+WIFIC=21,30`	Configure Listen Interval to 30 beacon periods (around 3 seconds)
←	`OK`	
→	`AT+PPSC=1,1`	Enable Secondary Oscillator (optional)
←	`OK`	
→	`AT+PPSC=2,100`	Set AUTOSLEEP Timeout to 100ms
←	`OK`	
→	`AT+PPSC=3,60`	Set PPS to be enabled for 60 seconds
←	`OK`	
→	`AT+PPS=1`	Enable PPS Mode
←	`OK`	
→	`AT+PPS=2,5`	Pause PPS temporarily for 5 seconds (timeout extends if sent repeatedly)
←	`OK`	
→	`AT+PPS=0`	Terminate PPS Operation
←	`OK`	

NOTE

- RNWF02 PPS operation requires using the UART wake-up protocol via WAKE_REQ and WAKE_RDY pins. No conventional RTS/CTS hardware flow control is available during PPS.
- To send an AT command, the host DTE must drive WAKE_REQ low to wake the device, and wait for WAKE_RDY to go low, see Wake Sequence.
- When PPS is active, the host DTE cannot send AT commands unless the wake-up protocol is actively used. In the example above, `AT+PPS=2,5` and `AT+PPS=0` are sent after PPS is enabled, so the host must wake the device before each of them.

 ©2023-2026

EXtreme Deep Sleep (XDS)

XDS mode puts the RNWF02 into the absolute lowest power state, with all but the bare minimum clocks and peripherals powered off. Use this mode when long idle periods are expected and Wi-Fi connectivity/state retention is not required.

XDS characteristics

- The system loses all context and network state (no Wi-Fi connection or open sockets preserved).
- The only wake-up methods available are hardware signals:
  - Pull **MCLR** pin low (hardware reset)
  - Drive **WAKE_REQ** (PB2, INT0) low

NOTE	Wake-up triggers a **full system reboot**.

XDS Operation details

To enter XDS mode, send the following command:

```
AT+PPS=10
```

The OK response is sent first; the RNWF02 then enters XDS when it is next idle.

Summary

PPS (Platform Power Save)

Balances power saving with connectivity. Maintains Wi-Fi and protocol state. Supports low-latency wake-up via WAKE_REQ/WAKE_RDY pins. Recommended to enable SOSC for precise timing.

XDS (EXtreme Deep Sleep)

Maximises power saving by powering down all but the bare minimum device functions. Requires hardware wake-up and full reboot on wake. Used for extended idle periods with no connectivity requirements.

Command Reference:

+PPSC

Description

This command is used to read or set the PPS configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+PPSC	Query all configuration elements	GGGG
AT+PPSC=<ID>	Query a single element	GGGG
AT+PPSC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+PPSC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<SEC_OSC_ENABLED>	Bool	Use secondary oscillator 0 Off. 1 On.	GGGG
2	<AUTO_SLEEP_TIMEOUT>	Unsigned Integer	Timeout between commands before entering sleep measured in ms Valid range is 10 to 3000	GGGG
3	<DURATION>	Unsigned Integer	Time to maintain PPS mode in seconds (0 for infinite) Valid range is 0 to 86400	GGGG

+PPS

Description

This command allows the firmware to change the state of platform power save.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+PPS=<PPS_STATE>	Set PPS state PPS_STATE must not be 2	GGGG
AT+PPS=<PPS_STATE>,<PAUSE_TIME>	Pause PPS for a period PPS_STATE must be 2	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<PPS_STATE>	Integer	PPS State 0 Disable. 1 PPS mode. 2 PPS pause sleep. 10 XDS mode.
<PAUSE_TIME>	Integer	PPS pause time in sec Valid range is 1 to 3600

 ©2023-2026

AEC Reference:

+PPS

Description

Indication of PPS event.

AEC Syntax

AEC	Description
+PPS:<PPS_EVENT>	PPS Event

AEC Element Syntax

Element Name	Type	Description
<PPS_EVENT>	Integer	PPS status code One of the PPS status codes PPS_TIMEOUT (PPS duration expired) or PPS_PAUSE_EXPIRED (PPS pause ended), see Status Response Codes.

Examples:

Example Command Sequence 91: Configuring and enabling PPS

→	`AT+WIFIC=21,30`	Configure Listen Interval to 30 beacon periods (around 3 seconds)
←	`OK`	
→	`AT+PPSC=1,1`	Enable Secondary Oscillator (optional)
←	`OK`	
→	`AT+PPSC=2,100`	Set AUTOSLEEP Timeout to 100ms
←	`OK`	
→	`AT+PPSC=3,60`	Set PPS to be enabled for 60 seconds
←	`OK`	
→	`AT+PPS=1`	Enable PPS Mode
←	`OK`	
→	`AT+PPS=2,5`	Pause PPS temporarily for 5 seconds (timeout extends if sent repeatedly)
←	`OK`	
→	`AT+PPS=0`	Terminate PPS Operation
←	`OK`	

 ©2023-2026

SYSLOG (Module ID = 32)

Introduction

When enabled with ENABLED, the SYSLOG module reports internal log events as +SYSLOG AECs.

Each event carries a timestamp in milliseconds, the ID of the module which raised it, a severity and a message of up to 127 characters. All events are reported, there is no filtering by module or severity.

Logging is disabled by default and the setting is not stored in the configuration archive. Currently only the TLS module reports events.

Command Reference:

+SYSLOGC

Description

This command is used to read or set the SYSLOG configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+SYSLOGC	Query all configuration elements	GGGG
AT+SYSLOGC=<ID>	Query a single element	GGGG
AT+SYSLOGC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+SYSLOGC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
1	<ENABLED>	Bool	DCE's SYSLOG 0 Disabled. 1 Enabled.	GGGG

AEC Reference:

+SYSLOG

Description

SYSLOG Event.

AEC Syntax

AEC	Description
+SYSLOG:<TIMESTAMP>,<MODULE_ID>,<EVENT_SEVERITY>,<MESSAGE>	Log message

AEC Element Syntax

Element Name	Type	Description
<TIMESTAMP>	Integer	System time in milliseconds when the message was logged Unsigned 32-bit value
<MODULE_ID>	Integer	ID of the module that logged the message Unsigned 8-bit value Module IDs are listed in the headings of the Status Response Codes tables.
<EVENT_SEVERITY>	Integer	Event severity 0 Emergency. 1 Alert. 2 Critical. 3 Error. 4 Warning. 5 Notice. 6 Info. 7 Debug.
<MESSAGE>	String	Log message

 ©2023-2026

ARB (Module ID = 33)

Introduction

The RNWF02 has an anti-rollback counter (ARB) which can be used to prevent vulnerable firmware from running on the device.

The value in the ARB is between 0 and 127 and can be increased but never decreased. If a firmware image has a security level which is lower than the value in the ARB, then that image will not boot.

Use the +ARB command to get the current value in the device's ARB and the current firmware's security level. If the device's ARB is lower than the firmware's security level, then consider increasing the value in the device's ARB to improve the security of the device.

For information about the security benefits of increasing the device's ARB, refer to RNWF02 device firmware release notes at microchip.com.

To increase the value in the device's ARB, use the +ARB command.

CAUTION	If the device has an image present in the alternate NVM partition, be aware that increasing the device's ARB may invalidate that image.

Command Reference:

+ARB

Description

This command is used to get or increase the DCE's ARB value, and to get the current firmware's security level.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+ARB	Get the DCE's ARB value and the current firmware's security level	GGGG
AT+ARB=<VALUE>	Increase the DCE's ARB value	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<VALUE>	Integer	The value to which to increase the DCE's ARB Valid range is 1 to 127

Response Syntax

Response	Description
+ARB:<VALUE>,<FW_SECURITY>	The DCE's ARB value and the current firmware's security level

Response Element Syntax

Element Name	Type	Description
<VALUE>	Integer	DCE's ARB value
<FW_SECURITY>	Integer	Current firmware's security level

 ©2023-2026

Examples:

Example Command Sequence 92: Example of using +ARB

→	`AT+ARB`	Get the DCE's ARB value and the current firmware's security level
←	`+ARB:0,1`	DCE's ARB value is 0; current firmware's security level is 1
←	`OK`	
→	`AT+ARB=1`	Increase the DCE's ARB value to 1
←	`+ARB:1,1`	DCE's ARB value is 1; current firmware's security level is 1
←	`OK`	
→	`AT+ARB=1`	Increase the DCE's ARB value to 1
←	`+ARB:1,1`	DCE's ARB value is 1; current firmware's security level is 1
←	`ERROR:33.0,"The Value Provided Would Not Increase The ARB"`	
→	`AT+ARB=2`	Increase the DCE's ARB value to 2
←	`+ARB:1,1`	DCE's ARB value is 1; current firmware's security level is 1
←	`ERROR:33.1,"The Value Provided Would Invalidate The Current Image"`	

HTTP (Module ID = 34)

Introduction

The RNWF02 supports comprehensive HTTP client capabilities via AT commands, enabling easy integration with web servers for a variety of applications such as firmware updates, data retrieval, and cloud communication.

Key Features

HTTP Configuration Management

Configure HTTP connection parameters including the server address (hostname or IP), port number, request path, TLS settings, timeouts, and more. These settings ensure flexible connection setups to suit diverse network environments and security needs.

HTTP GET Requests

Perform HTTP GET operations to request resources from a web server. You can specify the resource path, enabling the retrieval of files, JSON data, or other web content.

Direct File Downloads to Flash Filesystem

Download HTTP response data directly into the device's flash filesystem. By configuring a local filename and file type, the downloaded content is automatically stored in the flash filesystem without requiring manual data reading. This is particularly useful for downloading certificates, configuration files, firmware images, or other persistent data. The filesystem supports multiple file types including USER files, certificates (CERT), private keys (PRIKEY), Diffie-Hellman parameters (DHPARAM), and configuration files (CFG).

Data Reading

Efficiently read HTTP response data in chunks of defined length. This allows controlled data transfer suitable for devices with limited memory. Manual data reading can be performed using +HTTPRD when not downloading directly to the filesystem.

Response Handling

Receive detailed response information such as HTTP status codes, response headers, and body content, including full notification when the response body is completely received.

Data Integrity Verification

Optional hash calculation (SHA, SHA256, SHA224, SHA512, SHA384) during download to verify data integrity. The hash is computed automatically and reported in the +HTTPBODYEND asynchronous event.

 ©2023-2026

Security Support

Integrated TLS configuration allows HTTPS connections ensuring data privacy and integrity.

How It Works

Manual Data Retrieval

See examples HTTPS download and HTTPS download from TCP socket.

- Configure the HTTP client using +HTTPC to set the hostname, port, path, and other parameters.
- Optionally configure TLS with +TLSC for encrypted HTTPS connections.
- Initiate an HTTP GET request via +HTTPGET.
- Monitor response status and headers through AECs (+HTTPRSPCODE, +HTTPRSPFIELD).
- Read the HTTP response body in manageable blocks using +HTTPRD.
- Close the connection cleanly with +HTTPCL after data transfer completes.

Direct Filesystem Download

See example File download to local filesystem.

- Configure the HTTP client using +HTTPC to set the hostname, port, path, and other parameters.
- Set the local filename using +HTTPC ID 20 (FILENAME parameter).
- Set the local file type using +HTTPC ID 21 (FILETYPE parameter):
  - 0 = USER (user files)
  - 1 = CERT (certificates)
  - 2 = PRIKEY (private keys)
  - 3 = DHPARAM (Diffie-Hellman parameters)
  - 20 = CFG (configuration files)
- Optionally configure hash verification using +HTTPC ID 22 (CHECK_MODE parameter):
  - 0 = NONE (no hash verification)
  - 10 = SHA
  - 11 = SHA256
  - 12 = SHA224
  - 13 = SHA512
  - 14 = SHA384
- Optionally configure TLS with +TLSC for encrypted HTTPS connections.
- Initiate an HTTP GET request via +HTTPGET.
- Monitor response status and headers through AECs (+HTTPRSPCODE, +HTTPRSPFIELD).
- The downloaded data is automatically written to the flash filesystem.
- When complete, +HTTPBODYEND is sent with total bytes downloaded and optional hash value.
- A filesystem update notification +FSUP confirms the file has been stored.

- The connection closes automatically (non-persistent connection mode).

NOTE	When a local filename is configured, the HTTP client operates in non-persistent connection mode and automatically writes received data to the filesystem. Manual data reading via +HTTPRD is not available in this mode.

Advanced Features

Using an Existing Socket Connection

The HTTP client can utilize an existing TCP socket connection instead of creating its own. This is useful when you need more control over the connection setup, such as pre-configuring socket options or managing connection pooling.

- Create and connect a TCP socket using socket commands (+SOCKO, +SOCKBR).
- Optionally upgrade the socket to TLS using +SOCKTLS.
- Assign the socket ID to the HTTP configuration using +HTTPC ID 10 (SOCKET_ID parameter).
- Perform HTTP GET operations as normal using +HTTPGET.

NOTE	When using an existing socket, do not configure the HOST (ID 2) or PORT (ID 3) parameters in the HTTP configuration.

Path Override with Query Parameters

When issuing an HTTP GET request, you can specify a complete path including query parameters and fragments as the second parameter to +HTTPGET. This overrides the PATH (ID 4) and FILE (ID 5) configuration parameters for that specific request.

See example HTTPGET with path and query/fragments.

HTTP Authentication

HTTP basic authentication can be implemented by providing additional request headers using +HTTPC ID 8 (REQ_HEADERS parameter). The authorization header should contain the Base64-encoded credentials.

See example HTTP basic authentication using additional request headers.

Reading Configuration Values

The read-only configuration parameter **CONTENT_LENGTH** reports the Content-Length value from the HTTP response headers after they are received.

 ©2023-2026

Error Handling

The HTTP client reports errors via two mechanisms:

- **HTTP status codes** - The server's response status is reported via +HTTPRSPCODE. Non-2xx status codes (e.g., 401 Unauthorized, 404 Not Found, 500 Internal Server Error) indicate the server rejected the request.
- **Transport errors** - Connection-level errors are reported via +HTTPERR. These indicate failures before or during HTTP communication.

Common transport errors:

- **DNS Failed** - The server hostname could not be resolved
- **TLS Failed** - TLS handshake failed (wrong CA certificate, SNI mismatch, expired certificate)
- **Connect Failed** - TCP connection to server failed
- **Timeout** - No response received within the configured timeout period
- **Receive Failed** - Error receiving data from server
- **Send Failed** - Error sending request to server

Command Reference:

+HTTPC

Description

This command is used to read or set the HTTP configuration.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+HTTPC	Query configuration list	GGGG
AT+HTTPC=<CONF>	Query all configuration elements	GGGG
AT+HTTPC=<CONF>,<ID>	Query a single element	GGGG
AT+HTTPC=<CONF>,<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<CONF>	Integer	Configuration number Value is 1
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+HTTPC:<MIN_IDX>,<MAX_IDX>	Valid index range
+HTTPC:<ID>,<VAL>	Read response

 ©2023-2026

Response Element Syntax

Element Name	Type	Description
<MIN_IDX>	Integer	Lowest valid index
<MAX_IDX>	Integer	Highest valid index
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

Configuration Parameters

ID	Name	Type	Description	Sec
0	<PRESET>	Integer	Configuration preset 0 Default.	GGGG
2	<HOST>	String IPv4 Address IPv6 Address	Host domain name or IP address Maximum length of string is 255 Format of IPv4 address is 'a.b.c.d' Format of IPv6 address is 'a:b:c:d::e:f'	GGGG
3	<PORT>	Unsigned Integer	Server port (e.g. 80 for HTTP, 443 for HTTPS); must be set	GGGG
4	<PATH>	String	Path to remote file Maximum length of string is 1024	GGGG
5	<FILE>	String	Remote filename Maximum length of string is 1024	GGGG
6	<TIMEOUT>	Unsigned Integer	Timeout in seconds Positive unsigned 8-bit value	GGGG
7	<TLS_CONF>	Integer	TLS configuration index (see +TLSC). 0 disables TLS Valid range is 0 to 4	GGGG
8	<REQ_HEADERS>	String	Additional request headers Maximum length of string is 1024	GGGG
10	<SOCKET_ID>	Unsigned Integer	Socket ID	GGGG
20	<FILENAME>	String	Local filename Maximum length of string is 32	GGGG
21	<FILETYPE>	Unsigned Integer	Local file type 0 User. 1 Certificate. 2 Private Key. 3 Diffie-Hellman parameters. 20 Configuration File.	GGGG

ID	Name	Type	Description	Sec
22	<CHECK_MODE>	Unsigned Integer	Checking mode 0 None. 10 SHA. 11 SHA256. 12 SHA224. 13 SHA512. 14 SHA384.	GGGG
40	<CONTENT_LENGTH>	Unsigned Integer (Read Only)	Content length	GGGG
100	<ASYNC_NEXT_OFFSET>	Unsigned Integer	Async next data offset	GGGG
101	<ASYNC_WIN_SZ>	Unsigned Integer	Async window size	GGGG
102	<ASYNC_MAX_FRM_SZ>	Unsigned Integer	Async maximum frame size	GGGG

 ©2023-2026

+HTTPGET

Description

This command performs an HTTP GET operation.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+HTTPGET=<CONF>	Perform an HTTP GET operation	GGGG
AT+HTTPGET=<CONF>,<PATH>	Perform an HTTP GET operation for a path	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<CONF>	Integer	Configuration number Value is 1
<PATH>	String	Resource path

+HTTPRD

Description

This command performs an HTTP data read operation.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+HTTPRD=<CONF>,<LENGTH>	Read HTTP data	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<CONF>	Integer	Configuration number Value is 1
<LENGTH>	Integer	Number of bytes

Response Syntax

Response	Description
+HTTPRD:<CONF>,<DATA_LEN>,<DATA_OFFSET>,<DATA>	HTTP body data read

Response Element Syntax

Element Name	Type	Description
<CONF>	Integer	HTTP Configuration number Value is 1
<DATA_LEN>	Integer	Length of data
<DATA_OFFSET>	Integer	Offset of data bytes
<DATA>	Byte Array	HTTP data

 ©2023-2026

+HTTPCL

Description

This command performs an HTTP close operation.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+HTTPCL=<CONF>	Close HTTP session	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<CONF>	Integer	Configuration number Value is 1

AEC Reference:

+HTTPRSPCODE

Description

HTTP response code.

AEC Syntax

AEC	Description
+HTTPRSPCODE:<CONF>,<RSP_PROTOCOL>,<RSP_CODE>	HTTP response code

AEC Element Syntax

Element Name	Type	Description
<CONF>	Integer	HTTP Configuration number Value is 1
<RSP_PROTOCOL>	Integer	HTTP response protocol version Value is 1.1
<RSP_CODE>	Integer	HTTP response code Valid range is 100 to 599

 ©2023-2026

+HTTPRSPFIELD

Description

HTTP response field.

AEC Syntax

AEC	Description
+HTTPRSPFIELD:<CONF>,<RSP_FIELD_NAME>,<RSP_FIELD_VALUE>	HTTP response field

AEC Element Syntax

Element Name	Type	Description
<CONF>	Integer	HTTP Configuration number Value is 1
<RSP_FIELD_NAME>	String	HTTP response field name
<RSP_FIELD_VALUE>	String	HTTP response field value

+HTTPBODYRX

Description

HTTP body.

AEC Syntax

AEC	Description
+HTTPBODYRX:<CONF>,<PEND_DATA>	HTTP body data notify
+HTTPBODYRX:<CONF>,<DATA_LEN>,<DATA_OFFSET>,<DATA>	HTTP body data received

AEC Element Syntax

Element Name	Type	Description
<CONF>	Integer	HTTP Configuration number Value is 1
<PEND_DATA>	Integer	Number of pending bytes
<DATA_LEN>	Integer	Length of data
<DATA_OFFSET>	Integer	Offset of data bytes
<DATA>	Byte Array	HTTP data

 ©2023-2026

+HTTPBODYEND

Description

HTTP body end.

AEC Syntax

AEC	Description
+HTTPBODYEND:<CONF>,<DATA_LEN>	HTTP body data end
+HTTPBODYEND:<CONF>,<DATA_LEN>,<HASH_TYPE>,<HASH>	HTTP body data end

AEC Element Syntax

Element Name	Type	Description
<CONF>	Integer	HTTP Configuration number Value is 1
<DATA_LEN>	Integer	Length of data
<HASH_TYPE>	Integer	Hash algorithm 10 SHA. 11 SHA256. 12 SHA224. 13 SHA512. 14 SHA384.
<HASH>	Byte Array	Hash

+HTTPCL

Description

HTTP close.

AEC Syntax

AEC	Description
+HTTPCL:<CONF>	HTTP close

AEC Element Syntax

Element Name	Type	Description
<CONF>	Integer	HTTP Configuration number Value is 1

 ©2023-2026

+HTTPERR

Description

HTTP error.

AEC Syntax

AEC	Description
+HTTPERR:<CONF>,<ERROR_CODE>	HTTP error

AEC Element Syntax

Element Name	Type	Description
<CONF>	Integer	HTTP Configuration number Value is 1
<ERROR_CODE>	Integer	Error code See Status Response Codes

Examples:

Example Command Sequence 93: HTTPS download

←	`+TIME:3969426905`	Wait for time for correct TLS operation
→	`AT+TLSC=1,1,"ServerCACert"`	Set the server CA certificate
←	`OK`	
→	`AT+TLSC=1,6,"www.example.com"`	Set the server domain
←	`OK`	
→	`AT+HTTPC=1,2,"www.example.com"`	Set the server hostname
←	`OK`	
→	`AT+HTTPC=1,3,443`	Set the HTTPS port number
←	`OK`	
→	`AT+HTTPC=1,5,"/examplefile.bin"`	Set the file path
←	`OK`	
→	`AT+HTTPC=1,6,10`	Set 10 second timeout
←	`OK`	
→	`AT+HTTPC=1,7,1`	Use TLSC index 1
←	`OK`	
→	`AT+HTTPGET=1`	Get resource configuration 1
←	`OK`	
←	`+HTTPRSPCODE:1,1.1,200`	Server responds 200 OK
←	`+HTTPRSPFIELD:1,"Date","Wed, 5 Nov 2025 11:23:45 GMT"`	
←	`+HTTPRSPFIELD:1,"Last-Modified","Sun, 12 Oct 2025 21:15:27 GMT"`	
←	`+HTTPRSPFIELD:1,"Accept-Ranges","bytes"`	
←	`+HTTPRSPFIELD:1,"Content-Type","binary/octet-stream"`	
←	`+HTTPRSPFIELD:1,"Content-Length","2563"`	
←	`+HTTPBODYRX:1,1460`	1460 bytes received
→	`AT+HTTPRD=1,1460`	Read 1460 bytes
←	`+HTTPRD:1,1460,0,[FFFFFFFF…74780000]`	
←	`OK`	
←	`+HTTPBODYRX:1,1103`	1103 bytes received
←	`+HTTPBODYEND:1,2563`	All body data received
→	`AT+HTTPRD=1,1103`	Read 1103 bytes
←	`+HTTPRD:1,1103,1460,[65205072…1102D0A1]`	
←	`OK`	
←	`+HTTPCL:1`	Remote server closed connection

 ©2023-2026

Example Command Sequence 94: HTTPS download from TCP socket

←	`+TIME:3969426905`	Wait for time for correct TLS operation
→	`AT+SOCKO=2`	Open TCP socket
←	`+SOCKO:18870`	
←	`OK`	
→	`AT+TLSC=1,1,"ServerCACert"`	Set the server CA certificate
←	`OK`	
→	`AT+TLSC=1,6,"www.example.com"`	Set the server domain
←	`OK`	
→	`AT+HTTPC=1,5,"/examplefile.bin"`	Set the file path
←	`OK`	
→	`AT+HTTPC=1,6,10`	Set 10 second timeout
←	`OK`	
→	`AT+HTTPC=1,10,18870`	Assign socket ID to HTTP configuration
←	`OK`	
→	`AT+SOCKTLS=18870,1`	Upgrade socket to TLS
←	`OK`	
→	`AT+SOCKBR=18870,"www.example.com",443`	Bind socket to server
←	`OK`	
←	`+SOCKIND:18870,"1.2.3.4",61955,"5.6.7.8",443`	Socket connected
←	`+SOCKTLS:18870`	Socket upgraded to TLS
→	`AT+HTTPGET=1`	Get resource configuration 1
←	`OK`	
←	`+HTTPRSPCODE:1,1.1,200`	Server responds 200 OK
←	`+HTTPRSPFIELD:1,"Date","Wed, 5 Nov 2025 11:23:45 GMT"`	
←	`+HTTPRSPFIELD:1,"Last-Modified","Sun, 12 Oct 2025 21:15:27 GMT"`	
←	`+HTTPRSPFIELD:1,"Accept-Ranges","bytes"`	
←	`+HTTPRSPFIELD:1,"Content-Type","binary/octet-stream"`	
←	`+HTTPRSPFIELD:1,"Content-Length","2563"`	
←	`+HTTPBODYRX:1,1460`	1460 bytes received
→	`AT+HTTPRD=1,1460`	Read 1460 bytes
←	`+HTTPRD:1,1460,0,[FFFFFFFF…74780000]`	
←	`OK`	
←	`+HTTPBODYRX:1,1103`	1103 bytes received
←	`+HTTPBODYEND:1,2563`	All body data received
→	`AT+HTTPRD=1,1103`	Read 1103 bytes
←	`+HTTPRD:1,1103,1460,[65205072…1102D0A1]`	
←	`OK`	
←	`+HTTPCL:1`	Remote server closed connection

Example Command Sequence 95: File download to local filesystem

→	`AT+HTTPC=1,20,"testname"`	Set local filename
←	`OK`	
→	`AT+HTTPC=1,21,0`	Set local file type to USER
←	`OK`	
→	`AT+HTTPGET=1`	Get resource configuration 1
←	`OK`	
←	`+HTTPRSPCODE:1,1.1,200`	Server responds 200 OK
←	`+HTTPRSPFIELD:1,"Date","Wed, 5 Nov 2025 11:23:45 GMT"`	
←	`+HTTPRSPFIELD:1,"Last-Modified","Sun, 12 Oct 2025 21:15:27 GMT"`	
←	`+HTTPRSPFIELD:1,"Accept-Ranges","bytes"`	
←	`+HTTPRSPFIELD:1,"Content-Type","binary/octet-stream"`	
←	`+HTTPRSPFIELD:1,"Content-Length","1024"`	
←	`+HTTPBODYEND:1,1024`	All body data received
←	`+FSUP:1,0,"testname",1024`	Filesystem updated
←	`+HTTPCL:1`	Remote server closed connection
→	`AT+FS=2,0`	Query for USER files
←	`+FS:2,0,"testname"`	File now present in filesystem
←	`OK`	

Example Command Sequence 96: HTTPGET with path and query/fragments

→	`AT+HTTPGET=1,"/anypath/anyfile?att1=val1&att2=val2`	afragment"
←	`OK`	
←	`+HTTPRSPCODE:1,1.1,200`	

Example Command Sequence 97: HTTP basic authentication using additional request headers

→	`AT+HTTPC=1,8,"Authorization: Basic dXNlcm5hbWU6cGFzc3dk"`	
←	`OK`	

 ©2023-2026

WBCN (Module ID = 35)

Introduction

The WBCN module transmits Wi-Fi beacon frames. Together with periodic passive scanning, it supports application-defined beacon advertising and discovery.

Information is carried in the **beacon SSID**: the payload is an application-defined string of at most 32 characters. The format of the payload is left to the application. A common leading string allows the SSID prefix filter of the WSCN module to select matching beacons from those of ordinary access points.

Additional application information can be carried alongside the SSID in one or more vendor-specific IEs. Each IE contains up to 252 bytes of vendor-specific data.

Configure vendor-specific IEs with the AT+WIFIC command.

Beacons are transmitted with the +WBCN command described below; beacons from other devices are detected using the +WSCNP command.

Configuring the beacon

Set the beacon parameters using the +WBCNC command.

Five parameters control the beacon. Three of them decide what is transmitted:

- The channel to transmit the beacon on: CHANNEL. A non-zero channel selects that one channel; zero takes the channel set from CHANMASK24.
- The 2.4 GHz channel bit mask used when CHANNEL is zero: CHANMASK24
- The SSID carried in the beacon, which holds the payload: SSID

The other two decide how often the beacon is repeated, and how many times:

- The time between beacons, in milliseconds: INTERVAL
- The number of beacons to transmit, or zero to beacon until stopped: NUM_BEACONS

NOTE

The selected channel or channel mask must lie within the active regulatory domain's channel mask, otherwise the value is rejected. CHANNEL zero with an empty CHANMASK24 is refused when beaconing starts with WBCN_NO_CHANNELS. INTERVAL can be configured from 100 ms to 600000 ms. When beaconing starts, INTERVAL must be strictly greater than 20 ms multiplied by the number of selected channels; otherwise the start is refused with WBCN_INTERVAL_TOO_SHORT.

The SSID may be at most 32 characters. A payload longer than that is rejected.

Transmitting beacons

Once the beacon has been configured, start beaconing using the +WBCN command, and stop it with the same command.

Beaconing takes the form of a run rather than a single transmission: the configured beacon is transmitted on the configured channel set once every INTERVAL milliseconds, until NUM_BEACONS transmissions have completed, or indefinitely if NUM_BEACONS is zero. One transmission covers every selected channel, so NUM_SENT counts channel sets, not individual beacon frames.

A run which cannot continue ends with +WBCNERR.

A run always ends with one of two AECs:

- +WBCNDONE - the run ended as asked, either because NUM_BEACONS channel-set transmissions completed, or because it was stopped.
- +WBCNERR - the run was abandoned before completion.

Stopping a run is a request rather than an immediate halt: success is returned straight away, the run ends at the next point at which nothing is outstanding, and +WBCNDONE follows. Stopping when no run is in progress succeeds and does nothing.

The beacon configuration is locked while a run is in progress: a write with +WBCNC is rejected with STORE_UPDATE_BLOCKED, and a second attempt to start a run with WBCN_IN_PROGRESS. Reads are permitted throughout. Changing the payload therefore means ending the current run first: stop it, wait for +WBCNDONE, write the new SSID, then start a new run. The module goes idle before that AEC is sent, so the configuration is always writable by the time the host sees it.

Beaconing can overlap periodic passive scanning with +WSCNP, which allows a device to transmit and scan at the same time.

NOTE	When beaconing and periodic passive scanning overlap, scanning takes precedence. Beaconing resumes after the scan sweep completes.

NOTE	Beaconing is unavailable while soft-AP mode is running; a start is refused with WBCN_AP_MODE_ACTIVE.

Detecting beacons

Beacons transmitted by other devices are detected using periodic passive scanning.

Set the periodic scan channel (CHANNEL) to match the beacon channel, optionally adjust the passive scan slot time (PASV_SLOT_TIME) and interval (INTERVAL), then start periodic passive scanning using the +WSCNP command.

 ©2023-2026

Examples

Asset Tracking

Asset tracking is one application of beacon advertising and periodic passive scanning. A target asset advertises its identifier and listens for commands; a tracker asset detects target assets and delivers commands. The examples below show the exchange one stage at a time.

A target asset advertises its location by beaconing its asset identifier:

→	`AT+WBCNC=1,0`	Take the beacon channels from the channel mask
←	`OK`	
→	`AT+WBCNC=11,0x0421`	Beacon on channels 1, 6 and 11
←	`OK`	
→	`AT+WBCNC=2,"ASSET35"`	Set the beacon SSID to the asset identifier
←	`OK`	
→	`AT+WBCNC=7,5000`	Send one three-channel transmission every 5 s
←	`OK`	
→	`AT+WBCNC=8,0`	Keep advertising until told to stop, rather than stopping after a set count
←	`OK`	
→	`AT+WBCN=1`	Start beaconing
←	`OK`	
	`..`	
→	`AT+WBCN=0`	Request that beaconing stops
←	`OK`	The request is accepted; the run is not over yet
←	`+WBCNDONE:37`	The run is over: 37 beacons transmitted per channel

 ©2023-2026

A tracker asset runs a periodic passive scan and detects the target asset:

→	`AT+WSCNPC=1,6`	Set the periodic scan channel to 6
←	`OK`	
→	`AT+WSCNPC=3,500`	Listen for 500 ms in each scan slot
←	`OK`	
→	`AT+WSCNPC=4,2`	Two slots per sweep, so each sweep listens on the channel for a total of 1000 ms
←	`OK`	
→	`AT+WSCNPC=12,5000`	Start a sweep every 5000 ms
←	`OK`	
→	`AT+WSCNPC=6,-45`	Discard anything weaker than -45 dBm
←	`OK`	
→	`AT+WSCNPC=10,"ASSET*"`	Report only matching SSIDs, so beacons from ordinary access points are dropped
←	`OK`	
→	`AT+WSCNP=1`	Enable periodic scanning, the first sweep starts immediately. All of the scan configuration must be in place by now: it is locked while scanning is enabled
←	`OK`	
←	`+WSCNIND:-30,0,6,"F8:F0:05:12:34:56","ASSET35"`	A matching beacon is detected on channel 6
←	`+WSCNDONE:1`	Sweep completed, 1 result found
	`..`	
←	`+WSCNDONE:0`	A later sweep found no matching beacons
	`..`	
→	`AT+WSCNP=0`	Stop periodic scanning
←	`OK`	
←	`+WSCNPDONE:12`	The run is over: 12 sweeps were carried out

A tracker asset delivers an encoded command to a target asset by beaconing the command payload:

→	`AT+WBCNC=1,6`	Set the beacon channel to 6
←	`OK`	
→	`AT+WBCNC=2,"CMD3240138012398502495190247597"`	Set the beacon SSID to the encrypted command payload
←	`OK`	
→	`AT+WBCNC=7,1000`	Repeat the command every 1000 ms
←	`OK`	
→	`AT+WBCNC=8,0`	Repeat until told to stop, rather than a set number of times: the target only listens for part of each of its scan intervals, so a single transmission is easily missed
←	`OK`	
→	`AT+WBCN=1`	Start transmitting the beacons carrying the command
←	`OK`	
	`..`	
→	`AT+WBCN=0`	Stop repeating the command, the target's response having been received
←	`OK`	The request is accepted; the run is not over yet
←	`+WBCNDONE:12`	The run is over: 12 beacons transmitted

 ©2023-2026

The target asset runs a periodic passive scan and receives the command:

→	`AT+WSCNPC=1,6`	Set the periodic scan channel to 6
←	`OK`	
→	`AT+WSCNPC=3,500`	Listen for 500 ms in each scan slot
←	`OK`	
→	`AT+WSCNPC=4,2`	Two slots per sweep, so each sweep listens on the channel for a total of 1000 ms
←	`OK`	
→	`AT+WSCNPC=12,5000`	Start a sweep every 5000 ms
←	`OK`	
→	`AT+WSCNPC=6,-45`	Discard anything weaker than -45 dBm
←	`OK`	
→	`AT+WSCNPC=10,"CMD*"`	Report only SSIDs carrying command payloads
←	`OK`	
→	`AT+WSCNP=1`	Enable periodic scanning, the first sweep starts immediately. All of the scan configuration must be in place by now: it is locked while scanning is enabled
←	`OK`	
←	`+WSCNDONE:0`	A sweep which found no matching beacon
	`..`	
←	`+WSCNIND:-30,0,6,"F8:F0:05:AB:CD:EF","CMD3240138012398502495190247597"`	Matching SSID payload received
←	`+WSCNDONE:1`	Sweep completed, 1 result found
	`..`	
→	`AT+WSCNP=0`	Stop periodic scanning
←	`OK`	
←	`+WSCNPDONE:8`	The run is over: 8 sweeps were carried out

The two examples below then show each role's complete exchange, from the two sides of the same conversation.

The target asset advertises and listens at the same time, answers a command, and resumes advertising:

→	`AT+WBCNC=1,6`	Set the beacon channel to 6
←	`OK`	
→	`AT+WBCNC=2,"ASSET35"`	Set the beacon SSID to the asset identifier
←	`OK`	
→	`AT+WBCNC=7,1000`	Advertise once every 1000 ms
←	`OK`	
→	`AT+WBCNC=8,0`	Keep advertising until told to stop, rather than stopping after a set count
←	`OK`	
→	`AT+WSCNPC=1,6`	Listen for commands on the beacon channel
←	`OK`	
→	`AT+WSCNPC=3,500`	Listen for 500 ms in each scan slot
←	`OK`	
→	`AT+WSCNPC=4,2`	Two slots per sweep, so each sweep listens on the channel for a total of 1000 ms
←	`OK`	
→	`AT+WSCNPC=12,5000`	Start a sweep every 5000 ms
←	`OK`	
→	`AT+WSCNPC=6,-45`	Discard anything weaker than -45 dBm: a tracker asset that distant could not hear the response either
←	`OK`	
→	`AT+WSCNPC=10,"CMD*"`	Report only command payloads, so beacons carrying anything else are dropped
←	`OK`	
→	`AT+WBCN=1`	Start advertising. No connection is needed: the whole exchange runs in the disconnected state
←	`OK`	
→	`AT+WSCNP=1`	Start listening for commands, the first sweep starts immediately
←	`OK`	
←	`+WSCNDONE:0`	A sweep which overlapped no command beacon
	`...`	Advertise and listen until a command arrives. An advertisement which comes due while a sweep holds the radio waits and is sent once the sweep ends, rather than being skipped
←	`+WSCNIND:-30,0,6,"F8:F0:05:AB:CD:EF","CMD32401380123985024951902475 97"`	A command has arrived from the tracker asset
←	`+WSCNDONE:1`	Sweep completed, 1 result found
→	`AT+WBCN=0`	Stop advertising: the beacon configuration is locked while a run is in progress, so the response cannot be loaded until the advertising run has ended
←	`OK`	The request is accepted; the run is not over yet

 ©2023-2026

←	`+WBCNDONE:75`	The advertising run is over: all 75 beacons were confirmed transmitted, some delayed while a scan sweep held the radio
→	`AT+WBCNC=2,"RSP76200451188203944710293847 56"`	Set the beacon SSID to the encoded response payload
←	`OK`	
→	`AT+WBCNC=8,10`	Send the response 10 times: the tracker only listens for part of each of its scan intervals, so a single transmission is easily missed
←	`OK`	
→	`AT+WBCN=1`	Transmit the beacons carrying the response
←	`OK`	
←	`+WBCNDONE:10`	The response run is over: all 10 beacons were transmitted
→	`AT+WBCNC=2,"ASSET35"`	Restore the asset identifier
←	`OK`	
→	`AT+WBCNC=8,0`	Advertise continuously again
←	`OK`	
→	`AT+WBCN=1`	Resume advertising, the periodic scan having listened for the next command throughout
←	`OK`	

The tracker asset connects, discovers the target asset, commands it and collects the response:

→	`AT+WSTAC=1,"MyAP"`	Specify SSID of the Access Point to report findings over
←	`OK`	
→	`AT+WSTAC=2,3`	Set security type to WPA2-Personal
←	`OK`	
→	`AT+WSTAC=3,"MyAPPSK"`	Set WPA2 Personal passphrase for connecting to the Access Point
←	`OK`	
→	`AT+WSTA=1`	Connect
←	`OK`	
←	`+WSTALU:1,"00:01:02:03:04:05",6`	Link up: connected on channel 6 to an AP with BSSID 00:01:02:03:04:05
←	`+WSTAAIP:1,"192.168.0.20"`	IP address received via auto configuration (DHCP/SLAAC)
→	`AT+WSCNPC=1,6`	Set the periodic scan channel to 6
←	`OK`	
→	`AT+WSCNPC=3,500`	Listen for 500 ms in each scan slot
←	`OK`	
→	`AT+WSCNPC=4,2`	Two slots per sweep, so each sweep listens on the channel for a total of 1000 ms
←	`OK`	
→	`AT+WSCNPC=12,5000`	Start a sweep every 5000 ms
←	`OK`	
→	`AT+WSCNPC=6,-45`	Discard anything weaker than -45 dBm: a target asset that distant could not hear a command either
←	`OK`	
→	`AT+WSCNPC=10,"ASSET*"`	Add a filter for the advertisements of target assets
←	`OK`	
→	`AT+WSCNPC=10,"RSP*"`	Add a second filter for the responses they send back. Both filters must be in place before scanning starts, because the scan configuration is locked while it runs
←	`OK`	
→	`AT+WSCNP=1`	Enable periodic scanning, the first sweep starts immediately
←	`OK`	
←	`+WSCNDONE:0`	A sweep which found no target asset
	`…`	Sweep every 5000 ms until a target asset comes into range
←	`+WSCNIND:-30,0,6,"F8:F0:05:12:34:56","ASSET35"`	Target asset ASSET35 is in range
←	`+WSCNDONE:1`	Sweep completed, 1 result found
→	`AT+WBCNC=1,6`	Set the beacon channel to 6, the channel the target asset listens on
←	`OK`	
→	`AT+WBCNC=2,"CMD3240138012398502495190247597"`	Set the beacon SSID to the encrypted command payload
←	`OK`	

 ©2023-2026

→	`AT+WBCNC=7,1000`	Repeat the command every 1000 ms
←	`OK`	
→	`AT+WBCNC=8,0`	Repeat until told to stop, rather than a set number of times: the target only listens for part of each of its scan intervals, so a single transmission is easily missed
←	`OK`	
→	`AT+WBCN=1`	Start transmitting the command. Beaconing and periodic scanning run together: a beacon which comes due while a sweep holds the radio waits and is sent once the sweep ends
←	`OK`	
←	`+WSCNIND:-30,0,6,"F8:F0:05:12:34:56","ASSET35"`	A sweep which saw the advertisement again, but no response yet
←	`+WSCNDONE:1`	Sweep completed, 1 result found
	`...`	Repeat the command, and keep sweeping, until the target asset answers
←	`+WSCNIND:-30,0,6,"F8:F0:05:12:34:56","RSP76200451188203944710293847 56"`	The target asset has answered; it stops advertising its identifier while it sends the response
←	`+WSCNDONE:1`	Sweep completed, 1 result found
→	`AT+WBCN=0`	The response has been received, so stop repeating the command
←	`OK`	The request is accepted; the run is not over yet
←	`+WBCNDONE:15`	The run is over: all 15 beacons carrying the command were confirmed transmitted, some delayed while one of this asset's own scan sweeps held the radio
→	`AT+WSCNP=0`	Stop watching for target assets
←	`OK`	
←	`+WSCNPDONE:19,0`	The run is over: 19 sweeps were started and none were skipped

Command Reference:

+WBCNC

Description

This command is used to modify or query the behavior of the beacon function.

This command is a configuration command which supports setting and getting parameter values. The behaviour of configuration commands is described in general in the Configuration Commands section.

Security

Default security for the command is: `GGGG`

Command Syntax

Command	Description	Sec
AT+WBCNC	Query all configuration elements	GGGG
AT+WBCNC=<ID>	Query a single element	GGGG
AT+WBCNC=<ID>,<VAL>	Set a single element	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<ID>	Integer	Parameter ID number Unsigned 8-bit value
<VAL>	Integer String Byte Array	Parameter value

Response Syntax

Response	Description
+WBCNC:<ID>,<VAL>	Read response

Response Element Syntax

Element Name	Type	Description
<ID>	Integer	Parameter ID number
<VAL>	Any	Parameter value

 ©2023-2026

Configuration Parameters

<table>
<tr><th>ID</th><th>Name</th><th>Type</th><th>Description</th><th>Sec</th></tr>
<tr><td>0</td><td><PRESET></td><td>Integer</td><td>Configuration preset

0 Default.</td><td>GGGG</td></tr>
<tr><td>1</td><td><CHANNEL></td><td>Unsigned Integer</td><td>The channel to send the beacon on (or 0 to use CHANMASK24)

Valid range is 0 to 13</td><td>GGGG</td></tr>
<tr><td>2</td><td><SSID></td><td>String</td><td>The SSID advertised in the beacon
Maximum length of string is 32</td><td>GGGG</td></tr>
<tr><td>7</td><td><INTERVAL></td><td>Unsigned Integer</td><td>The interval between beacon transmissions, in milliseconds

Valid range is 100 to 600000</td><td>GGGG</td></tr>
<tr><td>8</td><td><NUM_BEACONS></td><td>Unsigned Integer</td><td>The number of beacons to send per channel; 0 to send continuously until a stop request

Unsigned 16-bit value</td><td>GGGG</td></tr>
<tr><td>11</td><td><CHANMASK24></td><td>Unsigned Integer</td><td>Beacon channel mask (2.4 GHz), used when CHANNEL is 0

<table>
<tr><td>15
-</td><td>14
-</td><td>13
CH14</td><td>12
CH13</td><td>11
CH12</td><td>10
CH11</td><td>9
CH10</td><td>8
CH9</td></tr>
<tr><td>7
CH8</td><td>6
CH7</td><td>5
CH6</td><td>4
CH5</td><td>3
CH4</td><td>2
CH3</td><td>1
CH2</td><td>0
CH1</td></tr>
</table>

<table>
<tr><th>Value</th><th>Label</th><th>Description</th></tr>
<tr><td>0x0001</td><td>CH1</td><td>Channel 1 - 2412 MHz.</td></tr>
<tr><td>0x0002</td><td>CH2</td><td>Channel 2 - 2417 MHz.</td></tr>
<tr><td>0x0004</td><td>CH3</td><td>Channel 3 - 2422 MHz.</td></tr>
<tr><td>0x0008</td><td>CH4</td><td>Channel 4 - 2427 MHz.</td></tr>
<tr><td>0x0010</td><td>CH5</td><td>Channel 5 - 2432 MHz.</td></tr>
<tr><td>0x0020</td><td>CH6</td><td>Channel 6 - 2437 MHz.</td></tr>
<tr><td>0x0040</td><td>CH7</td><td>Channel 7 - 2442 MHz.</td></tr>
<tr><td>0x0080</td><td>CH8</td><td>Channel 8 - 2447 MHz.</td></tr>
<tr><td>0x0100</td><td>CH9</td><td>Channel 9 - 2452 MHz.</td></tr>
<tr><td>0x0200</td><td>CH10</td><td>Channel 10 - 2457 MHz.</td></tr>
<tr><td>0x0400</td><td>CH11</td><td>Channel 11 - 2462 MHz.</td></tr>
<tr><td>0x0800</td><td>CH12</td><td>Channel 12 - 2467 MHz.</td></tr>
<tr><td>0x1000</td><td>CH13</td><td>Channel 13 - 2472 MHz.</td></tr>
<tr><td>0x2000</td><td>CH14</td><td>Channel 14 - 2485 MHz.</td></tr>
</table>
</td><td>GGGG</td></tr>
</table>

+WBCN

Description

This command is used to start or stop periodic beacon transmission. Exactly one of +WBCNDONE or +WBCNERR closes out every accepted start request. A stop request only requests the stop: its OK does not mark the end of the run, the +WBCNDONE which follows it does.

Security

Default security for the command is: GGGG

Command Syntax

Command	Description	Sec
AT+WBCN	Query state of the beacon function	GGGG
AT+WBCN=<STATE>	Set state of the beacon function	GGGG

Command Parameter Syntax

Parameter Name	Type	Description
<STATE>	Integer	State of the beacon function 0 Disable. 1 Enable.

Response Syntax

Response	Description
+WBCN:<STATE>	Query state

Response Element Syntax

Element Name	Type	Description
<STATE>	Integer	State of the beacon function 0 Disabled. 1 Enabled.

 ©2023-2026

AEC Reference:

+WBCNDONE

Description

Beacon transmission completed.

AEC Syntax

AEC	Description
+WBCNDONE:<NUM_SENT>	Beaconing complete

AEC Element Syntax

Element Name	Type	Description
<NUM_SENT>	Integer	Number of beacon transmitted. Each transmission covers every channel selected by CHANNEL/CHANMASK24, so this counts channel sets aired, not individual frames. A value below the configured NUM_BEACONS means the run was cut short by a stop request, or that a transmission could not be made

+WBCNERR

Description

Beacon run abandoned because the driver rejected a transmit request.

AEC Syntax

AEC	Description
+WBCNERR:<ERROR_CODE>	Beaconing abandoned

AEC Element Syntax

Element Name	Type	Description
<ERROR_CODE>	Integer	Error code See Status Response Codes

 ©2023-2026

Examples:

Example Command Sequence 104: Target asset: advertise its location

→	`AT+WBCNC=1,0`	Take the beacon channels from the channel mask
←	`OK`	
→	`AT+WBCNC=11,0x0421`	Beacon on channels 1, 6 and 11
←	`OK`	
→	`AT+WBCNC=2,"ASSET35"`	Set the beacon SSID to the asset identifier
←	`OK`	
→	`AT+WBCNC=7,5000`	Send one three-channel transmission every 5 s
←	`OK`	
→	`AT+WBCNC=8,0`	Keep advertising until told to stop, rather than stopping after a set count
←	`OK`	
→	`AT+WBCN=1`	Start beaconing
←	`OK`	
	`..`	
→	`AT+WBCN=0`	Request that beaconing stops
←	`OK`	The request is accepted; the run is not over yet
←	`+WBCNDONE:37`	The run is over: 37 beacons transmitted per channel

Example Command Sequence 105: Tracker asset: send a command to a target asset

→	`AT+WBCNC=1,6`	Set the beacon channel to 6
←	`OK`	
→	`AT+WBCNC=2,"CMD3240138012398502495190247597"`	Set the beacon SSID to the encrypted command payload
←	`OK`	
→	`AT+WBCNC=7,1000`	Repeat the command every 1000 ms
←	`OK`	
→	`AT+WBCNC=8,0`	Repeat until told to stop, rather than a set number of times: the target only listens for part of each of its scan intervals, so a single transmission is easily missed
←	`OK`	
→	`AT+WBCN=1`	Start transmitting the beacons carrying the command
←	`OK`	
	`..`	
→	`AT+WBCN=0`	Stop repeating the command, the target's response having been received
←	`OK`	The request is accepted; the run is not over yet
←	`+WBCNDONE:12`	The run is over: 12 beacons transmitted

Example Command Sequence 106: Target asset: a complete exchange with a tracker asset

→	`AT+WBCNC=1,6`	Set the beacon channel to 6
←	`OK`	
→	`AT+WBCNC=2,"ASSET35"`	Set the beacon SSID to the asset identifier

←	`OK`	
→	`AT+WBCNC=7,1000`	Advertise once every 1000 ms
←	`OK`	
→	`AT+WBCNC=8,0`	Keep advertising until told to stop, rather than stopping after a set count
←	`OK`	
→	`AT+WSCNPC=1,6`	Listen for commands on the beacon channel
←	`OK`	
→	`AT+WSCNPC=3,500`	Listen for 500 ms in each scan slot
←	`OK`	
→	`AT+WSCNPC=4,2`	Two slots per sweep, so each sweep listens on the channel for a total of 1000 ms
←	`OK`	
→	`AT+WSCNPC=12,5000`	Start a sweep every 5000 ms
←	`OK`	
→	`AT+WSCNPC=6,-45`	Discard anything weaker than -45 dBm: a tracker asset that distant could not hear the response either
←	`OK`	
→	`AT+WSCNPC=10,"CMD*"`	Report only command payloads, so beacons carrying anything else are dropped
←	`OK`	
→	`AT+WBCN=1`	Start advertising. No connection is needed: the whole exchange runs in the disconnected state
←	`OK`	
→	`AT+WSCNP=1`	Start listening for commands, the first sweep starts immediately
←	`OK`	
←	`+WSCNDONE:0`	A sweep which overlapped no command beacon
	`...`	Advertise and listen until a command arrives. An advertisement which comes due while a sweep holds the radio waits and is sent once the sweep ends, rather than being skipped
←	`+WSCNIND:-30,0,6,"F8:F0:05:AB:CD:EF","CMD3240138012398502495190247597"`	A command has arrived from the tracker asset
←	`+WSCNDONE:1`	Sweep completed, 1 result found
→	`AT+WBCN=0`	Stop advertising: the beacon configuration is locked while a run is in progress, so the response cannot be loaded until the advertising run has ended
←	`OK`	The request is accepted; the run is not over yet
←	`+WBCNDONE:75`	The advertising run is over: all 75 beacons were confirmed transmitted, some delayed while a scan sweep held the radio
→	`AT+WBCNC=2,"RSP76200451188203944710293847 56"`	Set the beacon SSID to the encoded response payload
←	`OK`	
→	`AT+WBCNC=8,10`	Send the response 10 times: the tracker only listens for part of each of its scan intervals, so a single transmission is easily missed

 ©2023-2026

←	`OK`	
→	`AT+WBCN=1`	Transmit the beacons carrying the response
←	`OK`	
←	`+WBCNDONE:10`	The response run is over: all 10 beacons were transmitted
→	`AT+WBCNC=2,"ASSET35"`	Restore the asset identifier
←	`OK`	
→	`AT+WBCNC=8,0`	Advertise continuously again
←	`OK`	
→	`AT+WBCN=1`	Resume advertising, the periodic scan having listened for the next command throughout
←	`OK`	

Example Command Sequence 107: Tracker asset: a complete exchange with a target asset

→	`AT+WSTAC=1,"MyAP"`	Specify SSID of the Access Point to report findings over
←	`OK`	
→	`AT+WSTAC=2,3`	Set security type to WPA2-Personal
←	`OK`	
→	`AT+WSTAC=3,"MyAPPSK"`	Set WPA2 Personal passphrase for connecting to the Access Point
←	`OK`	
→	`AT+WSTA=1`	Connect
←	`OK`	
←	`+WSTALU:1,"00:01:02:03:04:05",6`	Link up: connected on channel 6 to an AP with BSSID 00:01:02:03:04:05
←	`+WSTAAIP:1,"192.168.0.20"`	IP address received via auto configuration (DHCP/SLAAC)
→	`AT+WSCNPC=1,6`	Set the periodic scan channel to 6
←	`OK`	
→	`AT+WSCNPC=3,500`	Listen for 500 ms in each scan slot
←	`OK`	
→	`AT+WSCNPC=4,2`	Two slots per sweep, so each sweep listens on the channel for a total of 1000 ms
←	`OK`	
→	`AT+WSCNPC=12,5000`	Start a sweep every 5000 ms
←	`OK`	
→	`AT+WSCNPC=6,-45`	Discard anything weaker than -45 dBm: a target asset that distant could not hear a command either
←	`OK`	
→	`AT+WSCNPC=10,"ASSET*"`	Add a filter for the advertisements of target assets
←	`OK`	
→	`AT+WSCNPC=10,"RSP*"`	Add a second filter for the responses they send back. Both filters must be in place before scanning starts, because the scan configuration is locked while it runs
←	`OK`	

→	`AT+WSCNP=1`	Enable periodic scanning, the first sweep starts immediately
←	`OK`	
←	`+WSCNDONE:0`	A sweep which found no target asset
	`...`	Sweep every 5000 ms until a target asset comes into range
←	`+WSCNIND:-30,0,6,"F8:F0:05:12:34:56","ASSET35"`	Target asset ASSET35 is in range
←	`+WSCNDONE:1`	Sweep completed, 1 result found
→	`AT+WBCNC=1,6`	Set the beacon channel to 6, the channel the target asset listens on
←	`OK`	
→	`AT+WBCNC=2,"CMD3240138012398502495190247597"`	Set the beacon SSID to the encrypted command payload
←	`OK`	
→	`AT+WBCNC=7,1000`	Repeat the command every 1000 ms
←	`OK`	
→	`AT+WBCNC=8,0`	Repeat until told to stop, rather than a set number of times: the target only listens for part of each of its scan intervals, so a single transmission is easily missed
←	`OK`	
→	`AT+WBCN=1`	Start transmitting the command. Beaconing and periodic scanning run together: a beacon which comes due while a sweep holds the radio waits and is sent once the sweep ends
←	`OK`	
←	`+WSCNIND:-30,0,6,"F8:F0:05:12:34:56","ASSET35"`	A sweep which saw the advertisement again, but no response yet
←	`+WSCNDONE:1`	Sweep completed, 1 result found
	`...`	Repeat the command, and keep sweeping, until the target asset answers
←	`+WSCNIND:-30,0,6,"F8:F0:05:12:34:56","RSP76200451188203944710293847 56"`	The target asset has answered; it stops advertising its identifier while it sends the response
←	`+WSCNDONE:1`	Sweep completed, 1 result found
→	`AT+WBCN=0`	The response has been received, so stop repeating the command
←	`OK`	The request is accepted; the run is not over yet
←	`+WBCNDONE:15`	The run is over: all 15 beacons carrying the command were confirmed transmitted, some delayed while one of this asset's own scan sweeps held the radio
→	`AT+WSCNP=0`	Stop watching for target assets
←	`OK`	
←	`+WSCNPDONE:19,0`	The run is over: 19 sweeps were started and none were skipped

 ©2023-2026

List of Examples

Module	Description
Configuration Commands	Reading all parameter values
	To read a single value
	To set an additional value
	To replace an existing value
	Range of IDs
	Simple ID/Value
	Complex ID/Value
	Using preset 0
CFG	Saving active configuration to storage
	Restoring configuration from storage
	Storing a partial configuration
DNS	A record query
	NS record query
	CNAME record query
	SOA record query
	PTR record query
	MX record query
	TXT record query
	AAAA record query
	SRV record query
	DS record query
	NSEC record query
	DNSKEY record query
	ANY record query
	A record query With DNSSEC awareness
	NSEC3PARAM (unknown type) record query
	NSEC3 (unknown type) response to invalid A record query
	Multicast A record query
	MDNS-SD of local SSH services
FS	File transfer using XModem+CRC with +FS
	List and delete a file
	Get filesystem information
	File load using FS-TSFR protocol
	File store using FS-TSFR protocol
MQTT	MQTT connection directly over TLS
	MQTT connection over TLS socket
	Subscribe to MQTT topic and receive data (asynchronously)
	Subscribe to MQTT topic and receive data (polled)

OTA	Internal OTA HTTP Configuration
	Internal OTA HTTPS Configuration
	Internal OTA
	NVM OTA
SOCKET	Opening a TCP server socket
	Basic socket read
	In-band socket read
	Simple asynchronous socket read
	Reliable asynchronous socket read
	Set TLS configuration 1 for TLS client
	Open a TLS client socket
	Set TLS configuration 2 for TLS server
	Open a TLS server socket
	Reading partial datagram
TLS	Peer certificate digest pinning with +TLSC
WSCN	Scan for Access Points and filter by security type
	Setting SSID filter with +WSCNC
	Setting SSID prefix filters with +WSCNC
	Periodic passive scanning with an SSID prefix filter
	Periodic passive scanning with an SSID prefix filter
WSTA	WPA Personal: Configure STA connection parameters
	Connect at Wi-Fi and IP layers
	Some disconnections, reconnections and roaming
	+WSTA refused while a scan or periodic
	WPA Enterprise: Configure a TLS context for establishing a tunnel
	WPA Enterprise: Configure STA connection parameters for MSCHAPv2
	WPA Enterprise: Configure a TLS context for EAP-PEAPv1/TLS
	WPA Enterprise: Configure STA connection parameters for EAP-PEAPv1/TLS
DI	Query device information with +DI
EXTCRYPTO	Example of TLS configuration for secure signing via external crypto operations
	Example of signing request/response using +EXTCRYPTO
WIFI	Settings with +WIFIC
	+WIFIC coexistence settings for 3-wire PTA interface
	Configure vendor-specific information elements for transmission
	Configure vendor-specific information element reception
NVM	Simple example of NVM usage
DFU	Example of using +DFUADR and +DFUSEQ
PPS	Configuring and enabling PPS
ARB	Example of using +ARB
HTTP	HTTPS download
	HTTPS download from TCP socket
	File download to local filesystem

 ©2023-2026

HTTPGET with path and query/fragments

HTTP basic authentication using additional request headers

WBCN Target asset: advertise its location

Tracker asset: send a command to a target asset

Target asset: a complete exchange with a tracker asset

Tracker asset: a complete exchange with a target asset

Index

A

ARB (Command), 318
ASSOC (AEC), 253
ASSOC (Command), 252

B

BOOT (AEC), 26

C

CFG (Command), 31
CFGCP (Command), 34

D

DFUADR (Command), 299
DFUSEQ (Command), 300
DHCPSC (Command), 41
DI (Command), 269
DNSC (Command), 46
DNSERR (AEC), 53
DNSRESOLV (AEC), 50
DNSRESOLV (Command), 49

E

EXTCRYPTO (AEC), 274
EXTCRYPTO (Command), 273

F

FS (Command), 58
FSTSFR (Command), 62
FSUP (AEC), 64

G

GMI (Command), 27
GMM (Command), 28
GMR (Command), 29

H

HTTPBODYEND (AEC), 333
HTTPBODYRX (AEC), 332
HTTPC (Command), 324
HTTPCL (AEC), 334
HTTPCL (Command), 329
HTTPERR (AEC), 335
HTTPGET (Command), 327
HTTPRD (Command), 328
HTTPRSPCODE (AEC), 330
HTTPRSPFIELD (AEC), 331

I

IPR (Command), 30

M

MQTTC (Command), 80
MQTTCONN (AEC), 100
MQTTCONN (Command), 82
MQTTCONNACK (AEC), 101
MQTTDISCONN (Command), 91
MQTTLWT (Command), 90
MQTTPROPRX (AEC), 109
MQTTPROPRX (Command), 95
MQTTPROPTX (Command), 92
MQTTPROPTXS (Command), 98
MQTTPUB (Command), 88
MQTTPUBACK (AEC), 103
MQTTPUBCOMP (AEC), 104
MQTTPUBERR (AEC), 105
MQTTPUBSENT (AEC), 110
MQTTSUB (AEC), 106
MQTTSUB (Command), 83
MQTTSUBLST (Command), 84
MQTTSUBRD (Command), 85
MQTTSUBRX (AEC), 108
MQTTSUBRXERR (AEC), 111
MQTTUNSUB (AEC), 107
MQTTUNSUB (Command), 87

N

NETIFC (Command), 116
NETIFRX (AEC), 120
NETIFTX (Command), 119
NVMC (Command), 286
NVMCHK (AEC), 293
NVMCHK (Command), 290
NVMER (AEC), 292
NVMER (Command), 288
NVMERR (AEC), 294
NVMRD (Command), 291
NVMWR (Command), 289

 ©2023-2026

O

OTA (AEC), 132
OTAACT (Command), 130
OTAC (Command), 126
OTADL (Command), 128
OTAERR (AEC), 133
OTAINV (Command), 131
OTAVFY (Command), 129

P

PING (AEC), 138
PING (Command), 137
PINGERR (AEC), 139
PPS (AEC), 311
PPS (Command), 310
PPSC (Command), 308

R

RST (Command), 141

S

SI (Command), 255
SIHEAP (AEC), 259
SIHERR (AEC), 260
SIMERR (AEC), 257
SIMSTAT (AEC), 256
SISTACK (AEC), 261
SIWPKTS (AEC), 258
SNTPC (Command), 143
SNTPERR (AEC), 145
SOCKBL (Command), 153
SOCKBM (Command), 155
SOCKBR (Command), 154
SOCKC (Command), 166
SOCKCL (AEC), 171
SOCKCL (Command), 164
SOCKERR (AEC), 173
SOCKIND (AEC), 168
SOCKLST (Command), 165
SOCKO (Command), 152
SOCKRD (Command), 160
SOCKRDBUF (Command), 162
SOCKRXT (AEC), 169
SOCKRXU (AEC), 170
SOCKTLS (AEC), 172
SOCKTLS (Command), 156
SOCKWR (Command), 157
SOCKWRTO (Command), 158
SYSLOG (AEC), 316
SYSLOGC (Command), 314

T

TIME (AEC), 180
TIME (Command), 178
TLSC (Command), 193
TLSCSC (Command), 198
TLSERR (AEC), 200

W

WAP (Command), 205
WAPAIP (AEC), 206
WAPC (Command), 203
WAPERR (AEC), 209
WAPSC (AEC), 207
WAPSD (AEC), 208
WBCN (Command), 352
WBCNC (Command), 349
WBCNDONE (AEC), 353
WBCNERR (AEC), 354
WIFIC (Command), 278
WIFIVSIERX (AEC), 282
WPROV (Command), 265
WPROVAT (AEC), 266
WPROVC (Command), 263
WPROVDT (AEC), 267
WSCN (Command), 215
WSCNC (Command), 211
WSCNDONE (AEC), 222
WSCNIND (AEC), 221
WSCNP (Command), 220
WSCNPC (Command), 216
WSCNPDONE (AEC), 223
WSCNPERR (AEC), 224
WSTA (Command), 241
WSTAAIP (AEC), 242
WSTAC (Command), 238
WSTAERR (AEC), 244
WSTALD (AEC), 243
WSTALU (AEC), 245
WSTAROAM (AEC), 246